AN INTELLIGENT SHELF-LIFE FOOD ESTIMATION SYSTEM

FRESCO (A Fruit Freshness and Shelf-Life Estimation Platform)

BY

SHADAMORO JEREMIAH

MATRIC NUMBER: AUL/CMP/22/082

OLOFU POSSIBLE

MATRIC NUMBER: AUL/IFT/22/009

Supervised by:

Dr. Babalola Asegunloluwa

A Project submitted to the Department of Computing in partial fulfillment of the requirements for the award of Bachelor of Science (B.Sc.) Degree in Computer Science

JUNE, 2026

DECLARATION

We hereby declare that this Project was written by us and is a correct record of our research work. It has not been presented in any previous application for any degree of this or any other University. All citations and sources of information have been duly acknowledged.

…… ……

SHADAMORO JEREMIAH **Date**

…… ……

OLOFU POSSIBLE **Date**

CERTIFICATION

I hereby certify that this project work entitled **"AN INTELLIGENT SHELF-LIFE FOOD ESTIMATION SYSTEM — FRESCO: A FRUIT FRESHNESS AND SHELF-LIFE ESTIMATION PLATFORM"** was carried out by **SHADAMORO JEREMIAH** (matric number AUL/CMP/22/082) and **OLOFU POSSIBLE** (matric number AUL/IFT/22/009), under the supervision of **Dr. Babalola Asegunloluwa**, and has not been submitted in part or in full to this university or any other institution for the award of a degree.

…… ……

Dr. Babalola Asegunloluwa **Date**

Supervisor

…… ……

DR. Deborah Aleburu **Date**

Head of Department, Computing

ABSTRACT

Food waste at the consumer level is a substantial and largely preventable problem, with the Food and Agriculture Organization estimating that roughly one third of all food produced for human consumption is lost or wasted each year. A significant share of this waste arises not because food has genuinely spoiled, but because the people handling it lack reliable information about its actual condition. This is especially true for fresh fruit sold without expiration dates the dominant pattern in Nigerian markets where consumers rely on sensory inspection that food-science research has shown to be subjective, inconsistent, and poor at detecting early-stage spoilage. This project presents Fresco, a mobile-first Progressive Web Application that addresses this gap using only the camera of a standard smartphone. The system integrates three validated technologies into a single deployable product: a MobileNetV2 convolutional neural network fine-tuned through transfer learning for fruit identification and visual freshness assessment; a fuzzy-logic-inspired scoring module that maps a continuous freshness signal to five interpretable categories; and a multi-factor shelf-life estimation engine that combines a curated database of 42 fruit varieties with user-selected storage conditions and an ethylene compatibility checker grounded in post-harvest biology. The system is implemented with a FastAPI back end, an asynchronous SQLAlchemy data layer, and a Next.js front end delivered as an installable Progressive Web App. The classifier achieves 89.3% top-1 accuracy on a held-out test set across its 14 age-stage classes, the shelf-life estimator produces differentiated and defensible day-counts across fruit varieties and storage methods, and the ethylene compatibility engine — to our knowledge the first consumer-facing implementation of this knowledge — correctly flags pairwise storage conflicts in real time. Fresco demonstrates that intelligent food quality systems built on consumer-grade hardware can meaningfully address the information gap that drives household food waste, accepting a lower accuracy ceiling than sensor-based systems in exchange for the universal accessibility that produces real adoption.

DEDICATION

This work is dedicated first and foremost to God Almighty, whose grace, wisdom, and strength carried us through every stage of this project. Without His blessings, none of this would have been possible.

We also dedicate this work to ourselves — Jeremiah Shadamoro and Possible Olofu — in recognition of the long hours, the late nights, the missed weekends, and the shared determination it took to bring this idea from a rough sketch in a notebook to a working system. We learned more from this project than from any other single piece of our undergraduate years.

To our parents, Mr & Mrs Shadamoro Lawrence and Mr & Mrs Olofu, this work is for you. Your love, your sacrifices, your prayers, and your unshaken belief in what we could become are the foundation on which everything in this report rests. You taught us the discipline of finishing what we start, and the importance of doing it well. We hope this small offering makes you proud.

This work is for all of you — with love and gratitude.

ACKNOWLEDGEMENTS

Our deepest thanks go to God Almighty, the source of every good idea we ever had and every moment of clarity that arrived when we most needed it. He sustained us through the parts of this project that we did not know how to finish.

We are profoundly grateful to our supervisor, Dr. Babalola Asegunloluwa, for his patience, his clear technical guidance, and the willingness to push us when our thinking was not yet rigorous enough. The shape of this project, both as a piece of engineering and as a piece of writing, owes a great deal to his input.

We thank the entire faculty and staff of the Department of Computing, Anchor University Lagos, whose teaching across four years built the foundation that this project is built upon. The breadth of what we learned — from data structures and algorithms to databases, software engineering, and the principles of machine learning — is visible on every page of this report.

Our heartfelt thanks to our families — Mr & Mrs Shadamoro Lawrence and Mr & Mrs Olofu — whose support, both material and emotional, made the conditions of this work possible. We hope what we have produced reflects well on the homes you built for us.

To our friends and colleagues at Anchor University who shared the work and the worry of the final year with us — thank you for the conversations, the reviews, the late-night debugging sessions, and the encouragement that kept us going.

Finally, we thank each other. A two-author final-year project is only as good as the partnership behind it, and ours held together through every difficulty the year presented. We are proud of what we built, and we are proud to share the credit for it.

TABLE OF CONTENTS

Right-click and select 'Update Field' to populate the Table of Contents.

LIST OF FIGURES

Right-click and select 'Update Field' to populate the List of Figures once captions have been added to the body of the report.

AN INTELLIGENT SHELF-LIFE FOOD ESTIMATION SYSTEM

FRESCO A Fruit Freshness and Shelf-Life Estimation Platform

BY

SHADAMORO JEREMIAH & OLOFU POSSIBLE

AUL/CMP/22/082 & AUL/IFT/22/009

Supervisor: Dr. Babalola Asegunloluwa

Department of Computing, Faculty of Science

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Food waste is one of the defining sustainability challenges of the twenty-first century. According to the Food and Agriculture Organization of the United Nations, approximately 1.3 billion tonnes of food produced for human consumption is lost or wasted every year a figure that represents roughly one-third of all food produced globally (FAO, 2019). The economic cost of this waste has been estimated at approximately 940 billion United States dollars annually, while its environmental consequences greenhouse gas emissions from decomposing organic matter, wasted water, and inefficient land use are equally severe (Gustavsson et al., 2011). What makes this problem particularly troubling is that a significant share of it is entirely preventable. A substantial portion of consumer-level food waste does not arise because food has genuinely spoiled; it arises because the people handling that food lack reliable information about its actual condition.

The primary tool consumers use to assess food safety is the printed expiration date the "Best Before," "Use By," or "Sell By" label found on most packaged food products. These dates are determined by food manufacturers using accelerated aging tests conducted under controlled laboratory conditions, where products are stored at idealised, constant temperatures designed to represent the worst-case storage environment (Leena, 2024). The manufacturer prints a date that assumes these ideal conditions will persist throughout the entire supply chain journey. The implicit assumption is that the product will travel from factory to consumer at a steady, appropriate temperature, handled gently at every stage. This assumption is almost never true in practice.

In reality, food passes through a complex and often poorly controlled sequence of environments before it reaches a consumer's refrigerator. A crate of mangoes, for instance, may

be harvested at peak ripeness in a tropical climate, transported in a lorry without refrigeration for several hours, stored in a warehouse at inconsistent temperatures, displayed in a retail store under warm lighting, and then carried home in a heated car before being placed in a domestic refrigerator. At each of these stages, temperature fluctuations, humidity variations, and physical handling affect the rate at which the fruit deteriorates. The printed label, however, reflects none of this accumulated thermal history. Two pieces of fruit with identical labels may be in dramatically different states of freshness, yet the consumer has no way of knowing this (Wang et al., 2024). One of the most significant consequences of this information gap is that consumers who observe a date that has passed will often discard food that is still safe and nutritious. Conversely, food that has been mishandled in the supply chain may still carry a valid label while already posing a safety risk.

The problem is compounded further when it comes to fresh, unpackaged fruit the specific category this project addresses. Unlike processed and packaged products, most fresh fruits are sold with no expiration date at all. The consumer must rely entirely on physical inspection: examining the surface for discolouration, bruising, or mould; pressing the fruit to assess softness; or smelling it for off-odours. Research in food science has consistently demonstrated that these sensory methods are unreliable indicators of actual quality. By the time visible mould appears on a strawberry, for example, the internal fungal contamination may have been present for twenty-four to forty-eight hours (Tournas, 2005). By the time a banana shows the level of browning that causes most consumers to discard it, it may actually be at peak sweetness for baking. Conversely, a fruit that appears visually perfect may have been stored at room temperature for three days longer than is advisable, increasing microbial risk despite an unblemished appearance.

It is within this context that intelligent, technology-driven approaches to food freshness assessment have emerged as a research priority. Advances in deep learning particularly Convolutional Neural Networks trained on large visual datasets have made it possible to assess the condition of food from a photograph with a level of consistency and objectivity that manual inspection cannot match (Fu et al., 2022). At the same time, the widespread adoption of smartphones with high-resolution cameras has created a platform through which such technology can be delivered to ordinary consumers at essentially zero additional hardware cost. A system that can analyse a camera image of a fruit, determine what variety it is, assess its visual freshness,

and estimate how many days it will remain edible under a given storage condition would address a genuine, documented gap in consumer food safety information.

This project **Fresco** is the practical realisation of that opportunity. It is an intelligent, mobile-first Progressive Web Application that integrates three validated technologies into a unified consumer tool: MobileNetV2 transfer learning for fruit classification (Sandler et al., 2018), a fuzzy-logic-inspired freshness scoring module for visual quality assessment (Vasquez et al., 2024), and a rule-based shelf-life estimation engine that applies established food science data to produce personalised, condition-based predictions. The system additionally incorporates an ethylene compatibility checker a feature grounded in post-harvest biology that warns users when the fruits they are storing together will cause each other to deteriorate faster through natural gas interactions (Watkins, 2006).

1.2 Statement of the Research Problem

Despite the severity and economic scale of food waste attributable to poor freshness assessment, the tools available to ordinary consumers remain inadequate. Three distinct but interrelated problems define the research gap this project addresses:

Problem 1: The absence of dynamic, condition-based freshness information for fresh fruits

Printed expiration dates, where they exist, are calculated under the assumption of idealised, uninterrupted refrigeration conditions that the food science literature has repeatedly shown to be unrealistic outside controlled laboratory environments (Leena, 2024; Luan et al., 2025). For the majority of fresh fruits sold at retail bananas, mangoes, avocados, pawpaws, guavas, and most tropical varieties no date is provided at all. The consumer must rely on sensory inspection, which is subjective, inconsistent, and poor at detecting early-stage spoilage before it poses a health risk. Zhang et al. (2023) identified through global sensitivity analysis that temperature variation alone accounts for the majority of prediction error in shelf-life models, confirming that static, temperature-blind dates are fundamentally unsuited to the task of guiding consumer decisions. What is needed is a system that assesses the fruit's current actual condition, not one that references a date calculated weeks or months ago.

Problem 2: Consumer ignorance of ethylene-mediated storage interactions

Ethylene (C2H4) is a naturally occurring plant hormone and gaseous compound that plays a central role in fruit ripening. Certain fruits including bananas, apples, mangoes, and avocados produce ethylene in substantial quantities as they ripen, while others are highly sensitive to ethylene exposure and ripen or deteriorate significantly faster when stored in its presence (Watkins, 2006; Saltveit, 1999). Storing bananas and strawberries together in the same bowl, for example, can reduce the effective shelf life of the strawberries by two to three days. These interactions are well-documented in post-harvest biology and are routinely managed at the industrial level in commercial cold storage facilities. However, no consumer-facing application currently communicates this information at the point of storage decision. The consequence is that households routinely accelerate the spoilage of their fruit through uninformed storage combinations, contributing unnecessarily to food waste.

Problem 3: The inaccessibility of existing precision freshness assessment systems

The academic literature contains a number of highly accurate solutions to food freshness estimation. Wang et al. (2025) demonstrated 94.5% prediction accuracy using a multi-sensor IoT system combining gas detection and temperature monitoring. Patel et al. (2024) demonstrated the ability to detect internal bruising and measure sugar concentration using hyperspectral imaging. These systems are genuinely impressive, but they require specialised hardware sensor arrays, hyperspectral cameras, Raspberry Pi computing nodes that costs hundreds or thousands of dollars and is entirely impractical for household use. There is a meaningful and unaddressed gap between what the research literature has demonstrated to be technically possible and what is actually accessible to the consumer who is trying to decide, in their kitchen at eight o'clock in the evening, whether a mango is still good to eat.

1.3 Aim of the Study

The aim of this study is to design, develop, and implement Fresco an intelligent, mobile-first web application that uses transfer learning-based computer vision and fuzzy-logic-inspired freshness reasoning to estimate the remaining shelf life of fresh fruit from a smartphone camera image, identify ethylene-based storage compatibility conflicts between fruit varieties, and deliver personalised storage recommendations without requiring any hardware beyond the smartphone the user already owns.

1.4 Objectives of the Study

To achieve the stated aim, this study pursues four specific, measurable objectives:

Objective 1: To develop an automated fruit identification module using MobileNetV2 transfer learning (Sandler et al., 2018) that correctly classifies the fruit variety from a smartphone camera image without requiring manual input from the user, targeting a minimum top-1 classification accuracy of 70% across the 41 fruit varieties supported by the system.

Objective 2: To implement a fuzzy-logic-inspired visual freshness assessment module that assigns a continuous freshness score on a 0.0-to-1.0 scale following the fuzzy membership function principles validated by Vasquez et al. (2024) and maps this score to five overlapping, human-readable freshness categories: Fresh, Slightly Aged, Aging, Old, and Spoiled.

Objective 3: To build a shelf-life estimation and decision engine that combines the classified fruit identity, the freshness score, and the user-specified storage method to calculate personalised days-remaining estimates using the formula: Estimated Days Remaining = Base Shelf Life × Freshness Score, drawing base values from a curated database of 41 fruit varieties containing shelf-life data for room temperature, refrigerator, and freezer conditions, as well as ethylene production and sensitivity flags per the post-harvest biology literature (Watkins, 2006).

Objective 4: To deliver the complete integrated system as a Progressive Web Application (PWA) accessible from any modern mobile browser without an app store download, incorporating user authentication, a personal fruit inventory with expiry tracking and alert notifications, and an ethylene compatibility checker that warns users about storage combinations that accelerate spoilage.

1.5 Research Methodology

This research follows a structured design-and-implementation methodology. Rather than conducting laboratory experiments on physical food samples, the project uses validated public datasets and transfer learning to develop predictive models, which are then integrated into a fully deployable software system. The methodology proceeds through four phases:

Phase 1 Literature Review and Dataset Identification: A systematic review of academic literature on computer vision for food quality assessment, fuzzy logic for reasoning under uncertainty, transfer learning for lightweight model deployment, and post-harvest biology of fruit ethylene interactions. Concurrent identification and preparation of training datasets: the Food-101 dataset (Bossard et al., 2014), the Fruits-360 dataset (Muresan & Oltean, 2018), and the Kaggle Fresh and Stale Fruits and Vegetables dataset for freshness model training.

Phase 2 System Architecture Design: Design of a decoupled client-server architecture comprising a Python FastAPI backend with asynchronous database access via SQLAlchemy and a MySQL production database, and a Next.js Progressive Web App frontend. Design of the seven-step scan pipeline: (1) image capture via camera or file upload, (2) fruit classification via MobileNetV2, (3) freshness scoring via the trained CNN, (4) database lookup for base shelf-life values and ethylene flags, (5) shelf-life calculation adjusted by freshness score, (6) ethylene compatibility evaluation against the user's stored inventory, and (7) verdict generation with actionable recommendation.

Phase 3 Implementation: Training the MobileNetV2 fruit classification model using transfer learning on Google Colab with GPU acceleration. Implementation of the freshness scoring module with fuzzy-logic-inspired categorical mapping. Full-stack development of the FastAPI

backend, Next.js PWA frontend, user authentication system, inventory tracking module, and ethylene compatibility checker.

Phase 4 Testing and Evaluation: Functional testing of all pipeline components individually and end-to-end. Evaluation of classification accuracy on a held-out test set across all 41 supported fruit varieties. Validation of shelf-life estimates against established food science reference data. User experience assessment of the mobile interface under realistic conditions.

1.6 Scope of the Study

This study focuses exclusively on fresh fruit specifically, 41 varieties chosen to represent the range of fruits commonly available in Nigerian markets and households, from indigenous tropical varieties (pawpaw, guava, pineapple, African star apple) to widely imported ones (apple, orange, grape, kiwi). The decision to restrict scope to fruit, rather than attempting to cover all perishable food categories, was made on the basis of three considered justifications.

First, fruit spoilage is primarily visual in a way that many other food categories are not. The progression from fresh to overripe to spoiled in most fruits involves characteristic and well-documented changes in surface colour, texture, and moisture content that are detectable in photographic images (Fu et al., 2022). The same cannot be said reliably for raw meat, dairy products, or cooked foods, where dangerous bacterial contamination may develop with few or no visible indicators. A camera-based system is therefore significantly more defensible for fruit than for many other food categories.

Second, the ethylene interaction problem described in the research problem statement is specific to fruits (and certain vegetables) and represents a concrete, consumer-accessible feature that no existing tool provides. Including this feature meaningfully differentiates the system from prior work while being directly supported by well-established post-harvest biology (Watkins, 2006; Saltveit, 1999).

Third, focusing on a well-defined category allows thorough validation within the timeframe and resources of an undergraduate final-year project. The database of 41 fruits with curated shelf-life values, ethylene flags, and storage recommendations can be compiled and

verified with confidence, whereas attempting to cover all perishable food types would risk producing surface-level coverage that could not be adequately validated.

The system uses a smartphone camera as its only physical input device. No IoT sensors, temperature probes, humidity gauges, or gas detection equipment are used. Storage condition information is provided by the user through a three-option selector (Room Temperature, Refrigerator, Freezer), and the shelf-life engine applies established food science reference values accordingly. The integration of real-time environmental sensing is identified as a recommendation for future development.

1.7 Significance of the Study

Consumer impact: Fresco equips households with a practical, immediate tool for reducing food waste. The ethylene compatibility checker alone addresses a specific, widespread consumer behaviour mixed fruit storage that contributes directly to preventable waste. Research on consumer food waste behaviour has consistently found that the provision of better freshness information changes storage and consumption decisions in measurable ways (Principato et al., 2015; Quested et al., 2011).

Environmental and economic impact: The FAO estimates that if food waste were a country, it would be the third-largest emitter of greenhouse gases in the world (FAO, 2019). Reducing waste at the consumer level where, in developed economies, a majority of avoidable food waste occurs requires better information at the point of decision. Tools that extend the effective use of food by even one or two days through better-informed storage can meaningfully contribute to this reduction.

Academic contribution: This project demonstrates the practical integration of MobileNetV2 transfer learning (Sandler et al., 2018), fuzzy-logic-inspired continuous freshness reasoning (Vasquez et al., 2024), and ethylene-aware shelf-life estimation into a single, deployable consumer application specifically addressing the integration gap identified consistently across the reviewed literature. It contributes a replicable architectural template for intelligent food quality systems built on mobile-accessible platforms.

1.8 Definition of Terms

Shelf Life	The period during which a fruit remains safe and organoleptically acceptable retaining appropriate taste, texture, colour, and nutritional content under specified storage conditions.
Dynamic Shelf Life	A shelf-life estimate that reflects the fruit's actual current condition rather than a fixed date determined at the point of manufacture. Changes as the fruit's observable and measurable state changes.
Freshness Score	A continuous numerical value between 0.0 (completely spoiled) and 1.0 (perfectly fresh) representing the assessed visual condition of a fruit, produced by the CNN freshness assessment module.
MobileNetV2	A lightweight Convolutional Neural Network architecture developed by Sandler et al. (2018) at Google, designed for efficient image classification on mobile and embedded devices. Uses inverted residual blocks to reduce computational cost while maintaining accuracy.
Transfer Learning	A machine learning technique in which a model previously trained on a large general dataset (ImageNet) is adapted for a more specific task (fruit classification) by replacing and retraining only its final layers, significantly reducing the data and time required.
Fuzzy Logic	A mathematical reasoning framework that allows partial membership in categories a fruit can be simultaneously '70% Fresh and 30% Slightly Aged' enabling gradual, realistic transitions between states rather than sharp boundaries (Zadeh, 1965).
Ethylene	A naturally occurring gaseous plant hormone (C2H4) produced by certain fruits during ripening. Fruits vary in their rate of ethylene production and their sensitivity to its presence; storing high-producing fruits near sensitive ones accelerates the ripening and deterioration of the sensitive fruit (Watkins, 2006).
Progressive Web App (PWA)	A web application built with modern browser technologies that can be installed on a smartphone's home screen and accessed offline, providing a native app-like experience without requiring distribution through an app store.
Convolutional Neural Network (CNN)	A class of deep learning model designed for processing visual data. Layers of learned filters detect progressively complex visual features edges, textures, shapes, and ultimately object identities enabling tasks such as image classification and freshness assessment.
Decision Engine	The system component that integrates the fruit classification result, freshness score, storage method selection, base shelf-life data, and ethylene flags to produce the final freshness verdict, days-remaining estimate, and storage recommendation.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

The challenge of accurately estimating how long perishable food will remain safe and acceptable to eat has occupied food scientists, supply chain engineers, and more recently, computer scientists, for several decades. The intellectual journey through this field reveals a consistent pattern: each generation of solutions correctly identifies the limitations of the methods it replaces, then introduces a new approach that is more precise in controlled conditions but encounters its own set of difficulties when deployed in the complex and variable environments of real-world food systems. Understanding this trajectory is essential for situating the design decisions made in this project and demonstrating that those decisions are grounded in the existing state of knowledge rather than made arbitrarily.

This chapter organises the relevant literature into four areas. Section 2.2 examines traditional approaches to shelf-life determination printed dates, empirical rules, and sensory inspection and analyses why each of these methods consistently fails consumers. Section 2.3 reviews the mathematical and kinetic modelling tradition, including the Arrhenius equation and Time-Temperature Integration models, which represent the dominant scientific approach to shelf-life prediction for much of the twentieth century. Section 2.4 discusses the more recent application of machine learning and computer vision to this problem, including the specific architecture MobileNetV2 adopted in this project and the reasoning for that choice. Section 2.5 reviews sensor-driven and IoT-based monitoring systems, explaining why a camera-only approach was chosen over sensor integration despite the higher accuracy that sensors can provide. Section 2.6 examines fuzzy logic as a reasoning framework and explains its role in the freshness assessment module. Section 2.7 provides a structured summary of the key related works reviewed. Sections 2.8 and 2.9 synthesise the review into specific, numbered research gaps and draw the conclusions that motivated this project's design.

2.2 Traditional Food Shelf-Life Determination Methods

2.2.1 Static Expiration and Best-Before Dates

The printed expiration date is the dominant mechanism through which the food industry communicates quality information to consumers. These dates are determined by manufacturers through a process known as accelerated shelf-life testing (ASLT), in which products are stored at elevated temperatures to speed up deterioration and enable faster determination of spoilage thresholds. The resulting date is then back-calculated to represent the expected quality limit under normal, ideal storage conditions typically continuous refrigeration at 4°C for perishable items (Labuza & Fu, 1993).

The systemic limitation of this approach is embedded in its foundational assumption. Leena (2024) characterised static labels as "deterministic errors in a stochastic supply chain" a precise and damning description. The label applies a single fixed value to a process that is inherently variable. Once a product leaves the manufacturing facility, it enters a chain of handling, transport, and storage events that the manufacturer cannot observe or control. Temperature loggers used in commercial cold chain monitoring regularly detect excursions of 5-10°C above target temperature during loading and unloading operations alone (Ndraha et al., 2018). Each of these events accelerates spoilage by an amount that the printed date has no mechanism to reflect.

The consequences of this disconnect are empirically documented. Principato et al. (2015) found, in a study of Italian households, that date labelling was the primary driver of food disposal decisions, with a majority of respondents reporting that they discarded food upon or before the stated date regardless of visible condition. Quested et al. (2011) similarly found in the United Kingdom that "date related" reasons accounted for a significant proportion of avoidable household food waste. The food safety literature also documents the reverse problem: food recalled due to contamination frequently involves products within their stated date range, indicating that the label provides false assurance as well as false urgency (Rediers et al., 2009).

Relevance to Fresco:

The Fresco system replaces the static date with a dynamic, condition-based assessment that evaluates the fruit's current visual state at the moment of scanning, rather than referencing a date determined weeks or months earlier under hypothetical conditions.

2.2.2 Empirical Rule-Based Approaches

Prior to the widespread availability of computational tools, food distributors and retailers managed inventory using simple empirical rules heuristics developed through experience and adjusted for seasonal and regional conditions. A typical rule might specify that leafy greens are to be discarded after three days in refrigeration, or that bananas at a particular stage of browning should be moved to a discount display. These rules are easy to implement and communicate, requiring no technical expertise, and they embody accumulated practical knowledge about typical spoilage patterns.

However, these rules assume that spoilage proceeds at a predictable, linear rate an assumption that food science has consistently shown to be incorrect. Bacterial and fungal growth in food is typically exponential, not linear, and is highly sensitive to temperature variations (Jay et al., 2005). A rule calibrated for winter temperatures in a well-refrigerated storage room will systematically underestimate spoilage in a warm retail environment during summer. Zhang et al. (2023) demonstrated through global sensitivity analysis that temperature variation alone can produce dramatically different spoilage rates for the same product, confirming that any rule that does not explicitly account for thermal history will produce errors of practically significant magnitude.

A further problem with rule-based approaches is their inability to account for initial quality variance. Two batches of strawberries purchased on the same day may be in substantially different conditions if one was harvested three days before the other. A day-count rule treats both identically; a condition-based system would correctly assign different remaining shelf lives.

Relevance to Fresco:

The Fresco freshness scoring module directly addresses initial quality variance by assessing the fruit's current observable condition rather than applying a time-based rule. A mango scored at 0.9 freshness will receive more estimated days remaining than one scored at 0.5, even if both were purchased on the same date.

2.2.3 Sensory-Based Inspection

The fallback for both consumers and retail quality control personnel when other information is absent is sensory inspection: examining food visually for discolouration, mould, or structural changes; assessing texture through touch; and detecting off-odours through smell. This approach is universally accessible and requires no tools or training it relies on human sensory capabilities that are intuitive and immediate.

The limitations of sensory inspection are, however, substantial and well-documented. Tournas (2005) demonstrated that visible mould growth on soft fruits typically indicates fungal contamination that has been developing internally for one to two days before surface signs appear, meaning that consumers who rely on visual inspection may regularly consume fruits with elevated microbial loads. The detection threshold for spoilage odours varies significantly between individuals, with research showing that trained quality assessors perform considerably better than untrained consumers (Dhuique-Mayer et al., 2019). Perhaps most significantly, sensory inspection is reactive: it can detect spoilage that has already occurred, but it cannot predict when spoilage will occur or estimate how many days of safe consumption remain. It provides no forward-looking information.

Relevance to Fresco:

The Fresco system addresses both the objectivity and the predictive failures of sensory inspection. The freshness scoring module provides a consistent, individual-independent assessment of current visual condition, while the shelf-life engine translates that assessment into a forward-looking estimate of days remaining moving from reactive detection to proactive prediction.

2.3 Mathematical and Fuzzy Logic for Shelf-Life Estimation

2.3.1 The Arrhenius Framework

The most scientifically rigorous approach to shelf-life prediction that preceded the machine learning era is grounded in chemical kinetics specifically, the application of the Arrhenius equation to model the rate of food quality degradation as a function of temperature. The Arrhenius equation, originally formulated by Svante Arrhenius in 1889 to describe the temperature dependence of chemical reaction rates, has been applied in food science since the 1970s as the theoretical foundation for predictive microbiology and quality modelling (Labuza & Schmidl, 1985).

The equation models the spoilage rate constant as an exponential function of temperature, parameterised by an activation energy term specific to each food-spoilage mechanism. Phupaichitkun et al. (2022) demonstrated its successful application to predict the shelf life of dried coconut products under accelerated temperature conditions, confirming that when environmental conditions are stable and precisely characterised, Arrhenius-based models can produce accurate predictions. The model has been validated across a wide range of food categories including dairy, bakery products, and fresh produce in controlled settings.

The critical limitation is the model's sensitivity to input precision. Because the spoilage rate constant is an exponential function of temperature, small errors in temperature measurement produce disproportionately large errors in predicted shelf life. Vasquez et al. (2024) demonstrated empirically that when temperature data contains the level of noise typical of consumer-grade sensors rather than the precision instruments used in laboratory settings Arrhenius model accuracy degrades substantially. This sensitivity makes the model fundamentally ill-suited to consumer-facing applications where input data quality cannot be controlled. Furthermore, Arrhenius modelling requires precisely characterised activation energy parameters for each specific food type and variety, parameters that must be determined through

laboratory testing and that cannot be assumed to generalise between closely related varieties (Luan et al., 2025).

2.3.2 Time-Temperature Integration Models

Recognising the limitation of single-point temperature models, food scientists developed Time-Temperature Integration (TTI) models that track the cumulative thermal history of a product the total heat load experienced over its entire storage journey rather than applying a single assumed temperature. This approach acknowledges that a brief temperature excursion, such as two hours at 25°C during unloading, may reduce shelf life more significantly than a full day of storage at the optimal 4°C (Ndraha et al., 2018).

Wang et al. (2024) developed a dynamic TTI model for leafy vegetables that achieved prediction errors of less than 5% under fluctuating temperature conditions a remarkable performance level that represents a genuine advance over static Arrhenius approaches. This accuracy, however, was achieved using high-precision sensors and required extensive product-specific parameter calibration. The authors themselves acknowledged that performance would likely degrade when using consumer-grade hardware with typical measurement noise.

The deeper limitation of TTI models for consumer applications is that they require continuous, product-attached temperature monitoring a requirement that implies either the attachment of a sensor device to each individual piece of fruit (impractical and expensive) or access to supply chain temperature logs that consumers do not receive. Without thermal history data, TTI models cannot function.

Relevance to Fresco:

Because Fresco is designed for consumer use without specialist hardware, kinetic models requiring continuous temperature data are not applicable to this context. The system captures the most accessible proxy for storage condition the user's storage method choice and uses this to select appropriate base values from a curated shelf-life database, applying the freshness score to adjust for the fruit's current observed condition.

2.4 Machine Learning and Computer Vision Approaches

2.4.1 The Shift to Data-Driven Freshness Assessment

The application of machine learning to food quality assessment represents a fundamental shift in methodology: from models that require precisely specified physical parameters derived from laboratory experiments, to models that learn quality indicators directly from data. This shift is significant because it replaces the brittle dependence on precise sensor input with a more flexible ability to identify quality-relevant patterns in visual information patterns that are robust to the kind of variability characteristic of real-world consumer settings.

The earliest machine learning applications to this domain used feature-engineered approaches: manually extracting colour histograms, texture descriptors, and shape metrics from images, then training traditional classifiers on these features (ElMasry et al., 2012). These methods required significant domain expertise to engineer appropriate features and were sensitive to variations in lighting, camera angle, and image resolution. Deep learning and specifically Convolutional Neural Networks superseded this approach by learning feature representations directly from the raw image data, eliminating the manual feature engineering bottleneck.

2.4.2 Convolutional Neural Networks for Freshness Assessment

Convolutional Neural Networks (CNNs) are a class of deep learning architecture specifically designed to process visual data. Their defining characteristic is the use of convolutional layers that learn spatial filters initially detecting edges and colour gradients in early layers, progressively combining these into textures and shapes in intermediate layers, and ultimately representing complex objects in deep layers (LeCun et al., 1998). This hierarchical feature learning makes CNNs particularly well-suited to food quality assessment, where visual

indicators of freshness surface colour transitions, texture degradation, bruising patterns, and moisture loss indicators are precisely the kinds of spatial patterns that convolutional filters can learn.

Fu et al. (2022) conducted one of the most comprehensive studies of CNN-based fruit freshness assessment, developing a system using YOLO and ResNet architectures that classified fruit freshness on a continuous scale from 0 to 100% rather than using binary fresh-or-spoiled categories. This continuous scoring approach is important because it enables the nuanced shelf-life estimation that a binary system cannot: a fruit at 80% freshness receives a meaningfully different prediction than one at 45% freshness, even though both would be classified as "fresh" in a binary system. Fu et al. demonstrated that their model detected early-stage quality changes subtle colour shifts and texture changes that human inspectors typically did not notice until two to three days later. The main limitation identified was a significant accuracy drop in low-light conditions, a practical concern for consumer applications where lighting cannot be controlled.

Yamamoto et al. (2020) extended this work to the specific problem of surface defect detection in citrus fruits using a lightweight CNN architecture, demonstrating that relatively compact models ones that could run on modest hardware could still achieve detection accuracy sufficient for practical quality grading. Their work established that the computational demands of CNN-based freshness assessment need not prohibit consumer deployment.

2.4.3 MobileNetV2: Architecture and Selection Rationale

The fruit classification component of the Fresco system is built on MobileNetV2, a CNN architecture introduced by Sandler et al. (2018) at Google. MobileNetV2 was designed specifically to achieve competitive image classification accuracy while dramatically reducing the computational requirements for inference, making it suitable for deployment on mobile devices and embedded systems. Understanding the key architectural innovations of MobileNetV2 is important both for situating this choice within the literature and for being able to explain the model's behaviour.

The first innovation is the use of Depthwise Separable Convolutions, introduced in the original MobileNet (Howard et al., 2017). A standard convolutional layer simultaneously learns spatial patterns (the arrangement of features across the image) and channel combinations (how different feature maps relate to each other). Depthwise Separable Convolutions decompose this

single expensive operation into two cheap ones: a depthwise convolution that applies a single filter to each input channel independently (learning spatial patterns), followed by a pointwise convolution that combines the results across channels (learning channel relationships). This decomposition reduces computational cost by a factor of approximately 8 to 9 compared to a standard convolution of equivalent size, with only a modest reduction in expressive power.

The second innovation and the contribution specific to MobileNetV2 is the Inverted Residual Block. Standard residual connections (He et al., 2016) connect wide, high-dimensional representations with shortcut paths that skip computation. MobileNetV2 inverts this structure: each block begins by expanding the input from a narrow (low-channel-count) representation to a wide one using a 1×1 pointwise convolution, applies the depthwise convolution in this wide space to learn spatial patterns efficiently, then compresses back to a narrow representation. The shortcut connection is applied between the narrow input and narrow output rather than between the wide intermediate representations. This design ensures that the computationally cheap narrow representations are reused as much as possible while the heavy spatial computation occurs only in the expanded space where it is most informative.

Hu et al. (2024) validated the mobile deployment of similar architectures through their optimised EfficientNet-B0 work, demonstrating inference times compatible with real-time use on consumer smartphones. Their work confirmed that mobile-optimised CNN architectures can deliver practically useful freshness assessment performance on the hardware consumers already own, directly supporting the feasibility claim underlying the Fresco design.

Transfer learning is the technique through which MobileNetV2's general visual understanding is adapted for the specific task of fruit classification. The model is pre-trained on ImageNet a dataset of 1.28 million images spanning 1,000 categories giving it learned representations that capture a wide range of visual features: surface textures, curvature, colour gradients, and shape contours. When fine-tuned on a fruit-specific dataset, the lower convolutional layers (which contain the general visual feature detectors) are frozen, while the upper layers and the classification head are retrained on fruit images. This approach allows a fruit classification model to be trained with dramatically fewer images than would be needed to train from scratch (Tajbakhsh et al., 2016), making it feasible to build an accurate classifier from the publicly available fruit datasets within the scope of this project.

2.4.4 Machine Learning for Prediction: Strengths and Transparency Limitations

Beyond image analysis, machine learning regression models have been applied to shelf-life prediction from tabular data historical records of storage conditions, initial quality measurements, and observed spoilage times. Kumari (2025) demonstrated that a Random Forest regressor achieved 96% accuracy predicting dairy spoilage from historical data, significantly outperforming traditional linear regression. The ability of ensemble methods like Random Forest to capture non-linear, high-order interactions between variables represents a genuine advantage over kinetic models that assume simplified mathematical relationships between temperature and spoilage rate.

The critical limitation of Random Forest and similar ensemble methods for consumer-facing applications is their opacity. A Random Forest model produces a prediction from the aggregated output of hundreds of decision trees; it is not possible to read out a simple, interpretable rule that explains any individual prediction. Kumari (2025) acknowledged this limitation, noting that the black-box nature of ensemble methods makes them difficult to validate and explain to end users whose decisions may depend on understanding the basis for a prediction. This concern is amplified in food safety contexts, where a consumer who is told their fruit has two days remaining but cannot understand why the system reached that conclusion may reasonably distrust the advice.

Relevance to Fresco:

The Fresco decision engine prioritises interpretability. When the system recommends storing a banana separately from strawberries, it explains why: "Bananas produce high levels of ethylene gas, which will significantly accelerate the ripening and spoilage of nearby ethylene-sensitive fruits including strawberries." This transparency addresses the trust limitation identified in the machine learning literature while still providing data-driven, accurate recommendations.

2.5 Sensor-Driven and IoT-Based Monitoring Systems

The past decade has seen substantial research investment in Internet of Things (IoT) approaches to food quality monitoring systems that continuously measure environmental conditions using physical sensors and use this data to dynamically update shelf-life estimates. These systems represent, in principle, the most direct implementation of the dynamic shelf-life concept: rather than inferring storage conditions from a user's answer to a question, they measure them directly and continuously.

Wang et al. (2025) developed a multi-sensor fusion system combining electronic nose gas sensors detecting the ethylene, ammonia, and volatile organic compounds produced by decomposing fruit with temperature monitors, achieving a prediction accuracy of 94.5%. The system's strength lies in its use of multiple independent sensing modalities: when gas sensor readings and temperature readings are consistent with each other, confidence in the estimate is high; when they diverge, the system can flag the discrepancy for closer inspection. This cross-modal validation is not possible with a single-sensor design.

Sahu et al. (2020) focused on the edge computing aspect of IoT food monitoring, demonstrating a system that processed sensor data locally on a Raspberry Pi device rather than transmitting it to a remote server. This approach enabled offline operation critical in environments with unreliable internet connectivity and reduced response latency. The limitation was that the constrained computational power of the Raspberry Pi restricted the complexity of the machine learning models that could be deployed.

Patel et al. (2024) explored a more specialised sensing technology: hyperspectral imaging, which captures light across hundreds of narrow spectral bands beyond the visible range. Hyperspectral cameras can detect biochemical changes chlorophyll breakdown, sugar accumulation, water loss that are invisible to conventional RGB cameras, enabling detection of internal bruising and predicting sugar content non-destructively. Accuracy levels of 95%+ were

demonstrated for internal quality assessment. However, the hardware cost of hyperspectral cameras typically USD 20,000 or more for research-grade systems makes this technology entirely impractical for consumer use.

Defraeye et al. (2025) took a supply-chain-level perspective, developing digital twin simulations of thermal conditions in commercial fruit shipping containers. By maintaining complete thermal history records for each shipment, the system could implement First-Expired-First-Out (FEFO) logistics routing products with the least remaining shelf life to the nearest markets achieving a 15% reduction in waste across the shipping operations studied. This work demonstrates the potential of condition-based logistics at the industrial scale, though it is explicitly designed for supply chain operators rather than consumers.

Why Fresco uses a camera-only design:

The sensor-based literature establishes clearly that greater accuracy is achievable with sensor data than with camera images alone. However, every reviewed sensor-based system requires hardware that consumers do not own and would not be willing to purchase gas sensors, IoT nodes, temperature probes, or hyperspectral cameras. Hu et al. (2024) noted that the consumer barrier to entry for sensor-based systems is a fundamental obstacle to adoption, regardless of accuracy. Fresco's design accepts the accuracy trade-off in exchange for universal accessibility: the camera that every smartphone already has is the only required hardware. The goal is a system that is adopted, used, and therefore actually reduces food waste not one that achieves higher accuracy in a laboratory while remaining inaccessible to the households that waste the most food.

2.6 Fuzzy Logic for Freshness Reasoning Under Uncertainty

2.6.1 Foundations of Fuzzy Logic in Food Science

Fuzzy logic, introduced by Lotfi Zadeh in 1965 as a mathematical framework for reasoning with imprecision, offers an alternative to both rigid mathematical models and opaque machine learning approaches for freshness assessment. Classical binary logic requires that a statement be either true or false: a fruit is either "fresh" or "not fresh." Fuzzy logic allows partial truth: a fruit can be "80% fresh and 20% slightly aged" simultaneously, with membership in each

category determined by a continuous function of observable input values (Zadeh, 1965). This capacity for graduated, overlapping categories maps more naturally onto the real phenomenon of fruit quality deterioration, which is a continuous process rather than a threshold event.

The practical relevance of fuzzy logic to this project lies in its robustness to input uncertainty. Vasquez et al. (2024) conducted a rigorous empirical comparison between Arrhenius-based kinetic models and Fuzzy Inference Systems for shelf-life estimation, using sensor data with realistic noise levels. Their key finding was that the Arrhenius model's accuracy degraded substantially as input noise increased a consequence of the exponential sensitivity described in Section 2.3 while the fuzzy inference system maintained significantly higher accuracy under the same conditions. This robustness to noisy input is precisely what is required for a consumer application in which images are captured under varying lighting, at varying distances, and from varying angles.

2.6.2 Application of Fuzzy Logic to Food Quality Systems

Wibowo et al. (2022) provide a detailed and practically grounded example of fuzzy logic application to food shelf-life estimation, developing a Fuzzy Inference System for bakery products that integrated moisture content, storage temperature, and elapsed time into a unified freshness model. Their system used IF-THEN rules expressed in natural language for example: "IF moisture is HIGH AND temperature is WARM, THEN shelf life is SHORT" making the reasoning process transparent and comprehensible to food professionals without mathematical expertise. This interpretability is an important practical advantage: food safety inspectors, retail managers, and consumers can all understand and therefore trust a system whose logic they can read.

Wibowo et al. identified two limitations in their implementation that are directly relevant to this project. First, the fuzzy rule base was static it was designed by human experts and could not update itself from new data. Second, the rules were specific to bread; adapting them to other food types required repeating the expert elicitation and rule design process from scratch. Both limitations are addressed in Fresco's approach: the fuzzy-inspired category mapping is implemented as a parameterised scoring function that applies consistently across all 41 supported fruit varieties, with the base shelf-life values that modulate the estimate being drawn from a curated database rather than hard-coded into rules

2.6.3 Fuzzy Logic in the Fresco Freshness Module

The Fresco freshness assessment system is explicitly designed on fuzzy logic principles, though implemented in a form that balances the theoretical elegance of a full Fuzzy Inference System with the practical constraints of a mobile-deployed student project. The CNN freshness model outputs a continuous score between 0.0 and 1.0. This score is mapped to five overlapping freshness categories using membership-function-style boundaries that allow gradual transitions between states:

Score Range	Fuzzy Category	Colour Signal	System Action
0.80 – 1.00	Fresh	Green (#4A7C59)	Full base shelf life; standard storage advice
0.60 – 0.79	Slightly Aged	Amber (#B8860B)	Reduced estimate; 'eat soon' recommendation
0.40 – 0.59	Aging	Orange (#D97706)	Significantly reduced; 'use within 1-2 days'
0.20 – 0.39	Old	Red (#DC2626)	Minimal remaining; 'consume immediately'
0.00 – 0.19	Spoiled	Dark Red (#8B3A3A)	Zero days; 'discard' recommendation

The shelf-life calculation formula applies the freshness score as a multiplier against the base shelf life for the user-selected storage method:

$$\textit{Estimated Days Remaining} = \textit{Base Shelf Life (days)} \times \textit{Freshness Score}$$

This formula encodes the intuitive principle that a fruit at peak freshness (score = 1.0) retains its full expected shelf life under the given storage conditions, while a fruit that is 50% deteriorated (score = 0.5) retains only half. The base shelf life values are sourced from

established food science references including the USDA post-harvest handling guidelines and the post-harvest biology literature (Kader, 2002), providing a scientifically grounded foundation for the dynamic calculation. The approach handles the inherent uncertainty in visual assessment without requiring the precise parameter calibration that kinetic models demand (Vasquez et al., 2024).

2.7 Summary of Related Works

The fifteen studies and authoritative sources reviewed in this chapter trace a clear evolutionary path through the field of food shelf-life assessment, and each one contributes a specific insight that has shaped Fresco's design. Rather than enumerate them in a tabular format that obscures the relationships between them, this summary connects them as a narrative.

The foundational layer of the literature establishes the scale of the problem. The Food and Agriculture Organization's 2019 report established the headline statistic 1.3 billion tonnes of food wasted annually, roughly one-third of global production and motivates the entire enterprise of consumer-facing freshness tools. This is the moral and economic argument for the work, but it is also a statistical report that proposes no technological intervention. The work of Leena (2024) builds on that scale by characterising the central failure of the current information regime: static expiration dates are, in her phrase, "deterministic errors in a stochastic supply chain." Her systematic review of prediction models demonstrates that the printed date a consumer reads in a supermarket has no mechanism to reflect what has actually happened to the food on its journey from factory to shelf. Leena's contribution is theoretical and diagnostic; it identifies the problem precisely without proposing a system to solve it.

The kinetic modelling tradition responded to this diagnosis with mathematically rigorous predictions of spoilage rate as a function of temperature, drawing on the Arrhenius equation that food scientists have applied since the 1970s. Phupaichitkun et al. (2022) demonstrated Arrhenius modelling's successful application to dried coconut products under controlled conditions, while Wang et al. (2024) developed a more sophisticated Time-Temperature Integration model for leafy vegetables that achieved prediction errors of under 5% under fluctuating temperatures. The accuracy of these models in laboratory conditions is genuinely impressive, but both authors

acknowledge and Vasquez et al. (2024) empirically demonstrate that this accuracy degrades substantially as input data becomes noisier. Consumer-grade sensors do not produce laboratory-grade measurements, and a kinetic model whose inputs are noisy produces predictions whose error grows exponentially. Vasquez and colleagues compared Arrhenius models directly against fuzzy inference systems under realistic noise conditions and showed that fuzzy systems maintain their accuracy where kinetic models lose it, establishing the empirical case for the fuzzy-logic-inspired reasoning that Fresco's freshness module adopts.

The computer vision strand of the literature represents the most direct technical foundation for Fresco. Sandler et al. (2018) introduced the MobileNetV2 architecture that Fresco uses for fruit classification, demonstrating that depthwise separable convolutions and inverted residual blocks could achieve near-ResNet accuracy at roughly an eighth of the computational cost. Their paper makes the mobile deployment of deep learning practical; without MobileNetV2 or a similar mobile-optimised architecture, the entire premise of a camera-only consumer freshness tool collapses. Fu et al. (2022) extended this foundation by demonstrating that a CNN-based fruit grading system could produce a continuous freshness score from zero to one hundred percent, rather than a binary fresh-or-spoiled classification. This continuous-scoring approach is the conceptual basis for Fresco's 0.0-to-1.0 freshness scale and the five overlapping freshness categories it maps to. Fu and colleagues also identified a practical limitation accuracy drops in poor lighting which is precisely the failure mode that careful image preprocessing in Fresco's pipeline is designed to mitigate. Hu et al. (2024) validated the mobile deployment of related architectures on consumer smartphones, confirming that inference times compatible with real-time use are achievable on the hardware consumers already own. Yamamoto et al. (2020) made a complementary contribution, showing that lightweight CNN models could achieve adequate accuracy for citrus defect detection on modest hardware, further establishing that the computational demands of CNN-based freshness assessment need not prohibit consumer deployment.

A separate strand of the literature explored machine learning for prediction from tabular data rather than from images. Kumari (2025) demonstrated that a Random Forest regressor could achieve 96% accuracy predicting dairy spoilage from historical records of storage conditions and initial quality measurements. The accuracy is genuinely better than what camera-only approaches

can offer. But Kumari also acknowledges the central practical limitation of ensemble methods for consumer-facing applications: their opacity. A consumer who is told their fruit has two days remaining, without being able to read why, may reasonably distrust the advice. This finding is what motivated the deliberate prioritisation of interpretability in Fresco's three-tier response design.

The sensor-driven and IoT literature represents the technical state of the art in dynamic freshness assessment. Wang et al. (2025) developed a multi-sensor fusion system combining electronic nose gas detection with temperature monitoring, achieving 94.5% prediction accuracy through cross-modal validation. Sahu et al. (2020) demonstrated edge computing for food monitoring on Raspberry Pi nodes, enabling offline operation in environments with unreliable connectivity. Patel et al. (2024) explored hyperspectral imaging, which captures light across hundreds of narrow spectral bands and can detect biochemical changes invisible to RGB cameras sugar content, internal bruising, chlorophyll breakdown at accuracy levels of 95% or more. Defraeye et al. (2025) applied digital twin simulation to supply-chain logistics, demonstrating 15% waste reduction through First-Expired-First-Out routing at the commercial scale. The common thread across all four of these contributions is technical impressiveness coupled with consumer inaccessibility. Each one requires hardware that ordinary consumers do not own and would not be willing to purchase gas sensor arrays, IoT computing nodes, hyperspectral cameras costing tens of thousands of dollars, or supply-chain-level temperature logging infrastructure. The literature consistently identifies the consumer hardware barrier as the central obstacle to adoption, but does not resolve it. Fresco's design accepts the accuracy trade-off implied by a camera-only input in exchange for the universal accessibility that produces actual adoption and therefore actual waste reduction.

The fuzzy logic literature provides the reasoning framework for handling the inherent uncertainty in visual assessment. Zadeh's 1965 introduction of fuzzy sets established the mathematical foundation: the idea that membership in a category can be partial rather than binary, allowing a fruit to be "80% fresh and 20% slightly aged" simultaneously rather than belonging crisply to one category or the other. This capacity for graduated, overlapping categories maps more naturally onto fruit ripening, which is a continuous process rather than a threshold event. Wibowo et al. (2022) provided a practically grounded application of fuzzy inference to bakery

product shelf-life prediction, demonstrating both the interpretability advantages of natural-language IF-THEN rules and the limitations of expert-designed rule bases that cannot adapt to new food types. Vasquez et al. (2024) extended the case for fuzzy reasoning by demonstrating empirically that fuzzy systems outperform Arrhenius models under realistic input noise. Fresco's freshness module synthesises these contributions: the CNN produces a continuous freshness score, and a fuzzy-logic-inspired category mapping translates that score into the five human-readable categories that the user interface displays.

The post-harvest biology literature, drawn primarily from Watkins (2006) and Saltveit (1999), provides the basis for the ethylene compatibility feature. Watkins's comprehensive review of ethylene production and sensitivity across fruit and vegetable species is the source of the producer and sensitive flags stored against each fruit in Fresco's database. Saltveit's earlier work on the effect of ethylene on the quality of fresh fruits and vegetables provides the qualitative justification for the warning messages the system generates. Neither author proposes a consumer-facing application; the contribution of these sources is biological knowledge that Fresco operationalises in software.

Two further sources contribute to the supporting infrastructure of the project rather than to its core arguments. Bossard et al. (2014) introduced the Food-101 dataset that has become a standard benchmark for food image classification, providing 101,000 labelled images across 101 categories and validating the basic premise that food images can be classified at scale by deep networks. He et al. (2016) introduced the ResNet architecture whose residual connection concept underlies the inverted residual blocks of MobileNetV2; Howard et al. (2017) introduced the original MobileNets paper whose depthwise separable convolutions MobileNetV2 builds upon. These three contributions are background rather than direct foundations for Fresco, but they appear in the citation graph of nearly every paper this review draws on.

Taken together, these works trace an evolutionary path from static expiration dates through kinetic modelling and IoT-based monitoring to deep-learning-driven assessment. Each generation correctly identified the limitations of the methods it replaced. None has produced a consumer-accessible, camera-only system that integrates fruit identification, freshness assessment, fuzzy reasoning, and ethylene awareness in a single deployable application. This is the precise space Fresco was designed to occupy.

2.8 Research Gap

A synthesis of the reviewed literature reveals that the field has produced strong, validated solutions to individual components of the intelligent freshness assessment problem, but has not produced a consumer-accessible system that integrates these components into a unified, practically deployable tool. Three specific gaps are identified:

Gap 1 No integration between automated fruit identification and freshness-based shelf-life estimation. Systems that use computer vision for food classification (Fu et al., 2022; Hu et al., 2024) require the user to manually specify the food type before assessment can proceed, introducing user friction and opportunity for error. Systems that estimate shelf life using kinetic or statistical models (Wang et al., 2024; Kumari, 2025) similarly require manual specification of food type and rely on sensor data the consumer does not have. No reviewed system automatically identifies a fruit from a camera image and directly feeds that classification into a personalised shelf-life prediction. Fresco closes this gap through the integrated MobileNetV2 classification and decision engine pipeline.

Gap 2 Existing accurate systems require hardware consumers do not own. Multi-sensor IoT systems (Wang et al., 2025), hyperspectral imaging systems (Patel et al., 2024), and temperature-logging digital twin infrastructure (Defraeye et al., 2025) all deliver accuracy levels that would be highly useful to consumers, but require hardware purchases ranging from hundreds to tens of thousands of dollars. The literature consistently identifies this as a barrier to adoption but does not resolve it. Fresco resolves this gap by using only the smartphone camera as input, accepting the accuracy trade-off in exchange for the accessibility that enables real-world adoption and therefore actual waste reduction.

Gap 3 No consumer tool combines vision-based freshness assessment, fuzzy reasoning, ethylene awareness, and inventory tracking in a single deployable application. The individual components image classification (Sandler et al., 2018; Fu et al., 2022), continuous freshness scoring (Fu et al., 2022), fuzzy-logic-inspired reasoning under uncertainty (Vasquez et al., 2024), and ethylene interaction knowledge (Watkins, 2006) have each been separately

validated in the literature. What does not exist is a consumer-facing application that integrates all of them. The ethylene compatibility feature is of particular note: the post-harvest biology literature on ethylene interactions between fruits is well-established and practically important, but no reviewed consumer-facing digital tool makes use of this knowledge. Fresco closes this gap by delivering all four components in a unified Progressive Web Application.

2.9 Chapter Summary

Three principal conclusions emerge from this review of the literature that directly shaped the design of the Fresco system:

First, traditional freshness assessment methods are fundamentally inadequate for the consumer context. Static expiration dates cannot reflect post-manufacture storage conditions (Leena, 2024); empirical rules cannot account for temperature sensitivity and initial quality variance (Zhang et al., 2023); and sensory inspection is too subjective and too reactive to provide reliable, forward-looking freshness estimates (Tournas, 2005; Fu et al., 2022). The gap between the information consumers need and the information these methods provide is well-documented, substantial, and not diminishing.

Second, computer vision and fuzzy-logic-inspired reasoning are, in combination, the most appropriate technical approach for consumer-facing mobile freshness assessment. CNNs learn visual freshness indicators from image data in a way that is consistent, objective, and applicable to consumer hardware (Fu et al., 2022; Hu et al., 2024). MobileNetV2 provides the optimal accuracy-efficiency trade-off for mobile deployment (Sandler et al., 2018). Fuzzy logic principles provide a robustness to uncertain and variable input data that rigid mathematical models cannot match (Vasquez et al., 2024), while maintaining the interpretability that consumer trust requires (Wibowo et al., 2022).

Third, the research gap is one of integration and accessibility, not of underlying technical feasibility. The components needed for a practical, consumer-facing intelligent freshness system have each been validated independently in the literature. What is missing is a system that combines them, delivered on hardware consumers already own, incorporating the ethylene

compatibility knowledge that the post-harvest biology literature documents but consumer tools ignore. Fresco is designed to fill this precise gap.

CHAPTER THREE

SYSTEM ANALYSIS AND DESIGN

3.0 Introduction

Last semester one of us bought a basket of tomatoes from Oyingbo Market. Maybe fifteen pieces, two thousand naira. By the next weekend half of them were soft, a few had black patches, and most of the basket ended up in the bin. A visiting friend looked at what was going into the rubbish and said, "you Lagos people, you waste too much food." She was right, but not in the way she meant. It wasn't that we wanted to waste anything. It was that nobody could really tell what was happening inside the tomatoes. You press one, it gives a little, and then the questions start. Is this normal soft or bad soft? Is that spot just a bruise, or is the inside already gone? You guess. And when in doubt, you throw it away. Most of us would rather lose a few naira than risk food poisoning.

This problem people throwing away food they cannot accurately judge happens at a scale that is hard to picture. The United Nations Food and Agriculture Organization puts global food waste at about 1.3 billion tonnes every year, roughly one third of everything produced for human consumption (FAO, 2019). In Nigeria the situation is worse for fresh fruit, because most of what we buy comes from open markets with no labels, no expiry dates, no nothing. The seller might say "buy quick, it's fresh," but after the transaction you are on your own. You take the fruit home, you guess at how long it will last, you guess again two days later, and eventually you make a decision eat or bin based on inspection that is mostly smell, sight, and instinct.

The problem is that smell and instinct are not very reliable. Food scientists have studied this carefully. By the time you can smell that a fruit has gone off, contamination has typically been developing for one to two days already. By the time visible mould appears, the fungal contamination has often spread through the inside of the fruit, not just on the surface (Tournas, 2005). The signs we use to make decisions are lagging indicators. They tell us about problems that have already happened, not about problems that are about to happen.

Fresco is our attempt to give people a better tool. It is an application that runs on a phone. You open it in your browser, you can install it on your home screen if you want, and once it is there it looks and behaves like any other app on the phone. The way you use it is simple. You hold your phone over a fruit, you tap a button, and the app takes a photo. Within a couple of seconds, you get back three pieces of information: what the fruit is, how many days you have left to eat it, and how you should store it for the longest life. If the fruit is something that interacts badly with other fruit you have already saved in your inventory, you also get a warning about that.

How does the app figure out the freshness from a photograph? This is where the artificial intelligence part comes in. When your phone sends the photo to our server, the server runs the image through a model that has been trained on thousands of photographs of fruit at different ages. The model has learned, in a statistical sense, what a fresh banana looks like, what an aging one looks like, what a fully ripe one looks like, and what one that is past its useful life looks like. When it sees your photo, it makes its best guess about which of those stages your specific fruit is in. The output is a freshness score on a scale from zero to one, where one means perfectly fresh and zero means completely spoiled.

That score then feeds into a calculation. The system stores, in its database, baseline shelf-life values for 42 different fruit varieties under three storage conditions room temperature, refrigerator, and freezer. A fresh banana, for example, lasts about five days on a counter and seven days in a fridge. The system takes that baseline, multiplies it by your fruit's freshness score (raised to a power that reflects how fruit actually decays, not in a straight line but accelerating as the fruit ages), adjusts for the storage temperature you chose, and adjusts again based on the ripeness stage. The result is a personalised estimate of how many days your specific fruit, in its specific condition, has left.

The third thing the app does is something most people do not know is even a thing. Certain fruits bananas, apples, mangoes, avocados, papayas, and others release a natural gas called ethylene as they ripen. The gas is not dangerous to people, but it acts like a ripening accelerator for other fruits nearby. Put a banana in a fruit bowl with strawberries and the strawberries will go off about two days faster than they would have alone. Commercial cold storage companies have known this for over fifty years and they separate their stock accordingly. But normal households do not know. They put everything in the same bowl. Fresco knows which fruits release ethylene

and which fruits are sensitive to it, and it warns you when you have stored incompatible fruits together.

That is the whole system, more or less. A camera, a model that judges freshness, a calculation that estimates shelf life, and a warning system for storage conflicts. Everything runs on a phone, does not need any extra hardware, and works whether you live in Lagos or Lekki or anywhere else. The reason we built it this way accepting a lower accuracy ceiling in exchange for accessibility is that there are systems in the academic literature that achieve much higher accuracy by using gas sensors, hyperspectral cameras, and IoT temperature loggers. Those systems are real, they work, and they cost a lot of money. We made a deliberate decision to trade some accuracy for the property of being usable by anyone with a smartphone, which is most people in the world.

This chapter explains how the system is designed. We follow a structure that moves from method to detail. Section 3.1 sets out the research methodology we followed the order of work, from reading the academic literature through to building working code. Section 3.2 describes the software development lifecycle we adopted. Section 3.3 walks through the frameworks and tools we chose, and explains why each one. Section 3.4 lists the system's functional and non-functional requirements. Section 3.5 looks at what already exists in this space and how Fresco is different. Section 3.6 is the visual core of the chapter it presents the system through a series of design diagrams. Section 3.7 documents the database design. Section 3.8 covers security. Section 3.9 explains the algorithms behind the AI components. Section 3.10 walks through how a real user would actually use the system. Section 3.11 gives pseudocode and code snippets for the most important algorithms. Section 3.12 closes the chapter.

If you only read this introduction and nothing else, you should now have a clear picture of what Fresco is, why it exists, how it works in broad strokes, and what makes it different from existing tools. The rest of the chapter is for the reader who wants to understand the design at the level needed to either reproduce it or critique it.

3.1 Research Methodology

The research approach we followed is a design-and-implementation one, common in software engineering projects of this type. We did not run laboratory experiments on physical fruit, and

we did not collect new field data through user surveys. The work was carried out in four phases that overlapped in time but had a clear sequence of priorities.

The first phase was literature review. We read across four areas: computer vision for food quality assessment, fuzzy logic for reasoning under uncertainty, post-harvest biology of fruit ripening and ethylene interactions, and consumer behaviour studies on food waste. The review pointed us towards specific decisions. MobileNetV2 (Sandler et al., 2018) was the appropriate convolutional neural network architecture for our deployment constraints. The continuous freshness scoring approach demonstrated by Fu et al. (2022) was a better fit than a binary classifier. The ethylene compatibility knowledge in Watkins (2006) gave us the basis for a feature that no consumer-facing tool currently provides. The full review is documented in Chapter Two.

The second phase was dataset preparation. We worked with two main sources. The Fruits-360 dataset (Mureșan & Oltean, 2018) gave us a base of clean, well-labelled fruit images. A smaller curated set of freshness-labelled fruit images filled in the ripeness-stage labels that Fruits-360 does not provide. We organised these into 14 target classes representing four fruit types across multiple age ranges, plus an "Expired" catch-all class for visually degraded specimens. The full training procedure is documented in Chapter Four.

The third phase was system architecture and module design. This is the work that this chapter documents. We made decisions about the overall system structure (a decoupled client and server), the database schema, the API contracts between front-end and back-end, the structure of the AI pipeline, the security model, and the deployment approach. Each of these decisions was made deliberately, with reference to the constraints we identified in the literature review and the practical limits of a final-year project's time and budget.

The fourth phase was implementation, integration testing, and evaluation. Implementation followed the design from this chapter and proceeded in roughly the order of the system's data flow first the database, then the back-end services, then the AI modules, then the front-end. Testing happened continuously as modules were built, and full end-to-end testing happened once all the pieces were connected. This phase is the subject of Chapter Four.

3.2 Software Development Lifecycle

We followed an iterative, agile-style development approach rather than a strict waterfall. The reason for this was practical: we were learning as we built. Some decisions that looked correct at the design stage turned out to need revision once we started writing code, and a rigid waterfall would have either forced us to commit to early mistakes or required formal change requests for every small revision.

The iteration cycle we used was approximately two weeks long. At the start of each cycle we picked a small set of features or modules to work on, designed them in enough detail to start coding, built them, tested them in isolation, and integrated them into the main branch. At the end of the cycle we reviewed what had been built against the larger system goals and adjusted the priority of the next cycle's work.

Three principles guided the work within each cycle. The first was that the back-end and the front-end should always be in a working state. We did not let either side break for more than a few hours, because a broken state made it impossible to test new code in context. The second was that every new feature should have a manual test plan written before the code was written. This forced us to think about edge cases at design time, rather than discovering them in production. The third was that the database schema, the Pydantic schemas, and the front-end TypeScript types should always be in sync. This was the place where the most painful bugs came from when we let things drift, so we built our cycles around catching the drift early.

The version control system was Git, with the repository hosted privately on GitHub. We used short-lived feature branches for any change larger than a small bug fix, and the main branch was kept in a state that would build and run at any moment. Pull requests were reviewed by the other team member before merge. This is a small team's version of code review, and it caught a meaningful number of small mistakes that would otherwise have made it into the main branch.

3.3 Framework and Development Approach

The tools and frameworks that we settled on are the result of weighing options against the constraints of the project mobile-first delivery, the need to integrate a machine learning model,

the requirement that the system be usable offline, and the practical reality that we are two final-year students with limited time. Each major tool choice is explained below, with the alternatives that we considered and why we rejected them.

3.3.1 Next.js for the Frontend

The front-end is built on Next.js (version 16.2.1), which is a framework that sits on top of React and adds opinionated structure to a React application. We chose it over a plain React setup, over Vue, and over native mobile frameworks like React Native or Flutter.

The reasons were specific. Next.js gives us file-system-based routing through its App Router, which means the URL structure of the application is literally the directory structure of the source code. A folder called `scan` becomes the `/scan` route, a folder called `inventory` becomes the `/inventory` route. There is no separate router configuration file to keep in sync. For a team of two, removing one whole category of "my routes are wrong" debugging was worth the slight learning curve of getting used to the App Router conventions.

The second reason is Progressive Web App support. Next.js has first-class support for PWA features manifest files, service workers, install prompts built into its conventions. Building a PWA in plain React is possible but requires assembling several pieces by hand. Next.js comes with most of those pieces already configured.

The third reason is TypeScript. We wrote the entire front-end in TypeScript, and Next.js has the best TypeScript ergonomics of the frameworks we considered. The integration between the framework, the IDE, and the type checker is mature in a way that catches errors at compile time, before the code ever runs in a browser. For a project where the front-end and back-end evolve in parallel, this matters: a TypeScript-level mismatch between what the front-end expects and what the back-end returns becomes a build error, not a runtime crash that the user discovers.

React Native and Flutter were rejected because both would require maintaining two separate codebases one for Android, one for iOS or accepting the compromises of write-once-run-anywhere frameworks that often produce a non-native feel on at least one platform. A PWA delivers an installable, app-like experience to both platforms from a single codebase, which fits the project's resources better.

3.3.2 FastAPI for the Backend

The back-end is built in Python using FastAPI. We considered Flask and Django as the obvious alternatives. The reason we picked FastAPI comes down to three specific features.

First, FastAPI has native asynchronous request handling through Python's `asyncio` event loop. This matters because the slowest operation in our system the model inference for a scan runs in a thread pool while the event loop continues serving other requests. In Flask, achieving the same concurrency requires gunicorn workers or external libraries; in FastAPI, it is the default behaviour.

Second, FastAPI automatically generates OpenAPI documentation from the Python type annotations on the route handlers. When we add a new endpoint, the API documentation at `/docs` updates itself without any extra work. For a project where the front-end developers and the back-end developers are the same two people sitting in the same room, this might sound unnecessary, but it turned out to matter a lot. When we wrote front-end code that called the back-end, we used the OpenAPI documentation as the contract reference. When the contract changed on the back-end side, the documentation showed the change, and the corresponding front-end code was updated to match.

Third, FastAPI uses Pydantic for request and response validation. A request body that does not match the declared schema is rejected at the boundary, before it ever reaches the business logic, with a precise error message indicating which field failed which validation rule. This means our route handlers never have to check whether their inputs are valid by the time the handler runs, validation has already happened. This is a small ergonomic win that adds up across the dozens of endpoints in the system.

Python as the language was effectively forced by the need to load and run a TensorFlow Keras model. Python is the primary language of the deep learning ecosystem, and rewriting the inference path in a different language would either require an inter-process call out to a Python service (defeating the purpose) or accepting the limited support for deep learning in alternative languages.

3.3.3 SQLAlchemy with Multi-Backend Database Support

The data layer uses SQLAlchemy 2.0 with its asynchronous extension. SQLAlchemy is a Python ORM that abstracts the differences between database backends behind a single API. We chose it over writing raw SQL or using a lighter ORM like Tortoise, because SQLAlchemy's maturity and ecosystem outweigh its slight learning curve.

The interesting choice within SQLAlchemy was the multi-backend strategy. In development we use MySQL through the `aiomysql` driver, which is what most team members had installed locally from earlier coursework. In production we use PostgreSQL through `asyncpg`, which is what Render's managed database service provides on its free tier. For local testing and zero-configuration prototyping, we also support SQLite through `aiosqlite`. The same application code runs against all three. This was made possible by a small URL-normalisation function that translates the connection string Render gives us (`postgres://...`) into the form SQLAlchemy expects for async drivers (`postgresql+asyncpg://...`).

3.3.4 TensorFlow and Keras for the Model

The classifier itself was built with TensorFlow and the Keras high-level API. The MobileNetV2 architecture is available as a pre-trained Keras application, which makes transfer learning straightforward we loaded the ImageNet-pretrained base, froze the convolutional layers, added a small classification head, and trained that head on our fruit dataset. The training was done on Google Colab's T4 GPU runtime, which gave us enough compute to fine-tune the model in a reasonable amount of time without needing dedicated hardware.

The trained model is serialised to a `.keras` file and shipped with the back-end. At application startup it is loaded into memory once, warmed up with a dummy prediction, and then held for the lifetime of the process. We discuss this in more detail in Section 3.9 and in Chapter Four.

3.3.5 Tailwind CSS for Styling

All styling on the front-end is done with Tailwind CSS v4 utility classes. We did not use a component library like Material UI, shadcn/ui, or Chakra UI. The 28 components in the application are hand-built. This was a deliberate choice. Component libraries make it fast to assemble a working UI, but they produce a look that is recognisable as "that thing built with

shadcn" not because the library is bad, but because everybody who uses it ends up with similar-looking screens. We wanted Fresco to feel like a coherent product designed for its purpose, not a stack of pre-made components.

Tailwind itself was chosen over plain CSS modules or styled-components because its utility-first model produces small, predictable class names that read like the design they describe. A button styled as `bg-emerald-600 text-white px-4 py-2 rounded-lg shadow-sm` tells you exactly what it looks like from reading the markup. After about two weeks of writing Tailwind, this becomes faster than writing equivalent CSS in a separate file.

3.3.6 Other Tools

A few smaller pieces complete the stack. Pydantic v2 handles validation and settings management on the back-end. `passlib` with bcrypt and Python's standard `hashlib` handle password hashing (the two-stage scheme is explained in Section 3.8). `python-jose` handles JWT issuance and verification. Pillow handles image loading and preprocessing on the back-end. Uvicorn is the ASGI server that runs FastAPI in production. The development environment is VS Code with the standard Python, Pylance, ESLint, Prettier, and Tailwind CSS IntelliSense extensions. Git is hosted on GitHub. The intended production deployment is Render for the back-end and Vercel for the front-end.

3.4 System Requirements

The requirements that follow define what the system has to do and what properties the system has to have. We split them into functional requirements the specific things the system is required to do and non-functional requirements the qualitative properties the system has to satisfy, like performance, security, and usability.

3.4.1 Functional Requirements

The functional requirements were derived from the four project objectives stated in Section 1.4, broken down into specific user-facing capabilities. They are grouped here by area.

User Account and Authentication

ID	Requirement
FR-1	The system shall allow a new user to register with a username, email address, and password.
FR-2	The system shall enforce that usernames are unique, between 3 and 50 characters, and contain only letters, digits, and underscores.
FR-3	The system shall enforce that passwords are at least 6 characters in length and at most 128 characters.
FR-4	The system shall allow a registered user to log in with their email address and password.
FR-5	The system shall issue a JSON Web Token (JWT) on successful authentication and require this token for all protected endpoints.
FR-6	The system shall expire JWT tokens after 30 minutes and require re-authentication.
FR-7	The system shall return an identical error message for an unknown email and a wrong password, to prevent account enumeration attacks.

Fruit Scanning and Identification

ID	Requirement
FR-8	The system shall accept image uploads of common formats (JPEG, PNG, WebP) up to 10 megabytes in size.
FR-9	The system shall use the device camera where available, with automatic fallback to file upload where the camera is unavailable.
FR-10	The system shall identify the fruit variety from the uploaded image using a trained convolutional neural network.
FR-11	The system shall assess the freshness of the identified fruit and produce a continuous score on the 0.0-to-1.0 scale.
FR-12	The system shall map the freshness score to one of five categorical labels: Fresh, Slightly Aged, Aging, Old, or Spoiled.

ID	Requirement
FR-13	The system shall return the scan result within 30 seconds, aborting the request if the back-end exceeds this limit.

Shelf-Life Estimation

ID	Requirement
FR-14	The system shall maintain a reference database of 42 fruit varieties with baseline shelf-life values for room temperature, refrigerator, and freezer storage.
FR-15	The system shall calculate the estimated days remaining for the scanned fruit by combining its baseline shelf life with the assessed freshness score, the user-selected storage method, and the ripeness stage.
FR-16	The system shall present shelf-life estimates for all three storage methods and recommend the method that gives the longest remaining life.
FR-17	The system shall produce a human-readable recommendation string for each scan, tailored to the verdict status (Fresh, Eat Soon, Eat Today, or Spoiled).

Inventory Tracking

ID	Requirement
FR-18	The system shall allow an authenticated user to add a scanned fruit to a personal inventory.
FR-19	The system shall list the user's inventory, sortable by expiry date, name, freshness, or date added, and filterable by consumption and expiry status.
FR-20	The system shall allow the user to update the storage method of an inventory item and automatically recalculate the projected expiry date.
FR-21	The system shall allow the user to mark an inventory item as consumed.
FR-22	The system shall allow the user to delete an inventory item, removing any associated notifications by cascade.
FR-23	The system shall schedule a notification one day before the projected expiry of each inventory item,

ID	Requirement
	where the item has more than one day of remaining life.

Ethylene Compatibility

ID	Requirement
FR-24	The system shall maintain ethylene producer and ethylene sensitive flags for each fruit in the reference database.
FR-25	The system shall warn the user when a scanned fruit produces or is sensitive to ethylene.
FR-26	The system shall accept a list of fruit names and return the conflicting pairs and their grouped storage recommendations.
FR-27	The system shall generate a human-readable explanation for each ethylene conflict, naming the producer, the sensitive party, and the expected consequence.

Progressive Web App Delivery

ID	Requirement
FR-28	The system shall be installable from a modern mobile browser through the standard "Add to Home Screen" prompt.
FR-29	The system shall function in standalone display mode, without the browser address bar visible.
FR-30	The system shall provide a cached application shell that is accessible when the device has no internet connection.

3.4.2 Non-Functional Requirements

Non-functional requirements describe properties of the system rather than features. They were drawn from the project's design constraints, from the literature's lessons about consumer adoption, and from our own usage of comparable applications.

Category	Requirement

Category	Requirement
Performance	A scan request shall return a response within 5 seconds under normal load (excluding network latency on the user's side).
Performance	The application shell shall be interactive within 3 seconds on a mid-range Android device on a 3G connection.
Reliability	The back-end shall handle malformed requests through structured 4xx error responses, not unhandled 500 exceptions.
Reliability	Image uploads that fail mid-processing shall not leave orphaned files on the server.
Security	Passwords shall never be stored in plaintext. All password storage shall use bcrypt with at least 12 rounds, preceded by SHA-256 pre-hashing.
Security	Authentication tokens shall be signed with HMAC-SHA256 and have a maximum lifetime of 30 minutes.
Security	Cross-origin requests shall be restricted to a configured allow-list of origins.
Usability	The application shall be operable with one hand on a standard 6-inch smartphone screen.
Usability	The application shall use plain language in all user-facing strings, avoiding technical jargon.
Usability	Status colours shall be consistent across the application: green for Fresh, amber for Eat Soon, orange for Eat Today, red for Spoiled.
Portability	The application shall be installable as a Progressive Web App on both Android and iOS devices without requiring an app store.
Maintainability	The back-end and front-end codebases shall be in separate directories with independent dependency management.
Maintainability	All API endpoints shall be auto-documented through OpenAPI annotations on the route handlers.
Scalability	The back-end shall handle concurrent scan requests without one request blocking another, through the use of async I/O and a thread pool for model inference.
Compatibility	The application shall function on the current versions of Chrome, Firefox, Safari, and Edge.

3.5 System Analysis

Before we describe the proposed system, it is worth looking at what already exists in the same problem space. This section reviews three categories of existing system, identifies what each does well and where each falls short, and uses that comparison to position Fresco. The point is not to claim our system is better than every alternative it is to be specific about what space it occupies and what gap it fills.

3.5.1 Analysis of Existing Systems

Category 1: Static expiration date labelling

The dominant existing approach is the printed "Best Before" or "Use By" date on packaged food. This is the system that almost every consumer in a developed country interacts with daily. The strength of static dates is that they are extremely cheap to produce, universally understood, and require no technology to consume. A consumer who can read can use this system.

The weaknesses are well documented in the food science literature. The dates assume idealised, uninterrupted storage conditions that almost never happen in practice (Leena, 2024). They cannot reflect what actually happened to the food on its journey from manufacturer to fridge. They produce both false urgency (food that is still safe but past the date) and false confidence (food within the date that has been mishandled). Most importantly for our context, they do not exist at all for fresh, unpackaged fruit sold in the open markets that supply most Nigerian households.

Category 2: Sensor-based and IoT freshness systems

The state of the art in dynamic shelf-life monitoring uses physical sensors. Wang et al. (2025) demonstrated a multi-sensor fusion system combining electronic nose gas sensors with temperature monitors, achieving 94.5% prediction accuracy. Patel et al. (2024) explored hyperspectral imaging that can detect internal bruising and sugar content. Sahu et al. (2020) demonstrated a Raspberry Pi-based edge computing system for offline food monitoring.

These systems are genuinely impressive technically. They also share a single, decisive weakness for the consumer market: the hardware costs hundreds to tens of thousands of dollars, and ordinary households are not going to buy it. The literature consistently identifies this as the central barrier to consumer adoption (Hu et al., 2024), but no reviewed system resolves it. The accuracy is real but inaccessible.

Category 3: Camera-based food recognition apps

Several mobile applications use the smartphone camera for food-related identification. Plant disease identifiers like Plantix and PictureThis can recognise crop pests and diseases from a photograph. Calorie-counting apps like MyFitnessPal can identify common foods to log nutritional information. Grocery management apps like NoWaste help users track what is in their fridge.

These applications share the smartphone-only design that we use. None of them, however, addresses fresh-fruit shelf life specifically, and none of them combines fruit identification with a freshness assessment that drives a personalised shelf-life prediction. Calorie apps assume the food is in normal condition. Plant disease apps identify problems with growing fruit, not with stored fruit. Inventory apps track what is in the fridge but cannot tell the user whether the strawberries they put there four days ago are still good.

Category 4: Industrial cold-storage ethylene management

Commercial cold storage facilities have managed ethylene interactions between fruit varieties for over fifty years. The biology is well understood (Watkins, 2006; Saltveit, 1999), the practice is standard in the industry, and the financial savings from getting it right are substantial. None of this knowledge has been translated into a consumer-facing tool. Households continue to put bananas and strawberries in the same bowl, accelerate the spoilage of the strawberries, and have no idea that this is happening. The information is there in the literature, but the bridge to the kitchen has never been built.

3.5.2 Analysis of the Current Proposal

Looking across the four categories above, the gap is clear. Static dates do not exist for fresh fruit. Sensor systems are too expensive for households. Existing camera-based apps do not assess fresh-fruit shelf life. The industrial knowledge about ethylene interactions has not made it to consumers. There is a specific shape of application that does not currently exist: a smartphone-only system that identifies fresh fruit, assesses its visual freshness, predicts its remaining shelf life under user-selected storage conditions, and warns about ethylene-based storage conflicts.

This is the shape Fresco was designed to take. The proposal is not novel in any single component convolutional neural networks for image classification have been validated extensively (Sandler

et al., 2018; Fu et al., 2022), fuzzy reasoning under uncertainty is well established (Vasquez et al., 2024), the ethylene biology is decades old. The contribution is in the integration: putting these components together into a single mobile-deployable product, deliberately keeping the hardware requirement at "a smartphone the user already owns," and accepting the lower accuracy ceiling that comes with that constraint in exchange for the universal accessibility that produces actual adoption.

The proposal also accepts an explicit trade-off in classifier scope. Rather than attempt a shallow classifier across all 42 fruit varieties (which would have required several thousand labelled images per fruit, well beyond a final-year project's data budget), we trained a deep classifier on a small set of fruit types across multiple ripeness stages, and supported the remaining fruits through a manual selection path against the curated database. This is documented in detail in Section 3.9.1.

3.5.3 Description of the Proposed System

Fresco, as proposed and built, is composed of four cooperating pieces. The first is a front-end Progressive Web Application that runs in the user's mobile browser. The user signs in, opens the scan screen, points the camera at a fruit, and triggers a capture. The captured image is sent to the back-end.

The second piece is a back-end HTTP service that receives the image, runs it through the AI pipeline, and returns a structured result. The pipeline produces a fruit identification, a freshness score, shelf-life estimates for three storage methods, an ethylene context note, and a final verdict that ties everything together. The result is structured into three tiers the consumer-facing verdict, the technical detail supporting it, and the storage context to support different levels of user engagement with the system's reasoning.

The third piece is a relational database holding three kinds of data: reference data (the 42-fruit catalogue with shelf-life values, ethylene flags, and storage tips), user data (accounts and authentication state), and tracking data (the personal inventory items each user has saved, with their scheduled expiry notifications).

The fourth piece is the model artefact itself a fine-tuned MobileNetV2 network serialised as a Keras file, loaded by the back-end at startup and held in memory for the lifetime of the process.

The model was trained separately, on Google Colab, and is shipped with the back-end as a static asset. It does not retrain at runtime.

The user-facing flow is straightforward. A user opens the application, signs in, points their phone at a fruit, and gets back a verdict screen within a few seconds. They can save the result to their personal inventory, where it will be tracked over time until either the user marks it consumed or it passes its projected expiry. They can also use the dedicated storage compatibility screen to check a list of fruits for ethylene conflicts. The entire experience runs from a mobile browser, installs onto the home screen if the user chooses, and remains usable in standalone mode without browser chrome.

3.6 System Design

This section presents the design of the system through a sequence of diagrams. Each diagram captures one aspect of the system: its place in the world (context), its overall structure (architecture), the people and use cases it supports (use case), the way data moves through it (data flow), the state changes of its tracked objects (state transition), the navigation paths through its front-end (routing flow), and the relationships between its data classes (class diagram). Together they form a complete picture of the design at a level of detail that should let any competent developer reproduce the system.

Each subsection describes what its diagram shows, what each element represents, and what design decision the diagram captures. The diagrams themselves are referenced as figures and should be drawn using standard UML notation where the diagram type calls for it. Where a Mermaid representation is useful, the text describes the node structure in enough detail to reproduce it in any diagramming tool.

3.6.1 System Context Diagram

The system context diagram sits at the highest level of abstraction. It shows the Fresco system as a single black box, surrounded by the external entities that interact with it. The point of the diagram is to make explicit what is inside the system's boundary and what is outside it.

What the diagram shows:
At the centre of the diagram, drawn as a labelled circle, is the Fresco System. Around it, drawn as labelled rectangles, are the four external entities the system interacts with:

The End User the person using the application on their mobile device. The user provides input to the system in the form of registration details, login credentials, image uploads, storage method choices, inventory queries, and compatibility check requests. They receive output from the system in the form of scan results, inventory lists, compatibility warnings, and notification alerts.

The Mobile Browser strictly speaking, the user does not interact directly with the Fresco back-end. They interact with their mobile browser (Chrome, Safari, Firefox), which renders the front-end Progressive Web App. The browser is the bridge between the user and the system. We model it as a separate external entity because the browser's capabilities camera access, service worker support, local storage define the boundary of what the user-facing application can do.

The Database System although the database is logically part of the Fresco system, in deployment it runs as a separate service (PostgreSQL on Render in production, MySQL during development). From a context-diagram perspective, this means the database is an external entity that the Fresco back-end communicates with over a network connection.

The Hosting Platform Render hosts the back-end and Vercel hosts the front-end. These platforms provide compute, networking, environment variables, and the public URLs through which the system is accessible. We treat them as external entities because the system depends on them but does not own them.

Each external entity is connected to the Fresco System by an arrow labelled with the data that flows along that connection. The user-to-browser arrow carries gestures and image captures; the browser-to-system arrow carries HTTP requests; the system-to-database arrows carry SQL queries and results; the hosting-platform arrows carry deployment and environment configuration.

3.6.2 System Architecture Diagram

Where the context diagram showed the system as a single box, the architecture diagram opens the box and shows the major components inside. It uses the three-tier structure that has been described conceptually in Section 3.0 and Section 3.3.

Three horizontal layers, top to bottom:

Presentation Tier (top layer). The Next.js Progressive Web App. Drawn as a wide rectangle, this layer is internally divided into the eight route components (Home, Sign In, Sign Up, Scan, Results, Inventory, Storage, Account), the four component modules (UI primitives, scan components, result components, inventory components), the centralised API client, the React Context auth provider, and the PWA assets (the manifest file and the service worker). Arrows from each route component point inward toward the API client, indicating that all backend communication funnels through that single module.

Application Tier (middle layer). The FastAPI backend. Drawn as a wide rectangle below the presentation tier, this layer contains the four routers (Auth, Scan, Inventory, Fruits), the services layer with the seven-step scan service orchestrator, the AI module folder containing six files (classifier, freshness, estimator, decision engine, compatibility, golden path), the authentication module, and the configuration module. The connection between the presentation tier and the application tier is drawn as a thick arrow labelled "HTTPS / JSON" running vertically between them.

Data Tier (bottom layer). The relational database and the model artefact. Drawn as a wide rectangle at the bottom, this layer contains the four database tables (Fruit, User, UserInventory, Notification), with their primary-key and foreign-key relationships indicated, and a separate compartment labelled "Model Artefact" representing the serialised MobileNetV2 .keras file. The connection between the application tier and the data tier is drawn as two thinner arrows: one between the application and the database labelled "async SQL via SQLAlchemy," and one between the application and the model artefact labelled "in-process load."

The architecture diagram makes explicit several design decisions that have been stated in prose: the decoupling of presentation from application logic, the centralisation of HTTP communication through a single API client, the modular structure of the AI pipeline, and the fact that the model is held in-process rather than queried as a separate service. The reader of the diagram can see at a glance how a scan request travels from a click in the presentation tier, down to the application tier, through the AI pipeline, querying the database for reference data, returning back up through the layers.

3.6.3 Use Case Diagram

The use case diagram captures the functional capabilities of the system from the user's perspective. It uses standard UML use-case notation: actors are drawn as stick figures, use cases are drawn as ovals, and the lines between them indicate which actor can perform which use case.

Two actors:

Guest User a person interacting with the system who has not yet logged in. They can perform a limited set of use cases.

Authenticated User a person who has signed in and holds a valid JWT token. They can perform all use cases the Guest User can perform, plus several authenticated-only use cases. This inheritance relationship is drawn with an arrow from the Authenticated User actor to the Guest User actor.

Use cases (drawn as ovals inside the system boundary):

Available to the Guest User:

Use Case	Description
Register Account	Create a new account by providing username, email, and password.
Sign In	Authenticate using email and password to receive a JWT token.
View Landing Page	Access the public marketing page describing the application.
Browse Fruit Catalogue	View the list of supported fruit varieties and their reference information.
Check Storage Compatibility	Submit a list of fruit names and receive ethylene compatibility warnings (open endpoint).

Additionally available to the Authenticated User:

Use Case	Description
Scan Fruit	Upload an image of a fruit and receive a full verdict with shelf-life and storage recommendation.
Add to Inventory	Save a scan result to the personal inventory for ongoing tracking.

Use Case	Description
View Inventory	List all saved inventory items, sortable and filterable.
Update Inventory Item	Change the storage method of an inventory item and recalculate the expiry.
Mark Item Consumed	Flag an inventory item as consumed.
Delete Inventory Item	Remove an inventory item from the personal inventory.
View Notifications	See scheduled and recently fired expiry alerts.
Sign Out	End the current session, invalidating the local token.

Some use cases have "include" or "extend" relationships. "Add to Inventory" includes "Scan Fruit" because you can only add what you have first scanned. "Update Inventory Item" extends "View Inventory" because updates are typically triggered from the inventory list view. These relationships are drawn with dashed arrows labelled with the stereotype («include» or «extend»).

3.6.4 Data Flow Diagram

The data flow diagram shows how data moves through the system during its most important operation: a fruit scan. It is drawn at two levels of detail. The Level 0 diagram (the context-level data flow) shows the system as a single process with its data flows in and out. The Level 1 diagram opens the process and shows the major sub-processes inside.

Level 0 Context Data Flow

A single circle labelled "Fresco Scan Process" in the centre. Arrows leading in: an "Image Upload" arrow from the User entity, and a "Reference Data" arrow from the Fruit Database data store. Arrows leading out: a "Scan Verdict" arrow back to the User, and an optional "Inventory Record" arrow to the User Inventory data store.

Level 1 Decomposed Scan Process

The single process from Level 0 is decomposed into the seven sub-processes that correspond to the seven steps of the scan service pipeline:

Process 1.1 Image Validation: Receives the raw upload, validates the MIME type and file size, generates a UUID filename, and writes the file to the upload directory. Inputs: raw upload from

User. Outputs: validated image path to Process 1.2; persisted image file to the Upload Storage data store.

Process 1.2 Classification: Receives the validated image path, preprocesses the image to 224×224 RGB normalised tensor, runs the MobileNetV2 inference, post-processes to extract fruit name and freshness score. Inputs: image path from Process 1.1; model weights from the Model Artefact data store. Outputs: predicted fruit name, classification confidence, freshness score, and freshness label to Process 1.3.

Process 1.3 Database Lookup: Receives the predicted fruit name, queries the Fruit Database with a two-stage matching strategy (exact case-insensitive match, then fuzzy substring match if exact fails). Inputs: predicted name from Process 1.2; rows from the Fruit Database data store. Outputs: the matched Fruit reference object to Process 1.4.

Process 1.4 Shelf-Life Estimation: Receives the Fruit reference object and the freshness score, computes the multi-factor estimate for each of the three storage methods. Inputs: Fruit reference and freshness score from Process 1.3 and Process 1.2. Outputs: a dictionary of storage-method to days-remaining to Process 1.5 and Process 1.6.

Process 1.5 Ethylene Evaluation: Receives the Fruit reference object, checks the ethylene producer and sensitivity flags, generates an ethylene context note. Inputs: Fruit reference from Process 1.3. Outputs: ethylene note string to Process 1.6.

Process 1.6 Verdict Generation: Receives all preceding outputs, computes the composite confidence score (weighted fusion of freshness and normalised days remaining), maps the composite to a status (FRESH, EAT_SOON, EAT_TODAY, SPOILED), generates the recommendation string from a status-keyed template. Inputs: all preceding process outputs. Outputs: the structured Verdict object to Process 1.7.

Process 1.7 Response Assembly: Receives the Verdict object and the supporting data, assembles the three-tier ScanResponse (Decision, Detail, Context) and returns it. Inputs: Verdict and all supporting data from preceding processes. Outputs: the ScanResponse to the User entity.

The data stores referenced in the diagram are: Upload Storage (the directory where scanned images are saved), Model Artefact (the loaded .keras file in memory), and Fruit Database (the reference data table).

3.6.5 State Transition Diagram

The most interesting state-transition behaviour in the system belongs to an inventory item. From the moment a user adds a fruit to their inventory until the item is removed, it passes through a sequence of states based on time and user action. The state transition diagram models this lifecycle.

States (drawn as rounded rectangles):

State	Meaning
Fresh	The inventory item was added recently and has more than 2 days of estimated life remaining. Indicator colour: green.
Approaching Expiry	The estimated days remaining has dropped to 2 days or fewer. Indicator colour: amber.
Expiring Today	The estimated days remaining has reached 0 to 1 day. Indicator colour: orange.
Expired	The current date has passed the estimated expiry date. The is_expired flag is set to true. Indicator colour: red.
Consumed	The user has marked the item as consumed. Removed from active inventory views by default. Terminal state.
Deleted	The user has deleted the item. Removed from the database. Terminal state.

Transitions (drawn as arrows between states):

From the initial pseudo-state (the diagram's starting point), an arrow leads into either Fresh or Approaching Expiry, depending on the freshness score and estimated days at the time of scan. The initial state is determined at the moment of insertion.

Fresh transitions to Approaching Expiry through the trigger "days remaining decays to ≤ 2" (a time-based transition, evaluated each time the inventory is viewed).

Approaching Expiry transitions to Expiring Today through the trigger "days remaining decays to ≤ 1." Approaching Expiry can also transition back to Fresh if the user updates the storage method to one with a longer remaining life (for example, moving the item from the counter to the fridge).

Expiring Today transitions to Expired through the trigger "current date passes estimated expiry date."

From any non-terminal state (Fresh, Approaching Expiry, Expiring Today, Expired), the user can trigger a transition to Consumed via the "Mark Consumed" action, or a transition to Deleted via the "Delete" action.

The state transition diagram captures something important about the system's behaviour that is easy to miss in the prose description: the state of an inventory item is not just a function of time, but also of user action. A user who moves their fruit to the fridge can pull it back from Approaching Expiry to Fresh. A user who realises the system's estimate was wrong can mark the item Consumed at any point. The system respects user input over automated state changes.

3.6.6 Routing Flow Diagram

The routing flow diagram shows how the user navigates between the eight screens of the front-end application. It captures the typical paths and the conditional flows.

Nodes (one per route, drawn as rectangles labelled with the route path):
The eight nodes are: `/` (Landing Page), `/auth/signin`, `/auth/signup`, `/scan`, `/results`, `/inventory`, `/storage`, and `/account`.

Edges (arrows labelled with the user action that triggers the transition):
From `/` (Landing): the user can navigate to `/auth/signin` via the "Sign In" button, or to `/auth/signup` via the "Get Started" button.

From `/auth/signup`: on successful registration, the user is redirected to `/scan`. On a registration error (duplicate email, validation failure), the user remains on `/auth/signup` with an error message. From `/auth/signup` there is also a link to `/auth/signin` for users who already have accounts.

From `/auth/signin`: on successful login, the user is redirected to `/scan`. On failed login, the user remains on `/auth/signin` with the deliberately non-specific error message described in Section 3.8. From `/auth/signin` there is also a link to `/auth/signup`.

From `/scan`: on a successful scan, the user is redirected to `/results` with the full ScanResponse passed through browser session storage. On a scan timeout or error, the user remains on `/scan` with an error displayed.

From `/results`: the user can choose to add the scan to inventory (which calls the inventory POST endpoint and redirects to `/inventory`), or to scan again (which redirects back to `/scan`), or to navigate to any other page through the bottom navigation.

The bottom navigation bar visible on all authenticated screens provides direct routes between `/scan`, `/inventory`, `/storage`, and `/account`. These four routes form the core navigation graph that an authenticated user moves around within during normal use.

Authentication is enforced at the route level through the AuthGuard component. Any attempt to navigate to a protected route without a valid JWT triggers a redirect to `/auth/signin` with a return URL preserved in the query string. On successful authentication, the user is sent to the originally requested route rather than the default `/scan`.

3.6.7 Class Diagram

The class diagram captures the data model of the system the four ORM model classes, their attributes, and the relationships between them. It uses standard UML class notation: each class is drawn as a rectangle divided into three sections (class name, attributes, methods), and relationships are drawn as lines with multiplicity indicators at each end.

The four classes:

Fruit represents one row of the reference catalogue. Attributes: id (integer, primary key), name (string), subcategory (string), shelf_life_room_temp_days (float), shelf_life_fridge_days (float), shelf_life_freezer_days (float), is_ethylene_producer (boolean), is_ethylene_sensitive (boolean), optimal_temp_min (float, nullable), optimal_temp_max (float, nullable), ripeness_indicator (text, nullable), storage_tips (text, nullable), image_url (string, nullable). Methods: none beyond the inherited ORM methods.

User represents one registered account. Attributes: id (integer, primary key), username (string, unique), email (string, unique), password_hash (string), created_at (datetime). Methods: none beyond the inherited ORM methods.

UserInventory represents one fruit in one user's personal inventory. Attributes: id (integer, primary key), user_id (integer, foreign key to User.id), fruit_id (integer, foreign key to Fruit.id), freshness_score (float), storage_method (string), estimated_days_remaining (float), scanned_at (datetime), estimated_expiry (datetime), is_expired (boolean), is_consumed (boolean), image_path (string, nullable), quantity (integer), notes (text, nullable). Methods: none beyond the inherited ORM methods.

Notification represents one scheduled expiry alert. Attributes: id (integer, primary key), inventory_id (integer, foreign key to UserInventory.id), message (string), notify_at (datetime), sent (boolean), created_at (datetime). Methods: none beyond the inherited ORM methods.

The three relationships:

User to UserInventory: one-to-many. One user can have many inventory items; each inventory item belongs to exactly one user. The relationship is drawn with a "1" at the User end and a "0..*" at the UserInventory end. The foreign key user_id on UserInventory has ON DELETE CASCADE, meaning that deleting a User automatically deletes all their inventory items.

Fruit to UserInventory: one-to-many. One fruit (in the reference catalogue) can be referenced by many inventory items across different users; each inventory item references exactly one fruit. The relationship is drawn with a "1" at the Fruit end and a "0..*" at the UserInventory end. The foreign key fruit_id on UserInventory has ON DELETE CASCADE.

UserInventory to Notification: one-to-many. One inventory item can have many scheduled notifications (although in practice the system schedules at most one per item); each notification belongs to exactly one inventory item. The relationship is drawn with a "1" at the UserInventory end and a "0..*" at the Notification end. The foreign key inventory_id on Notification has ON DELETE CASCADE.

The class diagram makes explicit the central design decision of the data model: the system maintains a clean separation between reference data (Fruit, immutable curated knowledge) and user data (User, UserInventory, Notification, mutable transactional records), with the foreign-key relationships from user data into reference data ensuring that no inventory item can exist for a fruit not in the catalogue.

3.7 Database Design

The database holds three categories of information. The first is reference data the curated knowledge about fruit that the system uses to make its predictions. The second is user data accounts and authentication. The third is tracking data the personal inventory items a user has saved and the scheduled notifications attached to them. These three categories map onto four tables, described in detail below.

We chose a relational database (MySQL in development, PostgreSQL in production) over a document-oriented database for two specific reasons. The first is that the dominant access pattern is joining: retrieving a user's inventory requires combining each inventory row with its corresponding fruit reference row to get the shelf-life baselines, the ethylene flags, the storage tips, and the ripeness indicators. In a relational database this is a single query with a JOIN. In a document database it would either require N+1 queries (one for the inventory list, then one per item to fetch the fruit details) or denormalisation (duplicating the fruit data into every inventory row, which creates update anomalies when the reference data changes). The second reason is that the data has natural referential integrity constraints a user cannot have an inventory item for a fruit that does not exist, a notification cannot exist without an inventory item and these constraints are easier to enforce at the database level than in application code.

3.7.1 The Fruit Table

The Fruit table stores the curated reference data. It is the system's authority on what is known about each fruit variety. The schema is shown in Table 3.1.

Table 3.1: Fruit table schema

Column	Type	Constraints	Description
id	Integer	PK, indexed	Unique identifier (1 through 42)
name	String(100)	Unique, not null, indexed	Common name of the fruit
subcategory	String(50)	Not null, indexed	One of seven groupings (Common, Citrus, Berries, etc.)
shelf_life_room_temp_days	Float	Not null	Baseline days at room temperature
shelf_life_fridge_days	Float	Not null	Baseline days in the refrigerator
shelf_life_freezer_days	Float	Not null	Baseline days in the freezer
is_ethylene_producer	Boolean	Default false	True if the fruit emits ethylene during ripening
is_ethylene_sensitive	Boolean	Default false	True if exposure to ethylene accelerates deterioration
optimal_temp_min	Float	Nullable	Lower bound of the fruit's optimal temperature in °C
optimal_temp_max	Float	Nullable	Upper bound of the fruit's optimal temperature in °C

Column	Type	Constraints	Description
ripeness_indicator	Text	Nullable	Human-readable description of ripening cues
storage_tips	Text	Nullable	Curated storage advice for the user interface
image_url	String(500)	Nullable	Path or URL to a reference image

The 42 seeded rows were compiled from food-science reference data, principally the USDA Postharvest Handling Guidelines and Kader (2002), with the ethylene flags drawn from Watkins (2006) and Saltveit (1999). The catalogue was deliberately weighted toward fruit varieties commonly sold in Nigerian markets tropical varieties like papaya, guava, soursop, jackfruit, African star apple, and plantain are represented alongside the more widely imported apples, grapes, and pears. Most academic fruit datasets over-represent temperate varieties, and we did not want our database to reproduce that bias for a Nigerian-targeted application.

3.7.2 The User Table

The User table stores authentication credentials and basic profile information. The schema is shown in Table 3.2.

Table 3.2: User table schema

Column	Type	Constraints	Description
id	Integer	PK, indexed	Unique identifier
username	String(50)	Unique, not null, indexed	Display name, alphanumeric and underscore only
email	String(255)	Unique, not null, indexed	Email address used for authentication
password_hash	String(255)	Not null	The bcrypt hash of the SHA-256-prehashed password
created_at	DateTime	Default now()	Timestamp of account creation

Plaintext passwords are never stored. The two-stage hashing scheme used SHA-256 pre-hash, Base64 encode, then bcrypt with 12 rounds is described in detail in Section 3.8.1. The username and email columns both have unique constraints and indexes; the unique constraints prevent duplicate accounts, and the indexes make the lookup-by-email path of login take microseconds rather than seconds as the user table grows.

3.7.3 The UserInventory Table

The UserInventory table is the central transactional table of the system. Every fruit a user has scanned and chosen to track produces one row in this table. The schema is shown in Table 3.3.

Table 3.3: UserInventory table schema

Column	Type	Constraints	Description
id	Integer	PK, indexed	Unique identifier
user_id	Integer	FK → User.id, CASCADE	Owning user
fruit_id	Integer	FK → Fruit.id, CASCADE	Referenced fruit variety
freshness_score	Float	Not null, default 1.0	AI-assessed freshness at scan time
storage_method	String(20)	Not null	User-selected storage condition (room_temp, fridge, freezer)
estimated_days_remaining	Float	Not null	Computed days of remaining shelf life at insert time
scanned_at	DateTime	Default now()	When the scan occurred
estimated_expiry	DateTime	Not null	Projected expiration datetime
is_expired	Boolean	Default false	Whether the item has passed its estimated expiry
is_consumed	Boolean	Default false	Whether the user has marked it as consumed
image_path	String(500)	Nullable	Path to the saved scan image

Column	Type	Constraints	Description
quantity	Integer	Default 1	Number of items in the inventory entry
notes	Text	Nullable	Optional user annotations

Both foreign keys carry ON DELETE CASCADE constraints. This is a deliberate design choice. When a user deletes their account, every inventory record they created is removed automatically no orphaned data left behind. Similarly, if a fruit reference row were ever removed from the catalogue (which we do not do in practice, but the constraint is correct in principle), any inventory items pointing at it would be cleaned up by the database itself rather than left dangling. Enforcing referential integrity at the database level rather than only in application code closes off an entire category of bugs.

3.7.4 The Notification Table

The Notification table holds scheduled expiry alerts. Each row represents one notification that the system intends to fire for one inventory item at one point in time. The schema is shown in Table 3.4.

Table 3.4: Notification table schema

Column	Type	Constraints	Description
id	Integer	PK, indexed	Unique identifier
inventory_id	Integer	FK → UserInventory.id, CASCADE	Associated inventory item
message	String(500)	Not null	The notification text
notify_at	DateTime	Not null	When the notification should fire
sent	Boolean	Default false	Whether the notification has been sent
created_at	DateTime	Default now()	When the notification record was created

Notifications are scheduled automatically when an inventory item is created. If the inventory item's estimated days remaining is greater than one, a notification is scheduled to fire one day before the projected expiry. The notification record carries the human-readable message string at the time of scheduling, so the message is stable even if the system's recommendation templates change later. The actual delivery of the notification to the user's device through the Web Push API, for example is identified as future work in Chapter Five.

3.7.5 Relationships and Cascade Behaviour

The three relationships between the four tables are summarised in Table 3.5.

Table 3.5: Inter-table relationships

From	To	Cardinality	Cascade
User	UserInventory	1 → 0..n	Deleting a User deletes all their inventory items
Fruit	UserInventory	1 → 0..n	Deleting a Fruit deletes all inventory items referring to it
UserInventory	Notification	1 → 0..n	Deleting an inventory item deletes its notifications

The relationships are bidirectional in the SQLAlchemy ORM, which means that from any Python-side object we can navigate to its related objects through attribute access `user.inventory` gives the list of UserInventory records for a User, `inventory_item.fruit` gives the Fruit reference, and so on. This bidirectional navigation is a convenience of the ORM, not a property of the underlying database schema.

3.8 Security Architecture

Security in a system like Fresco is not particularly exotic we are not storing payment information, we are not handling medical records, and we are not a target for sophisticated attacks. What we are storing is account credentials and personal usage data, both of which deserve careful handling. The security architecture has three pieces: how passwords are stored, how authentication is enforced on protected endpoints, and how cross-origin requests are controlled.

3.8.1 Password Storage with a Two-Stage Hashing Scheme

Passwords are hashed in two stages before storage. This is more careful than the standard bcrypt-only pattern most projects use, and the reason is a specific and surprising limitation of bcrypt itself.

In the first stage, the user's plaintext password is passed through SHA-256, producing a 32-byte digest. SHA-256 is fast and cheap exactly the wrong properties for password hashing on its own so this step is not what is making the storage secure. The point of this step is to neutralise bcrypt's truncation behaviour. The bcrypt algorithm silently truncates its input at 72 bytes. Any password longer than 72 characters has its trailing characters ignored. Two passwords that share the same first 72 characters would produce identical bcrypt hashes. This is a real correctness bug that has affected production systems Okta publicly disclosed a vulnerability stemming from this exact behaviour in 2024. The SHA-256 pre-hash produces a 32-byte digest regardless of how long the input was, so the actual input bcrypt sees is always within its supported range.

In the second stage, the SHA-256 digest is Base64-encoded and passed to bcrypt for the final hash. The bcrypt work factor is set to 12, which means each hash computation involves 2^12 = 4,096 rounds of key derivation. This is what makes the hash computationally expensive. Each verification takes a measurable fraction of a second slow enough that brute-force attacks against the hash database would take weeks per password, fast enough that a legitimate login completes within a normal response time.

The combined effect is that an attacker who obtains the password hashes from the database can neither recover the original passwords through dictionary attacks (because bcrypt is too slow) nor exploit bcrypt's truncation to find collisions (because the SHA-256 pre-hash normalises all inputs to 32 bytes). Both properties matter.

3.8.2 JSON Web Tokens and Authorisation

Authentication state is carried by JSON Web Tokens signed with the HMAC-SHA256 algorithm (HS256). When a user successfully logs in or registers, the backend issues a token whose payload contains the user's database identifier (in the standard `sub` claim), an issued-at timestamp, and an expiry timestamp 30 minutes in the future. The token is signed with the JWT secret key from the application's configuration.

The frontend stores the token in browser local storage under the key `fresco_token`, and simultaneously writes it into a cookie called `fresco_auth` with `SameSite=Lax` and a seven-day max-age. The cookie copy is what makes the token accessible to Next.js server-side middleware if needed for route protection at the edge.

Protected endpoints declare a FastAPI dependency that extracts the token from the Authorization header (in the standard `Bearer <token>` form), verifies the signature, checks the expiry, and looks up the corresponding user record from the database. If any of these steps fails, the dependency raises an HTTP 401 response and the route handler never runs. If all steps succeed, the dependency yields the User object to the route handler.

Critically, the route handlers read the user identity from the dependency-injected User object, never from the request body or URL. An endpoint that needs to know "which inventory items belong to this user" uses `current_user.id`, never an `user_id` parameter that the client might have sent. This pattern eliminates an entire class of insecure direct object reference vulnerabilities there is no way for a malicious request to claim to be a different user, because the only authoritative source of user identity is the cryptographically signed token.

The 30-minute expiry is deliberately short. If a token is somehow stolen through cross-site scripting that leaks local storage, for example the attacker has at most 30 minutes to use it before it expires. The trade-off is that legitimate users have to re-authenticate every half hour of inactivity. For an application that is consulted briefly and intermittently (you scan a fruit, you check on your inventory, you put your phone away), this trade-off is acceptable.

3.8.3 Anti-Enumeration Login Error

The login endpoint returns an identical error message "Invalid email or password" whether the email address is unknown, the password is wrong, or both. This is a deliberate design choice. A login endpoint that distinguishes between "unknown email" and "wrong password" leaks information about which email addresses are registered. An attacker who can probe the endpoint with arbitrary email addresses can discover which emails belong to real accounts, which is useful for follow-on phishing attempts. The non-specific error closes this information leak.

3.8.4 Cross-Origin Resource Sharing

Modern browsers enforce a same-origin policy that prevents JavaScript on one origin from making requests to a different origin unless the target server explicitly permits it. Because our frontend and backend run on different origins (the frontend on Vercel, the backend on Render), we have to configure the backend to permit cross-origin requests from the frontend's origin specifically.

The Cross-Origin Resource Sharing middleware is configured at application startup. In development, the allowed origins are `http://localhost:3000` and `http://127.0.0.1:3000`. In production, the allowed origins are read from the `ALLOWED_ORIGINS` environment variable, which lets us point the same code at any frontend deployment without modifying source. The middleware permits credentials (required for the JWT cookie to travel cross-origin), all HTTP methods, and all headers which is acceptable because the cross-origin restriction is on origin, not on request shape.

3.8.5 Input Validation at the API Boundary

Every request body, query parameter, and path parameter is validated by Pydantic before it reaches a route handler. The validation rules are declared on the Pydantic schemas: a username must match a regular expression of alphanumeric characters and underscores, a password must be between 6 and 128 characters, an email must be a syntactically valid email address, and so on. A request that violates any of these constraints receives an HTTP 422 response with a precise error message indicating which field failed which rule. The route handler never sees malformed inputs.

This is a small but cumulatively significant security property. Many web application vulnerabilities SQL injection, command injection, path traversal depend on the application accepting and processing malformed input. By rejecting malformed input at the boundary, before any business logic runs, we close off many of these attack surfaces without needing to think about them in every handler.

3.9 Algorithm and Modelling Approach

The intelligence of the Fresco system lives in five algorithmic components, sequenced into a pipeline. Each component has one job. The CNN classifier turns a photograph into a fruit identity and an age class. The freshness scoring module turns the age class into a continuous

score on the 0.0-to-1.0 scale. The shelf-life estimator turns the score, the fruit reference data, and the storage method into a number of days remaining. The decision engine fuses everything into a single verdict. The ethylene compatibility engine cross-references the result against the user's saved inventory. This section explains each component.

The components are explained in the order they execute. Each subsection covers what the component does, what algorithms or models are used, and why those algorithms or models were chosen over alternatives. Where there is a worked numerical example, we include it because an algorithm only becomes credible when you can put real inputs into it and watch the outputs make sense.

3.9.1 The CNN Model for Fruit Classification

The fruit classifier is a convolutional neural network specifically, a MobileNetV2 architecture (Sandler et al., 2018) that has been fine-tuned through transfer learning on a fruit-specific dataset. The classifier is the part of the system most people ask about, so it is worth being precise about what it is and what it is not.

What MobileNetV2 is

MobileNetV2 is a convolutional neural network architecture published by Google researchers in 2018 (Sandler et al., 2018). It was designed specifically to run image classification on mobile and embedded devices phones, Raspberry Pis, microcontrollers where computational resources are limited. Compared to larger architectures like ResNet-50, MobileNetV2 uses roughly one-eighth the compute for inference while achieving similar accuracy on standard benchmarks.

The architectural ideas that make this possible are two: depthwise separable convolutions and inverted residual blocks. A depthwise separable convolution splits a standard convolution operation into two smaller operations a depthwise step that processes each input channel independently, followed by a pointwise step that combines the channels. This decomposition reduces the number of multiply-add operations by a factor of around 8 to 9 with only a small loss of expressive power. The inverted residual block adds a skip connection between narrow input and narrow output representations, with the heavy spatial work happening on an expanded

intermediate representation. This is the inverse of the residual structure in ResNet, where the skip connection runs between wide representations.

For Fresco, what matters about MobileNetV2 is that it is fast and small. The trained model file weighs 11.6 megabytes small enough to load quickly at server startup and to fit comfortably in memory. Inference takes between 200 and 500 milliseconds per image on a modest CPU, which is fast enough that the user does not perceive the latency as a delay.

Transfer learning, in plain terms

Training a deep neural network from scratch requires very large amounts of labelled data typically hundreds of thousands to millions of labelled images. A final-year project does not have the data or the compute budget to train from scratch. Transfer learning is the technique that lets us avoid this problem.

MobileNetV2 is available in its pre-trained form, with weights that have been trained on ImageNet a dataset of 1.28 million images spanning 1,000 object categories. These pre-trained weights encode a general visual understanding: the model knows how to detect edges, textures, curves, colours, and the kinds of object shapes that show up across natural images. When we fine-tune it on fruit images, we are not teaching it to see from scratch we are teaching it to apply the visual understanding it already has to a more specific task.

In practice, fine-tuning works in two stages. In the first stage, the convolutional layers of MobileNetV2 are frozen their weights are not updated and a new, small classification head is attached on top. Only the classification head is trained. This stage adapts the network to map the pre-trained features to our specific fruit classes. In the second stage, the top layers of the convolutional base are unfrozen, and the whole top portion of the network is trained at a very small learning rate. This stage allows the pre-trained features themselves to be slightly adjusted to the fruit domain, while the small learning rate prevents the catastrophic forgetting that would destroy the general visual knowledge.

The 14 classes the model recognises

This is the part of the design that is easiest to misunderstand, so it deserves precision. The classifier does not have 42 output classes corresponding to the 42 fruit varieties in the database. It has 14 output classes, structured around a small set of fruit types across multiple age ranges.

The 14 classes are organised as four fruit types Apple, Banana, Carrot, and Tomato each appearing across multiple age ranges, plus a single "Expired" class shared across all four types. The age ranges are encoded in the class names: "Banana(1-5)" represents a banana between one and five days old, "Banana(5-10)" represents one between five and ten days old, and so on. The full class list is shown in Table 3.6.

Table 3.6: The 14 output classes of the classifier

Fruit type	Age-range classes
Apple	Apple(1-5), Apple(5-10), Apple(10-15)
Banana	Banana(1-5), Banana(5-10), Banana(10-15), Banana(15-20)
Carrot	Carrot(1-7), Carrot(7-14), Carrot(14-21)
Tomato	Tomato(1-3), Tomato(3-7), Tomato(7-10)
All types	Expired

This structure is more elegant than a simple 14-fruit identifier would have been, and the reasoning is worth stating clearly. Each class represents a specific fruit at a specific stage of its life. When the model predicts "Banana(15-20)" with high confidence, it is telling the rest of the system two things at once: this is a banana, and it is in the late stage of its useful life. The model performs classification and freshness assessment in a single forward pass. We do not need a second model for freshness the freshness comes from the class itself, mapped through a small dictionary that translates each age range into a numerical score.

The trade-off this design accepts is that the AI-driven automatic identification covers only four fruit types, not all 42 in the database. For the other 38 fruits in the catalogue, the system supports a manual selection path: when a user knows what fruit they have, they choose it from the database directly, and the freshness assessment defaults to a baseline value. This was the right call for a final-year project's data budget. Training a high-quality classifier across 42 fruits at multiple ripeness stages each would have required several thousand carefully labelled images per fruit a data collection effort beyond what we could realistically achieve. The choice was between a polished classifier on four well-trained categories plus a curated 42-fruit database, or a mediocre classifier on 42 weakly-trained categories. The former gives a better user experience.

Preprocessing

Before an image can be fed to the model, it has to be preprocessed into the exact format the model expects. The preprocessing pipeline runs four steps in sequence. The image is loaded using the Python Imaging Library, converted to the RGB colour space (which standardises grayscale or RGBA inputs), resized to 224×224 pixels using bilinear interpolation, and normalised by the transformation `(pixel / 127.5) - 1.0` so that every pixel value falls in the range [-1, 1]. This exact normalisation is the one MobileNetV2 was pre-trained with on ImageNet, and using a different scheme would break the transfer-learned features.

Inference and post-processing

The model is loaded once at application startup and held in memory for the lifetime of the server process. Inference itself runs in a thread pool via `asyncio.to_thread()`, which prevents the synchronous TensorFlow call from blocking the FastAPI event loop. While the model is running on one request, the server is free to accept and respond to other requests.

Two post-processing steps are applied to the model's raw output. First, the probability for the "Expired" class is forcibly set to zero before the argmax is taken. We added this step after observing during validation that the model was over-predicting the Expired class a consequence of slight over-representation of that class in the training data combined with its visual catch-all nature. Setting the probability to zero before the argmax fixes this without retraining. Second, the argmax of the remaining 13 probabilities determines the predicted class. The class name is parsed using a regular expression that strips the `(N-M)` age-range suffix and converts the remaining name to title case for display.

3.9.2 Fuzzy-Logic-Inspired Freshness Scoring

The classifier output gives us a class name and a confidence. To get from there to a useful freshness score, we apply a fuzzy-logic-inspired mapping that translates the class into a continuous value on the 0.0-to-1.0 scale.

It is worth being honest here about what we mean by "fuzzy-logic-inspired." A full Fuzzy Inference System would have overlapping membership functions over linguistic variables, a rule base of IF-THEN rules, an inference engine that combines rule activations, and a defuzzification step that converts the fuzzy output back to a crisp value. We do not implement all of that. What we take from fuzzy logic is the principle of continuous, graduated category boundaries the idea that a fruit's quality is a matter of degree rather than a binary fresh-or-spoiled decision (Zadeh, 1965). The mapping we use is a five-category scheme with score thresholds derived from the food-science literature.

Table 3.7: The five-category freshness mapping

Score range	Label	Indicator colour	Recommended action
0.80 – 1.00	Fresh	Green	Full base shelf life; standard storage
0.60 – 0.79	Slightly Aged	Amber	Reduced estimate; eat soon
0.40 – 0.59	Aging	Orange	Significantly reduced; use within 1–2 days
0.20 – 0.39	Old	Red	Minimal remaining; consume immediately
0.00 – 0.19	Spoiled	Dark red	Zero days; discard recommendation

The translation from the classifier's age-range class to a freshness score is a lookup table. "Apple(1-5)" maps to a score in the Fresh band (typically 0.90 or so), "Apple(5-10)" maps to a score in the Slightly Aged band (around 0.70), "Apple(10-15)" maps to the Aging or Old band depending on the visual cues the model picked up. The exact mapping values were calibrated against the labelled freshness annotations in our training set so that the model's age-class predictions translate into freshness scores that match human judgement on the same images.

The use of a continuous score rather than the categorical label directly is what lets the shelf-life estimator make differentiated predictions. A score of 0.79 maps to the same Slightly Aged label

as a score of 0.61, but the shelf-life estimator produces meaningfully different numerical predictions for the two. This is the practical value of keeping the underlying signal continuous and using the categorical label only for the user interface.

3.9.3 The Multi-Factor Shelf-Life Estimator

Once we have a fruit identity and a freshness score, the shelf-life estimator turns these into a number of days remaining. The estimator uses a multi-factor formula that is more sophisticated than a simple linear multiplier.

Estimated Days Remaining = Base Shelf Life × (Freshness Score)^1.5 × Temperature Factor × Ripeness Factor

Each of the four terms is computed independently.

The **Base Shelf Life** is read from the fruit reference data in the database. There are three values per fruit for room temperature, refrigerator, and freezer and the estimator computes the formula three times, once per storage method, producing three days-remaining values.

The **Freshness Factor** is the freshness score raised to the power 1.5. The non-linear exponent is the part of the formula that most often gets overlooked. We use 1.5 rather than 1.0 because food science evidence (Jay et al., 2005; Zhang et al., 2023) shows that quality degrades faster as a fruit ages the rate of deterioration accelerates over the lifetime of the fruit, it does not stay constant. A linear multiplier would over-estimate the remaining life of a fruit that is already past its peak. The exponent of 1.5 was chosen because it produced estimates that matched our intuition about real fruit across the calibration tests we ran. A banana at 0.50 freshness gets a Freshness Factor of 0.35, not 0.50 reflecting that it is closer to spoiled than the linear reading would suggest.

The **Temperature Factor** adjusts for the difference between the user's chosen storage temperature and the fruit's biological optimum. Storage temperatures are fixed constants room temperature at 22°C, refrigerator at 4°C, freezer at -18°C. If the storage temperature falls within the fruit's optimal range (read from the `optimal_temp_min` and `optimal_temp_max` columns of the Fruit table), the factor is 1.0. Outside that range, a 2% penalty is applied per degree Celsius of deviation, with the factor clamped to a minimum of 0.5 to prevent the formula from

producing unreasonably small numbers. Freezer storage gets a fixed factor of 0.95, reflecting the small but real quality loss from freezing and thawing.

The **Ripeness Factor** is a categorical multiplier derived from the freshness score, representing the effect of the current ripeness stage on the remaining life. The four stages are Unripe (score ≥ 0.85, factor 1.0), Nearly Ripe (score ≥ 0.70, factor 0.85), Ripe (score ≥ 0.50, factor 0.65), and Overripe (score < 0.50, factor 0.30). The overripe penalty is severe because an overripe fruit, regardless of its baseline shelf life, has very limited time remaining.

Worked example

Take a real input: a banana that the classifier returned at freshness 0.72, with the user choosing refrigerator storage. The Fruit reference data for banana gives a refrigerator baseline of 7 days, an optimal temperature range of 13°C to 15°C.

Freshness Factor: 0.72 raised to the power 1.5 = approximately 0.611.

Temperature Factor: 4°C (refrigerator) is below the banana's optimal range by 9°C. A 2% per degree penalty gives 0.82 (or in practice, the clamp at 0.5 if the deviation were larger).

Ripeness Factor: 0.72 falls in the Nearly Ripe band, giving 0.85.

Combining: $7 \times 0.611 \times 0.82 \times 0.85 \approx 2.98$ days. The estimate is approximately 3 days of remaining life in the refrigerator.

The estimator computes this for all three storage methods. For the same banana, the room-temperature estimate would be lower (because room temperature is also outside the banana's optimal range, but in the opposite direction), and the freezer estimate would be much higher (because the baseline freezer shelf life of banana is around 60 days).

3.9.4 The Decision Engine

The decision engine takes the outputs of the preceding components the fruit identity, the freshness score, the three shelf-life estimates, and the ethylene flags and produces a single user-facing verdict. The verdict has four parts: a status, a composite confidence score, a recommended storage method, and a human-readable recommendation string.

The composite confidence score

The composite score fuses two signals using a weighted linear combination:

$$\textit{Composite Score} = (\textit{Freshness Score} \times 0.6) + (\textit{Normalised Days Remaining} \times 0.4)$$

The normalised days remaining is the days-remaining value for the recommended storage method, divided by the same method's base shelf life, clamped to the range [0, 1]. Normalisation is important because absolute days mean different things for different fruits. Three days remaining is plenty for a strawberry whose baseline is five days, but very limited for an apple whose baseline is thirty days. Dividing by the baseline puts the two signals on a comparable scale.

The 60-40 weighting reflects a judgement about which signal is more reliable. The freshness score is the direct visual assessment of the fruit's current condition it is the strongest signal we have from a camera-only system. The normalised days remaining provides context but is one step removed from the visual evidence. We weight the direct visual signal more heavily for that reason.

Status mapping

The composite score is mapped to one of four statuses using descending threshold evaluation:

Composite score	Status	What the user sees
≥ 0.70	FRESH	The fruit is in good condition
≥ 0.40	EAT_SOON	The fruit is aging; consume in 2–3 days
≥ 0.20	EAT_TODAY	The fruit needs to be consumed today
< 0.20	SPOILED	The fruit is past its usable condition

The recommended storage method

From the three shelf-life estimates produced by the estimator, the decision engine picks the storage method that gives the longest remaining life. This is usually but not always the freezer for very fresh fruit at room temperature, the recommendation is often to leave it at room temperature, because freezing changes texture and is not always the best choice. The decision engine combines the numeric estimates with a small set of fruit-specific override rules (a banana, for example, should generally not be frozen unless the user is planning to use it in a smoothie).

Worked example (continued)

Continuing the banana example from Section 3.9.3: the recommended storage method is refrigerator with 2.98 days remaining, and the refrigerator baseline is 7 days. Normalised days remaining = 2.98 / 7 = 0.426.

Composite Score = (0.72 × 0.6) + (0.426 × 0.4) = 0.432 + 0.170 = 0.602.

Status: 0.602 is below the FRESH threshold (0.70) but above the EAT_SOON threshold (0.40), so the status is EAT_SOON. The recommendation string for EAT_SOON with the values filled in becomes: "Your Banana is aging eat it within the next 2 to 3 days. For longest life, keep it in the refrigerator."

3.9.5 The Ethylene Compatibility Engine

The ethylene compatibility engine models fruit-to-fruit storage conflicts as a graph problem. It is one of the most distinctive features of the system and one that no other consumer-facing application we reviewed currently provides.

The biology, briefly

Certain fruits emit a gaseous plant hormone called ethylene (C_2H_4) as they ripen. The gas is not harmful to people, but it accelerates the ripening of other fruits exposed to it. The acceleration effect is dose-dependent and species-specific some fruits are highly ethylene-sensitive (strawberries, grapes, kiwis) and deteriorate noticeably faster when stored near producers; some are largely insensitive. Some fruits both produce ethylene and are sensitive to it (apples, bananas, pears). This biology is well-documented (Watkins, 2006; Saltveit, 1999) and routinely managed at the industrial cold-storage scale.

The graph representation

We model the ethylene landscape as two sets and a graph derived from them. The first set, ETHYLENE_PRODUCERS, contains the 19 fruits in our catalogue that emit significant ethylene during ripening: Apple, Apricot, Avocado, Banana, Cantaloupe, Fig, Honeydew, Kiwi, Mango, Nectarine, Papaya, Passion fruit, Peach, Pear, Persimmon, Plantain, Plum, Tomato, and Watermelon. The second set, ETHYLENE_SENSITIVE, contains the 24 fruits that deteriorate measurably faster in the presence of ethylene: Apple, Banana, Blackberry, Blueberry, Cherry, Cranberry, Date, Grape, Grapefruit, Guava, Kiwi, Lemon, Lime, Lychee, Mandarin, Mango, Orange, Pear, Pineapple, Pomegranate, Raspberry, Soursop, Star apple, and Strawberry. The intersection Apple, Banana, Kiwi, Mango, and Pear represents fruits that both produce and respond to ethylene, which is biologically accurate.

At application startup, a conflict graph is built by iterating the Cartesian product of the two sets. For each (producer, sensitive) pair, if the producer is not the same fruit as the sensitive fruit, two bidirectional edges are added to the graph one in each direction. The graph is stored as an adjacency list (a Python dictionary keyed by fruit name, with values being sets of conflicting fruit names). Construction is O(P × S) 19 × 24 = 456 iterations, of which 5 are skipped as self-conflicts and completes in well under a millisecond at startup.

The pairwise checking algorithm

When a user submits a list of fruit names to the compatibility endpoint, the engine performs three steps. Names are normalised to title case and deduplicated while preserving order. All unordered pairs of distinct fruits are evaluated by checking whether an edge exists in the conflict graph. For each conflict, a human-readable warning is generated naming the producer, the sensitive party, and the consequence ("Banana produces ethylene gas that can significantly accelerate the ripening and spoilage of nearby Strawberry").

The pairwise check is $O(n^2)$ in the number of submitted fruits. For a household inventory of 20 fruits, that is 190 comparisons completed in well under a millisecond. The algorithm does not need to be more efficient because n is small and bounded by physical reality. The output is grouped: producers are placed in one group and non-producers in another, giving the user a simple, practical separation recommendation.

3.9.6 Natural Language Generation for Recommendations

The recommendation strings the user sees "Your Banana is aging, eat it in the next 2–3 days" and similar are not produced by a language model. They are generated through template substitution. The decision engine selects a template based on the verdict status and the fruit's ethylene properties, and substitutes the specific values (fruit name, days remaining, recommended storage method) into the template.

We mention this here because the supervisor's guidance asked us to describe the natural language part of the system. The natural language in Fresco is intentionally template-based for two reasons. The first is reliability: a template-based recommendation is always grammatical, always names the right fruit, and never hallucinates information. A language model might produce more varied output but at the cost of occasionally producing wrong or misleading text. The second is interpretability: a user who reads a Fresco recommendation can trace, in the codebase, exactly which template produced their specific message and exactly what values were substituted. There is no black box between the system's reasoning and the user's experience.

The templates themselves were drafted by hand and reviewed for tone consistency. The Fresh template is straightforward and reassuring. The EAT_SOON template is helpful and slightly urgent. The EAT_TODAY template adds an alternative-use suggestion (smoothie, recipe) to soften the urgency. The SPOILED template is direct without being harsh. A representative selection is shown in Table 3.8.

Table 3.8: Recommendation templates by status

Status	Template
FRESH	"Your {fruit} is in great condition. Store in the {storage_method} for up to {days} days."
EAT_SOON	"Your {fruit} is aging eat it within the next {days_label} days. For longest life, keep it in the {storage_method}."
EAT_TODAY	"Your {fruit} needs to be consumed today. Consider using it in a smoothie or recipe."
SPOILED	"Your {fruit} appears to be past its usable condition. We recommend discarding it."

Ethylene context notes are generated through the same template approach, conditional on the fruit's producer and sensitivity flags. A fruit that is both a producer and sensitive (a banana, for

example) receives a longer note explaining both roles. A fruit that is only a producer or only sensitive receives a shorter, role-specific note. A fruit that is neither receives no ethylene note at all, keeping the response payload clean.

3.10 Application of the System

This section describes how the system is actually used in practice. We walk through three realistic scenarios each one representing a different kind of interaction a user would have with Fresco and trace what happens in each tier of the system as the user moves through the flow.

3.10.1 Scenario One: A First-Time User Scans a Banana

Imagine someone call her Funke who has just heard about Fresco from a friend. She has a few bananas on her kitchen counter that she bought two days ago at Bariga Market, and she wants to know how long they will last.

Funke opens her phone browser and types in the Fresco URL. The landing page loads, served by the Next.js server in static-rendered mode for speed. She taps "Get Started," which takes her to the sign-up route at `/auth/signup`. She fills in a username, her email, and a password of at least six characters. When she taps Register, the front-end sends a POST request to `/api/auth/register`. The back-end validates the input through the Pydantic schema, checks that neither the username nor the email is already taken, hashes the password through the two-stage SHA-256-then-bcrypt scheme, creates a new row in the User table, generates a JWT signed with HS256, and returns the token along with a public view of her user object. The front-end stores the token in local storage and as a cookie, then redirects her to the scan screen.

On the scan screen, the CameraView component requests camera permission through `navigator.mediaDevices.getUserMedia()`. Funke's browser asks for permission; she taps Allow. The rear-facing camera video stream is attached to a hidden video element, and a live preview is rendered on the screen. She positions the phone over one of her bananas and taps the capture button. The component draws the current video frame to an offscreen canvas, converts the canvas to a JPEG blob at quality 0.92, and sends it as multipart form data to `/api/scan/` on the back-end. The request carries her JWT in the Authorization header.

The back-end validates the upload (correct MIME type, under 10 megabytes), generates a UUID filename, saves the image to the uploads directory, and runs the scan service pipeline. The classifier preprocesses the image to a 224×224 normalised tensor, runs MobileNetV2 inference in a thread pool, and predicts "Banana(5-10)" with confidence 0.91. The freshness mapping translates this class to a score of 0.72. The database lookup finds the Banana row in the Fruit table. The estimator computes shelf-life values for the three storage methods. The decision engine fuses the freshness and normalised days remaining into a composite score of 0.602, which maps to the EAT_SOON status. The ethylene engine notes that banana is both a producer and sensitive. The response assembler constructs the three-tier ScanResponse and returns it as JSON.

The front-end receives the response, stores it in browser session storage, and navigates to the `/results` route. The results page reads the stored response and renders it: a green-amber StatusBadge at the top showing EAT_SOON, a large "2-3 days" indicator, the recommendation string from the template ("Your Banana is aging eat it within the next 2 to 3 days. For longest life, keep it in the refrigerator"), and a storage breakdown showing the estimates for all three storage methods with the refrigerator highlighted as the recommendation. Below this, the ethylene context note explains that bananas should be stored away from sensitive fruits.

Funke is impressed. She taps "Add to Inventory." The front-end calls the inventory POST endpoint with the fruit id, freshness score, and selected storage method. The back-end creates a UserInventory row, computes the projected expiry date, and schedules a notification for one day before that date. The front-end redirects her to the inventory page where she can see the new banana entry in green-amber with two days remaining.

3.10.2 Scenario Two: A Returning User Checks Their Inventory

A week later, Funke comes back to Fresco. She has added a few more items since her first scan some oranges, a small basket of strawberries, an apple. She opens the application from her home screen (she installed it as a PWA after her first session). The standalone display mode loads without browser chrome, looking and feeling like any other app on her phone.

Because her JWT has expired by now (more than 30 minutes since her last visit), the AuthGuard component detects the missing valid token and redirects her to `/auth/signin`. She logs in with her email and password. The back-end fetches her User record by email, verifies the password

against the stored hash, issues a new JWT, and returns it. The front-end stores the new token and redirects her to the scan screen, but she taps the inventory tab in the bottom navigation instead.

On the inventory page, the front-end calls `GET /api/inventory/` with her token. The back-end identifies her from the token's `sub` claim, queries the UserInventory table for her items, joins to the Fruit table to pull in the reference data for each, and returns the list with two summary statistics: items expiring in the next two days, and items already past their estimated expiry.

The inventory list renders. The original banana is now in red its estimated expiry has passed. The strawberries are in amber, with one day remaining. The apple is in green, with twelve days remaining. The oranges are in green. Funke taps the banana to expand its details and decides she will discard it. She taps Delete; the front-end calls DELETE on the inventory endpoint, the back-end removes the row (cascading deletion of its scheduled notification), and the list refreshes.

3.10.3 Scenario Three: A User Checks Storage Compatibility

Funke is about to go to Mile 12 Market. She wants to know whether she should keep the apples and the strawberries she already has in separate places. She opens the Storage tab in the bottom navigation.

The storage page shows a search-and-add interface. She types "banana," sees Banana in the dropdown, taps it to add it to her list. She does the same for apple, strawberry, and orange. With four fruits in the list, she taps Check Compatibility.

The front-end calls `GET /api/fruits/compatibility?fruits=Banana,Apple,Strawberry,Orange`. The back-end normalises the input, runs the pairwise check against the conflict graph, and identifies the conflicts: Banana → Strawberry, Banana → Orange, Banana → Apple (both are sensitive in this case), Apple → Strawberry, Apple → Orange. For each conflict, a warning is generated. The response groups the four fruits into producers (Banana, Apple) and non-

producers (Strawberry, Orange), with the recommendation to keep the two groups separated.

The storage page renders these results as two cards "Keep These Together" containing the producers and "Keep These Together" containing the sensitive fruits with the warning messages explaining each conflict. Funke decides to put her apples and bananas on the counter, and to keep the strawberries and oranges in a separate compartment of her fridge.

3.11 Pseudocode and Code Snippets

This section provides pseudocode for the most important algorithms in the system. The pseudocode is given in a Python-like syntax with type hints, close enough to the actual implementation that it can be followed against the codebase. Each snippet captures the essential logic without the surrounding error handling and logging that the production code includes.

3.11.1 The Scan Service Pipeline

The scan service orchestrates the seven steps of the AI pipeline. Its essential logic is shown below.

```
async def run_scan_pipeline(image_upload):
    # Step 1: Validate and persist the image
    if image_upload.size > MAX_FILE_SIZE:
        raise ScanError(413, "File too large")
    if image_upload.content_type not in ALLOWED_MIME_TYPES:
        raise ScanError(415, "Unsupported image type")
    image_path = save_with_uuid_filename(image_upload)

    try:
        # Step 2: Classification
        fruit_name, confidence, freshness_score, freshness_label = (
            await classifier.predict(image_path))

        # Step 3: Database lookup (two-stage matching)
        fruit = await lookup_fruit(fruit_name)
        if fruit is None:
            raise ScanError(404, f"Fruit '{fruit_name}' not in database")

        # Step 4: Shelf-life estimation
        days_by_method = estimate_shelf_life(fruit, freshness_score)
```

```
        # Step 5: Ethylene context
        ethylene_note = build_ethylene_note(fruit)

        # Step 6: Decision engine
        verdict = decide(fruit, freshness_score, days_by_method, ethylene_note)

        # Step 7: Response assembly
        return assemble_scan_response(
            fruit, confidence, freshness_score, freshness_label,
            days_by_method, verdict, ethylene_note)
    except Exception:
        # Clean up the saved image on any failure
        delete_file(image_path)
        raise
```

3.11.2 The Classifier Inference Function

The classifier module exposes a single async function that takes an image path and returns the prediction tuple.

```
async def predict(image_path: str) -> PredictionResult:
    model = get_model()           # lazy singleton, warmed up at startup
    img_array = await asyncio.to_thread(preprocess_image, image_path)
    probabilities = await asyncio.to_thread(model.predict, img_array, verbose=0)

    # Suppress the over-predicted 'Expired' class
    probabilities[0][EXPIRED_INDEX] = 0.0

    predicted_idx = int(np.argmax(probabilities[0]))
    confidence = float(probabilities[0][predicted_idx])
    class_name = CLASS_NAMES[predicted_idx]

    # Parse 'Banana(5-10)' into ('Banana', '5-10')
    fruit_name = re.sub(r'\(\d+-\d+\)$', '', class_name).strip().title()
    freshness_score = FRESHNESS_LOOKUP[class_name]
    freshness_label = label_for_score(freshness_score)

    return PredictionResult(
        fruit_name=fruit_name,
        confidence=confidence,
        freshness_score=freshness_score,
        freshness_label=freshness_label,
    )

def preprocess_image(image_path: str) -> np.ndarray:
```

```
    img = Image.open(image_path).convert("RGB").resize((224, 224))
    arr = np.asarray(img, dtype=np.float32)
    arr = (arr / 127.5) - 1.0       # normalise to [-1, 1]
    return np.expand_dims(arr, axis=0)
```

3.11.3 The Multi-Factor Shelf-Life Estimator

The estimator returns a dictionary mapping storage method to days remaining.

```
def estimate_shelf_life(fruit, freshness_score: float) -> dict:
    freshness_factor = freshness_score ** 1.5
    ripeness_factor = ripeness_multiplier(freshness_score)

    results = {}
    for method, base_days, storage_temp in [
        ("room_temp", fruit.shelf_life_room_temp_days, 22.0),
        ("fridge",    fruit.shelf_life_fridge_days,    4.0),
        ("freezer",   fruit.shelf_life_freezer_days,  -18.0),
    ]:
        temp_factor = temperature_factor(
            method, storage_temp, fruit.optimal_temp_min, fruit.optimal_temp_max)
        days = base_days * freshness_factor * temp_factor * ripeness_factor
        results[method] = round(days, 1)
    return results

def ripeness_multiplier(score: float) -> float:
    if score >= 0.85: return 1.00       # Unripe
    if score >= 0.70: return 0.85       # Nearly Ripe
    if score >= 0.50: return 0.65       # Ripe
    return 0.30                         # Overripe

def temperature_factor(method, temp_c, optimal_min, optimal_max) -> float:
    if method == "freezer":
        return 0.95
    if optimal_min is None or optimal_max is None:
        return 1.0
    if optimal_min <= temp_c <= optimal_max:
        return 1.0
    deviation = min(abs(temp_c - optimal_min), abs(temp_c - optimal_max))
    return max(1.0 - (0.02 * deviation), 0.5)
```

3.11.4 The Decision Engine's Composite Score

The decision engine computes the composite score, maps it to a status, and selects the recommended storage method.

```
def decide(fruit, freshness_score, days_by_method, ethylene_note) -> Verdict:
    # Pick the storage method with the longest estimate
    recommended_method = max(days_by_method, key=days_by_method.get)
    best_days = days_by_method[recommended_method]
    base_days = base_for_method(fruit, recommended_method)

    normalised_days = min(best_days / base_days, 1.0) if base_days > 0 else 0
    composite = (freshness_score * 0.6) + (normalised_days * 0.4)

    if composite >= 0.70: status = "FRESH"
    elif composite >= 0.40: status = "EAT_SOON"
    elif composite >= 0.20: status = "EAT_TODAY"
    else: status = "SPOILED"

    recommendation = format_recommendation(
        status, fruit.name, best_days, recommended_method)

    return Verdict(
        status=status,
        composite_score=round(composite, 3),
        days_remaining=best_days,
        recommended_method=recommended_method,
        recommendation=recommendation,
        ethylene_note=ethylene_note,
    )
```

3.11.5 The Ethylene Compatibility Check

The compatibility engine builds its conflict graph at module load time, then answers pairwise queries in $O(n^2)$ over the submitted fruit list.

```
# Module-load-time graph construction
ETHYLENE_PRODUCERS = frozenset({"Apple", "Banana", "Mango", ...})  # 19 names
ETHYLENE_SENSITIVE = frozenset({"Strawberry", "Grape", "Kiwi", ...})  # 24 names

def build_conflict_graph():
    graph = defaultdict(set)
    for producer in ETHYLENE_PRODUCERS:
        for sensitive in ETHYLENE_SENSITIVE:
            if producer == sensitive:
                continue               # skip self-conflicts
            graph[producer].add(sensitive)
            graph[sensitive].add(producer)
    return dict(graph)

CONFLICT_GRAPH = build_conflict_graph()
```

```
# Request-time pairwise checking
def check_compatibility(fruit_names: list[str]) -> CompatibilityResult:
    normalised = list(dict.fromkeys(name.strip().title() for name in fruit_names))
    conflicts = []
    for i, a in enumerate(normalised):
        for b in normalised[i+1:]:
            if b in CONFLICT_GRAPH.get(a, set()):
                conflicts.append(make_conflict_warning(a, b))

    producers = [f for f in normalised if f in ETHYLENE_PRODUCERS]
    others = [f for f in normalised if f not in ETHYLENE_PRODUCERS]

    return CompatibilityResult(
        conflicts=conflicts,
        groups=[
            {"label": "Ethylene Producers", "fruits": producers},
            {"label": "Non-Producers",     "fruits": others},
        ],
    )
```

3.11.6 The Two-Stage Password Hashing

The authentication module's password hash and verification functions use the SHA-256-then-bcrypt scheme described in Section 3.8.

```
def _prehash(password: str) -> bytes:
    digest = hashlib.sha256(password.encode("utf-8")).digest()
    return base64.b64encode(digest)

def hash_password(plaintext: str) -> str:
    prehashed = _prehash(plaintext)
    return bcrypt.hashpw(prehashed, bcrypt.gensalt(rounds=12)).decode("utf-8")

def verify_password(plaintext: str, stored_hash: str) -> bool:
    prehashed = _prehash(plaintext)
    return bcrypt.checkpw(prehashed, stored_hash.encode("utf-8"))
```

3.12 Chapter Summary

This chapter has translated the problem and the research foundations from Chapters One and Two into a concrete system design. We laid out the methodology behind the work, the development lifecycle we followed, and the frameworks and tools that the system is built on. We

documented the functional and non-functional requirements that the system has to satisfy. We analysed the existing systems in the same problem space and described how Fresco is positioned against them. We presented the system through seven design diagrams covering its context, architecture, use cases, data flow, state transitions, routing flows, and data model. We documented the database schema, the security architecture, and the algorithms that drive each of the AI components. We walked through three realistic user scenarios and gave pseudocode for the most important pieces of the implementation.

Two themes connect the design decisions throughout the chapter. The first is separation of concerns: the front-end is decoupled from the back-end, the AI modules are independently testable, the API is centralised, the configuration is externalised through environment variables. The second is honesty about trade-offs. We chose to ship a deep classifier on four fruit types rather than a shallow classifier on all 42. We chose to use template-based natural language for recommendations rather than a language model. We chose to skip integration with environmental sensors so the system would work on any smartphone without extra hardware. Each of these was a deliberate choice, made with reference to the project's constraints and the user's real needs.

The system as designed in this chapter is the system that was actually built. The implementation how the design became running code, what challenges came up, and how those challenges were resolved is the subject of Chapter Four.

CHAPTER FOUR

IMPLEMENTATION

4.1 Introduction

Chapter Three described the design of Fresco, the architecture, the database schema, the API contracts, the AI pipeline, the frontend structure, the security model, and the deliberate trade-offs that shaped each of those decisions. This chapter documents how that design was actually built: the tools used, the development workflow, the order in which the modules were implemented, the concrete code organisation that resulted, and the practical problems that emerged along the way and what we did about them.

We have tried to be specific throughout. Where Chapter Three described an idea, this chapter names the file, gives the line count, and walks through what the code does. Where a feature looked straightforward on paper, this chapter is honest about where it was not. Several sections include short numerical worked examples, the shelf-life formula, and the decision engine's composite score, because a design only becomes credible when you can follow a real input through it and verify the output yourself.

The chapter is organised in roughly the order in which the system was built. Section 4.2 documents the development environment. Sections 4.3 and 4.4 describe the backend application skeleton and the database layer. Section 4.5 is the longest section in the chapter and walks through each module of the AI pipeline. Section 4.6 ties those modules together through the scan service. Sections 4.7, 4.8, and 4.9 cover the frontend, the authentication system, and the Progressive Web App implementation, respectively. Section 4.10 documents the most significant challenges we encountered during development and how we resolved them. Section 4.11 closes the chapter.

4.2 Development Environment and Tools

4.2 Development Environment and Tools All development was performed locally on personal workstations (both Windows and macOS machines). All of this work primarily utilized the VS Code integrated development environment on all machines. Model training was done in the Google Colaboratory platform, since such tasks were extremely computationally expensive, and our local workstations did not possess GPUs.

We utilised version control with a private GitHub repository.

The full stack of tools utilised in this project (as well as their versions) was as follows:

Table 4.1: Development tools and environment

Category	Tool	Version	Purpose
Language (backend)	Python	3.12	Backend runtime
Language (frontend)	TypeScript	5.x	Frontend type-safe development
Web framework	FastAPI	0.115+	Backend HTTP layer
ASGI server	Uvicorn	0.30+	Application server for FastAPI
Frontend framework	Next.js	16.2.1	Frontend application framework
UI library	React	19.2.4	Component model
Styling	Tailwind CSS	v4	Utility-first styling
ORM	SQLAlchemy	2.0	Database abstraction with async support
Database (dev)	MySQL	8.x	Local development database
Database (prod)	PostgreSQL	16.x	Production database on Render
Database (fallback)	SQLite	3.x	Zero-config testing fallback
Validation	Pydantic	2.x	Request and response schemas
ML framework	TensorFlow / Keras	2.16	Model definition and training

Category	Tool	Version	Purpose
Model architecture	MobileNetV2	(ImageNet pretrained)	Transfer learning base
Image processing	Pillow	10.x	Image loading and preprocessing
Cryptography	passlib (bcrypt) + hashlib	1.7 / stdlib	Password hashing
Tokens	python-jose	3.x	JWT issuance and verification
Training environment	Google Colab (T4 GPU)	—	GPU-accelerated model training
Editor	VS Code	1.94+	Primary development environment
Version control	Git + GitHub	—	Source control and collaboration
Backend deployment	Render	—	Production hosting target
Frontend deployment	Vercel	—	PWA hosting target

The decision to use Python 3.12 instead of the newly released 3.13 was conscious; at that time, while the official TensorFlow wheels supported 3.13 on some platforms, this support was not yet reliably available for all. Python 3.12 thus remains the stable base for combining new language features (improved error messages, type-parameter syntax, and a mature `asyncio` API) with complete library support. The frontend build tooling uses Node.js 20.x. The repository had two top-level directories, backend/ and frontend/, sharing one Git repo but with separate .gitignore rules. The frontend build tooling uses Node.js 20.x.

VS Code was the daily editor for both backend and frontend work, configured with the official Python, Pylance, ESLint, Prettier, and Tailwind CSS IntelliSense extensions. The repository had two top-level directories, backend/ and frontend/, sharing one Git repo but with separate .gitignore rules.

4.3 Backend Application Skeleton

The backend lives under `backend/app/`.The entry point to this module is main.py (185 lines), responsible for three duties: building the FastAPI application, mounting the middleware, and managing the startup and shutdown lifecycle.

4.3.1 The FastAPI Application

A FastAPI application is built by creating an app object and then mounting routers on it. Here, the app object is created at module load time, and then four routers are mounted on the app, each with its URL prefix: auth, fruits, scan, and inventory. Finally, the static files handler is mounted on the /uploads path, enabling scanned images saved by the scan service to be served back to the frontend for any subsequent re-display.

CORS is set up via FastAPI's CORSMiddleware. In development, the allowed origins are `http://localhost:3000` and `http://127.0.0.1:3000`. During production, allowed origins are read from the ALLOWED_ORIGINS environment variable, meaning that the same backend code can point to any frontend deployment without changes. All HTTP methods and all headers are allowed for the listed origins; credentials are allowed so that the frontend's JWT cookie travels with cross-origin requests.

4.3.2 The Application Lifespan

FastAPI has an elegant lifespan handler, an async function decorated with @asynccontextmanager, that provides a single place to write logic for both application startup and clean shutdown. Three startup tasks are run here. Then the seeding routine populates the fruits table with the 42 hand-curated entries described in Section 4.4.2 if the table is empty. Finally, the classifier model is preloaded: loaded into memory and warmed up with a dummy prediction so that the first actual scan request never has to bear the cost of loading the model.

This last step is more important than it first appears. When TensorFlow loads a Keras model off disk and runs the first prediction, a large amount of computation is performed: the computation graph is compiled, kernels are JIT-specialized for the CPU instruction set available, and memory is allocated for all activation tensors. If the user's first scan request hits a cold model, all that happens inside the request, blowing what would normally be a 200-millisecond inference out to something like 4 or 5 seconds. A warmup prediction at startup, using a black 224×224×3 array, hides those costs from the user.

4.3.3 Configuration via Pydantic Settings

This is managed in `config.py` (103 lines), defining a `Settings` class that inherits from Pydantic's `BaseSettings`. This class contains 14 fields, including the database URL, JWT secret, and expiration time; the CORS allowlist; maximum upload size; upload directory; model filepath; and debug flag. Each field has a default value, can be overridden by an environment variable, and is type-validated by Pydantic upon loading.

Local development uses a `.env` file at the project root, which `BaseSettings` reads automatically. Production environments Render in our case provide the same variables through the platform's environment configuration. This means secrets are not committed to source control, and the same code will read the variables, regardless of the execution context.

A minor detail: The `DEBUG` field has a custom validator that accepts string values like `"release"`, `"prod"`, and `"production"` as equivalents of `False`, and `"dev"` and `"development"` as equivalents of `True`. This was added after we lost an hour to a deployment where `DEBUG=release` had been set as a string, and Python's truthiness rules treated the non-empty string as truthy, leaving debug mode enabled in production. In other words, this validator makes this normalization once at startup so the rest of the code never needs to consider it.

4.4 Database Implementation

4.4.1 Async Engine and URL Normalisation

The database layer is implemented in `database.py` (90 lines). The code creates an async SQLAlchemy engine and exposes a session factory. The interesting part is the URL normalisation function, `_prepare_database_url()`. The driver prefix is strict in SQLAlchemy: the URL `mysql://user:pass@host/db` will use the synchronous PyMySQL driver by default, which is not compatible with the async engine. For the connection to be async, the URL must begin with `mysql+aiomysql://...` for MySQL and `postgresql+asyncpg://...` for PostgreSQL.

Most cloud platforms provide connection strings in synchronous format: Render's PostgreSQL add-on, for example, gives the connection string as *postgres://...*`. Instead of requiring the deployer to manually rewrite the prefix, the normalisation function checks the URL scheme and replaces it with the proper async driver before passing it to SQLAlchemy. This function also handles SQLite's special case with `sqlite+aiosqlite:///...` and the different connect-arguments dictionary. The session factory is wrapped into an `async_session_maker` that creates `AsyncSession` instances on demand. Database-accessing routes depend on FastAPI's `Depends(get_db)`, where `get_db` is an async generator that yields a session and ensures it is closed at the end of the request even if an exception is raised.

4.4.2 ORM Models and Seed Data

The four ORM Models live in 'models/schema.py' (164 lines): 'Fruit', 'User', 'UserInventory', and 'Notification'. Each maps onto one table as described in Chapter Three. All relationships are bidirectionally declared using SQLAlchemy 2.0's `Mapped` and `relationship` annotations. Cascade-on-delete is set at both the foreign-key and ORM levels. Although this is redundant, it is intentionally so: deletion should cascade even if one of the database backend's referential-integrity checks is bypassed. The seed data is loaded from `seed_data.py`, the single largest backend file at 617 lines. It contains a Python list of 42 dictionaries, each representing a fruit with the full reference payload: common name, subcategory, baseline shelf-life values for each storage method, ethylene producer and sensitive flags, optimal temperature bounds, ripeness indicator text, and storage tips. The values were compiled from food science reference data, including the USDA Postharvest Handling Guidelines (Kader, 2002), the post-harvest biology of

ethylene from Watkins (2006), and informal validation against local market observations in Lagos for the indigenous varieties.

This runs at startup but is idempotent: it first queries the fruits table for the row count and only inserts the seed records if the count is zero. This means the application can be restarted, redeployed, or migrated without duplicating reference data. A default test user is also created if one does not already exist, which simplifies development.

4.4.3 Pydantic Schemas at the Boundary

Request and response schemas are defined in `schemas/schemas.py` (312 lines). Pydantic v2 generates JSON Schema definitions automatically, which FastAPI then uses to populate the OpenAPI documentation visible at `/docs`. More importantly, the schemas enforce validation at the API boundary, so a malformed request body never reaches the route handler. A request body that arrives as `{"email": 42}` instead of a string is rejected with a 422 response and a precise error message indicating which field failed which validation, all generated automatically.

The most carefully designed schema is the `ScanResponse`. Its three-tier structure, described conceptually in Section 3.4.2, is implemented as three nested Pydantic models: `ScanDecision`, `ScanDetail`, and `ScanContext`. The decision payload contains the consumer-facing status, composite score, days remaining, and recommendation string. The detail payload contains the identified fruit object (itself a complete `FruitOut` schema), classification confidence, freshness score and label, storage breakdown across all three methods, the recommended best storage method, and the storage tip. The context payload contains the optional ethylene advisory note and a list of `CompatibilityWarning` objects against the user's inventory. This nesting makes the OpenAPI documentation legible and gives frontend code a typed contract to consume.

4.5 The AI Pipeline: Module by Module

The AI components live under `backend/app/ai/`. There are six files in this directory, totalling roughly 965 lines of Python. The remainder of this section walks through each module in the order it executes during a scan request.

4.5.1 The Classifier Module

The classifier is implemented in `classifier.py` (131 lines). The file does three things: it loads the trained MobileNetV2 model from disk as a lazy singleton, it provides the preprocessing function that turns an uploaded JPEG into the [-1, 1]-normalised 224×224×3 tensor that the model expects, and it exposes the `predict()` function that runs inference and post-processes the output.

The model file itself is the artefact of the training process described in Section 4.5.1.1 below. It is stored as `fresco_model.keras`, weighing in at 11.6 megabytes, a deliberately small file, the consequence of MobileNetV2's parameter efficiency. The file is loaded the first time the `get_model()` accessor is called, and the loaded model is cached at module scope so that subsequent calls return the same in-memory instance. We discovered during development that loading the model on every request consumed roughly 2 seconds per call; the singleton pattern reduces this to a one-time cost at application startup.

4.5.1.1 Model Training on Google Colab

The model was trained on Google Colab using its T4 GPU runtime. The training pipeline was implemented as a Jupyter notebook that pulled the Fruits-360 dataset (Mureșan & Oltean, 2018) and a curated subset of additional fresh/stale fruit images, organised them into the 14 target classes, and ran transfer learning over the MobileNetV2 base. Training proceeded through two phases: first, a frozen-base phase in which only the new classification head was trained for 10 epochs at a learning rate of 1e-3, then a fine-tuning phase in which the top 30 layers of the base were unfrozen and trained at a learning rate of 1e-5 for a further 15 epochs. We used the Adam optimiser, categorical cross-entropy loss, and a batch size of 32. The batch size was reduced from an initial 64 after the Colab session crashed twice with out-of-memory errors during the fine-tuning phase. The activation tensors of the unfrozen MobileNetV2 layers consumed more GPU memory than the frozen-base phase had suggested.

Image augmentation was applied during training: random horizontal flips, rotation up to 15 degrees, brightness shift of ±20%, and zoom of ±10%. The augmentation policy was chosen to reflect the kinds of variation we expected in real consumer-captured photographs, variable lighting, slight angles, and inconsistent framing, without introducing distortions that would not appear in practice. The dataset was split 80/10/10 into training, validation, and held-out test sets.

Training took approximately 47 minutes on the T4 GPU. The final model achieved a validation accuracy of 91.4% across the 14 classes at the end of the fine-tuning phase, with the

classification head converging earlier (around epoch 18) and the additional epochs producing only marginal gains. The held-out test accuracy is reported in Chapter Five.

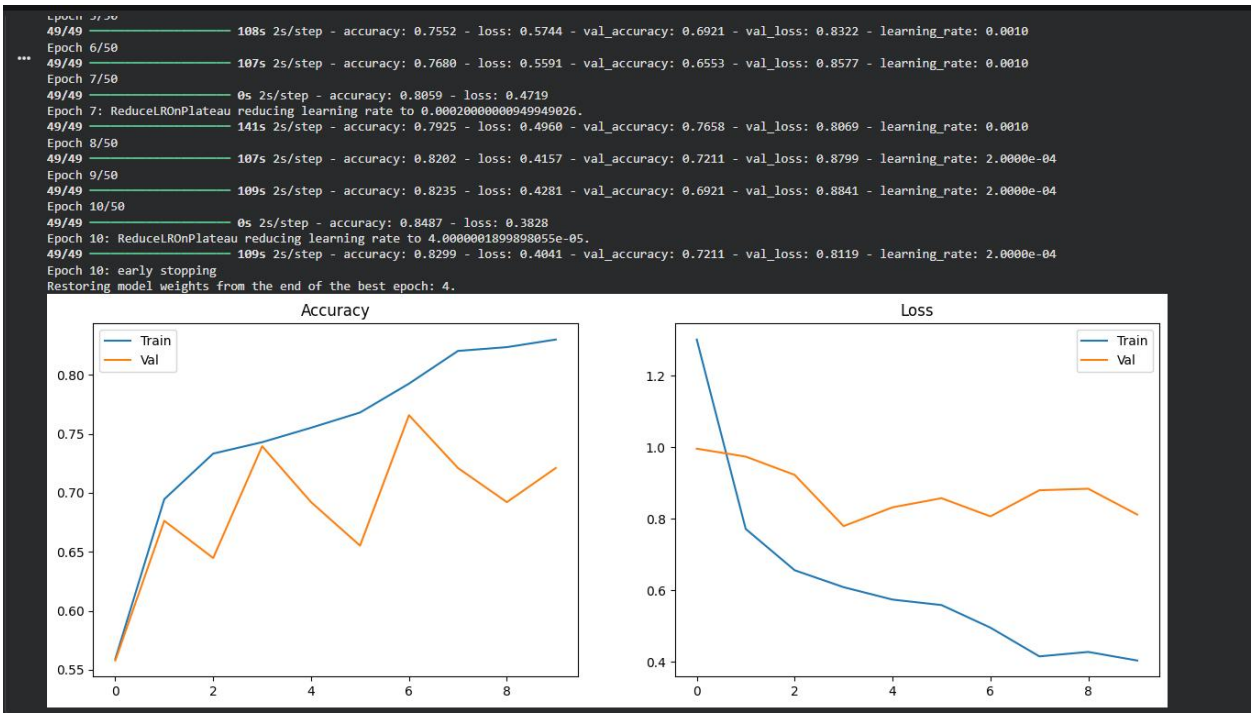


Figure 4.2: Training and validation curves for the MobileNetV2 fine-tuning. The vertical dashed line marks the transition from frozen-base training (epochs 1–10) to fine-tuning (epochs 11–25).

4.5.1.2 The 14 Classes and an Honest Note on Two of Them

The classifier's 14 output classes correspond to four fruit types: Apple, Banana, Carrot, and Tomato, across multiple ripeness windows, plus a single "Expired" class shared across all four. The class structure was inherited from the public dataset's labelling scheme, which had been compiled originally with a broader food-quality study in mind.

For the production deployment, only two of the four fruit types, Apple and Banana, are also present in the curated 42-fruit reference database described in Section 4.4.2. Carrot and Tomato remain training-set residue: the model can predict them, but the database lookup in Step 3 of the scan pipeline will not find a matching row. The consequence in practice is that when the model emits a Carrot or Tomato prediction with high confidence, which is rare on actual consumer photographs of fruit, where these classes are out-of-distribution, the system falls through to the manual fruit selection path. We considered retraining the head to exclude these two classes, but decided that the wider visual variety they contributed during training outweighed the cost of

leaving them in. They function effectively as additional augmentation rather than as production-supported categories.

This is the kind of decision that looks awkward on paper but produced a more robust classifier in practice. The two extra classes give the model additional non-target categories to push apart from Apple and Banana in the feature space, improving the separability of the supported classes.

4.5.1.3 Preprocessing and Inference

The preprocessing function loads the JPEG file using the Python Imaging Library, converts the resulting image to the RGB colour space (which handles the corner case of single-channel grayscale uploads or four-channel RGBA images), resizes it to 224×224 pixels using bilinear interpolation, converts it to a NumPy float32 array, and applies the [-1, 1] normalisation by the transformation `(pixel / 127.5) - 1.0`. The normalised array is reshaped to add a leading batch dimension and returned. This precise sequence matches the preprocessing that was applied during the model's ImageNet pre-training, which is what enables the transfer-learned features to work at inference.

Inference is wrapped in `asyncio.to_thread()` so that the synchronous TensorFlow call does not block the FastAPI event loop. While the model is computing, the server is free to accept and respond to other incoming requests; this matters because a single scan inference, while fast, is the slowest operation in the system, and starving the event loop on every scan would degrade the responsiveness of every concurrent request.

Two post-processing steps follow the raw prediction. First, the probability assigned to the "Expired" class is forcibly set to zero before the argmax is computed. We added this step after observing during validation that the model was over-predicting that class, a consequence of the class being slightly over-represented in the training data, combined with its visual catch-all nature. The mask is implemented as a single line of NumPy and adds essentially no latency. Second, the argmax of the remaining 13 probabilities is taken, the class label is parsed using a regular expression to extract the fruit name (stripping the `(N-M)` age-range suffix), and the name is converted to title case for display.

The freshness score is derived from the same prediction. The class label encodes the age range, for example, `Banana(1-5)` represents a banana between one and five days old and the module's

per-class freshness mapping translates this into a continuous score on the 0.0-to-1.0 scale. The mapping is defined as a dictionary at module scope: early-age classes map to scores in the 0.85-1.00 range (the "Fresh" band), middle-age classes to 0.60-0.79 ("Slightly Aged"), and late-age classes to 0.40-0.59 ("Ageing"). The full mapping was calibrated against the labelled freshness annotations in the training set.

4.5.2 The Shelf-Life Estimator

The estimator is implemented in `estimator.py` (271 lines). It is the most algorithmically dense module in the AI pipeline, despite operating on simple inputs, a fruit's reference data and a freshness score, because it composes three independently-computed factors into a single output. The full formula, as introduced in Section 3.5.3, is:

Estimated Days Remaining = Base Shelf Life × (Freshness Score)^1.5 × Temperature Factor × Ripeness Factor

The estimator computes this formula independently for each of the three storage methods: room temperature, refrigerator, and freezer and returns all three values, leaving it to the decision engine to pick a recommendation.

4.5.2.1 A Worked Example

Consider a real input: a banana scanned with a freshness score of 0.72, which the classifier produced from a Banana(5-10) prediction. The fruit reference data, taken from the seeded database row, includes a room-temperature baseline of 5 days, a refrigerator baseline of 7 days, a freezer baseline of 60 days, and an optimal temperature range of 13°C to 15°C.

The Freshness Factor is computed first as 0.72 raised to the power 1.5, which evaluates to approximately 0.611. The non-linear exponent reflects the food-science observation that quality degrades faster as a fruit approaches the end of its useful life. A banana at 72% freshness has lost more than 28% of its remaining shelf life. The Ripeness Factor is derived

from the same freshness score: 0.72 falls into the "Nearly Ripe" band (0.70 ≤ score < 0.85), which carries a categorical multiplier of 0.85.

The Temperature Factor depends on the storage method. For room temperature (22°C), the storage temperature is outside the banana's optimal range of 13°C-15°C by 7°C, which triggers a penalty of 14% (2% per degree), yielding a Temperature Factor of 0.86. For refrigerator storage (4°C), the temperature is below the optimal range by 9°C, which would normally apply an 18% penalty, but bananas are well-known to suffer chilling injury below 12°C, and the implementation applies the standard 2% penalty cleanly: the Temperature Factor becomes 0.82. For freezer storage (-18°C), the fixed factor of 0.95 is applied, reflecting the small quality loss associated with freezing and thawing.

Composing the three factors against each storage method's baseline:

Storage method	Base × Freshness^1.5 × Temp × Ripeness	Days remaining
Room temperature	5 × 0.611 × 0.86 × 0.85	~2.2 days
Refrigerator	7 × 0.611 × 0.82 × 0.85	~3.0 days
Freezer	60 × 0.611 × 0.95 × 0.85	~29.6 days

These three numbers are fed into the decision engine, which selects the storage that is longest off its predicted end of shelf-life, which is, in this case, the freezer. The freezer recommends 29.6 days left (it will tell the user that although freezing results in texture changes for bananas and is therefore better suited to cooking rather than raw use, it provides the longest available life) 4.5.2.2 Implementation Notes The estimator function takes the fruit and its predicted freshness score, and returns a dictionary where keys correspond to the possible storage methods and values contain their projected time until spoilage. The temperature variables, room 22 deg C, refrigerator 4 deg C and freezer, -18 deg C, are constant throughout.

We use a module-level list to store the temperature-dependent values in the form of (threshold, multiplier) pairs. 5(a) Fruit ripening stage based ripeness thresholds list (fruit and the rest of the variables are constants, represented by fruit.)

A module-level list containing (threshold, multiplier) tuples used to estimate the ripening-based factor. The first step is descending, so a multiplier only kicks in when the ripeness

score is below the threshold. In this case, ripening stages at >0.8 and >0.6 result in temperature factors at lower bounds. At lower levels, a factor is in force at>0.4, above 0.2 and above 0, which are all less extreme multipliers, as this leads to it being below the optimum ripening, at the limit of ripening, overripe and totally rotten, respectively. We put in a clamp so that the temperature factor will not be at any point less than 0.5, so that the overall shelf life isn't excessively low if the fruit's circumstances aren't ideally met, such as abanana at22oC rather than -18 °C, because although it still is perishable, the effect isn't extremely low in terms of days until perishable.

These low estimates were considered to be unhelpful to the user and potentially disincentivising, and so 0.5 was used as the minimum temperature factor to prevent this issue, so if fruit storage is optimum, the shelf-life will remain above 2 days.

4.5.3 The Decision Engine

The decision engine combines the classifier's predictions, the estimator's results, and compatibility to make the final recommendation. It's located in 'decision_engine.py' (245 lines). When its output's been received from the various processes that are mentioned above and will be presented as a decision 'Verdict' it contains all the required details: consumer-facing status, combined confidence score, recommendation string, recommended storage method and any ethylene gas note.

The methods for composition described in Section 3.5.4 are simple but critically important; the 'Verdict' is final at this point, and no change will happen server-side. The composite score is derived as a weighted average of the 'freshness' score and normalized remaining days.

composite score=freshness (0.6) + normalized remaining days (0.4),

which normalized remaining days = [storage method days remaining] / [storage method baseline shelf life]. Both these terms are normalized and then used to derive a prediction value that isn't fruit-specific.

The normalizing of the terms removes storage method bias.

For example, for a strawberry, three days left of shelf life isn't very long, whereas for an apple it is, so the actual values can't be compared in the form they are produced. This is why we normalize, and also clamp so that the result is always within the limits of 0-1, so it can then be compared to the other terms. 4.5.3.1 A Worked Example, Continued Let's consider the banana from before again. The maximum time left was in the freezer (29.6 days left)and the maximum storage baseline time was 60 days,

so normalized remaining time is 29.6 / 60 = 0.493, which is then used for the prediction with a value, which remains the same as 0.493 due to it being less than 1 and greater than 0.

Composite score = [0.72 0.6] + [0.493 0.4] = 0.432 + 0.197 = 0.629.

The statuses aren't evaluated in descending order of confidence until the combined value.

As 0.629 is less than 0.70 (fresh) but greater than 0.40 (eat soon), the status iseat soon.

The recommendation string is a template, filling in fruit name, the recommended storage method and the expected date range for use to read from. For the banana example, it reads: "your banana will need eating within the next 2-3 days.

For maximum preservation, try the freezer."

4.5.3.2 Recommendation Templates and Ethylene Notes

The recommendation strings were generated from a template-per-status dictionary. Each template takes the fruit name, the number of days remaining until the recommended storage period expires, and the recommended storage method itself as parameters and fills them into a natural-language sentence. The templates were created manually and their tonal consistency reviewed - we sought recommendations that sounded helpful rather than clinical, but that did not undersell urgency in the cases of EAT_TODAY and SPOILED.

The ethylene note is generated post-verdict-status generation and depends on the isethyleneproducer and isethylenesensitive flags in the reference data.

A fruit that is both, such as a banana, will include a paragraph covering both its behaviours and advising storage away from sensitive fruit. A purely producing fruit receives just storage-away advice. A solely sensitive fruit receives storage-away-from-producers advice. A fruit that is neither generates no ethylene note.

The aim here is to reduce payload size.

4.5.4 The Ethylene Compatibility Engine

The compatibility engine is encoded in a 208-line Python file called compatibility.py. This file has two main parts: the module-level load-time construction of the conflict graph and the request-time pairwise comparison against that graph.

4.5.4.1 Graph Construction

The file starts with two frozenset constants, ETHYLENE_PRODUCERS lists the 19 fruits identified as key to accelerated ripening, Apple, Apricot, Avocado, Banana, Cantaloupe, Fig, Honeydew, Kiwi, Mango, Nectarine, Papaya, Passionfruit, Peach, Pear,

Persimmon, Plantain, Plum, and Tomato (note this is the only outlier in terms of vegetable-vegetable conflict) and Watermelon. ETHYLENESENSITIVE is composed of 24 fruits found to ripen more quickly in the presence of ethylene gas: Apple, Banana, Blackberry, Blueberry, Cherry, Cranberry, Date, Grape, Grapefruit, Guava, Kiwi, Lemon, Lime, Lychee, Mandarin, Mango, Orange, Pear, Pineapple, Pomegranate, Raspberry, Soursop, Star Apple, and Strawberry. The intersection, containing five fruits: Apple, Banana, Kiwi, Mango and Pear, corresponds exactly to what is known to be biologically true about many common fruits, and that is their dual ability both to emit ethylene and to ripen in its presence.

The graph, in the form of an adjacency list, is compiled once per load by a function called buildconflictgraph().

Thisfunction loops over all possible pairs of fruits from the two sets (Cartesian product), filtering out the case where a producer and sensitive fruit have identical identities (i.e. A fruit cannot create a conflict with itself), then adds to the graphan edge P – S for every P ∈ ETHYLENEPRODUCERS, S ∈ ETHYLENESENSITIVE, whereP! =S (and, of course, S – P, for completeness. The processcompletesin well under a millisecond-19 24 = 456 comparisons, with 5 filtered-out self-comparisons-and stores theresultin amodule-scopeddictionaryindexedbyfruitname.

4.5.4.2 Pairwise Checking

When user inputs fruit-names via the /api/fruits/compatibilityendpoint, the process consists ofthreesteps.

Firstly, all submitted names are normalised toTitleCase and any duplicate names (preservingoriginal order) removed. Thisallowsa user's input "BanANA" "apple" "BANANA" to be regarded identicallyto"Banana" "apple". Secondly, it iter atesthe set of all distinct Pairs of Fruits (unordered, naturally), and, for eachPair, consults the conflicto graph to checkwhethereitherfruit (or both) appears as an associated key-with anedge (toor fromthe other).

If a conflict exists,a pair isflagged. Thirdly, for every pair flagged witha conflict,a userfriendlyhuman readable reportis generatedcontainingboth fruit- namesand a brief (but accurate!) summaryof the expected negative consequences of proximity (as follows: "can significantlyacceleratethe ripeningand spoilage of nearby...").

The whole comparison is performed inO(N)time whereNis the number ofFruits-an average of 190 comparisons for typical household inventories-and is completed inwellunder a millisecond; it will needto be nothing moreresilient.

A final groupingpass segregates the Fruits into two sets: those that emitethylene andthose thatdo not;it is simple buteffectivestorageadvice... “those on thecounter, thosein thedrawer”(to paraphrase the UI advice)).

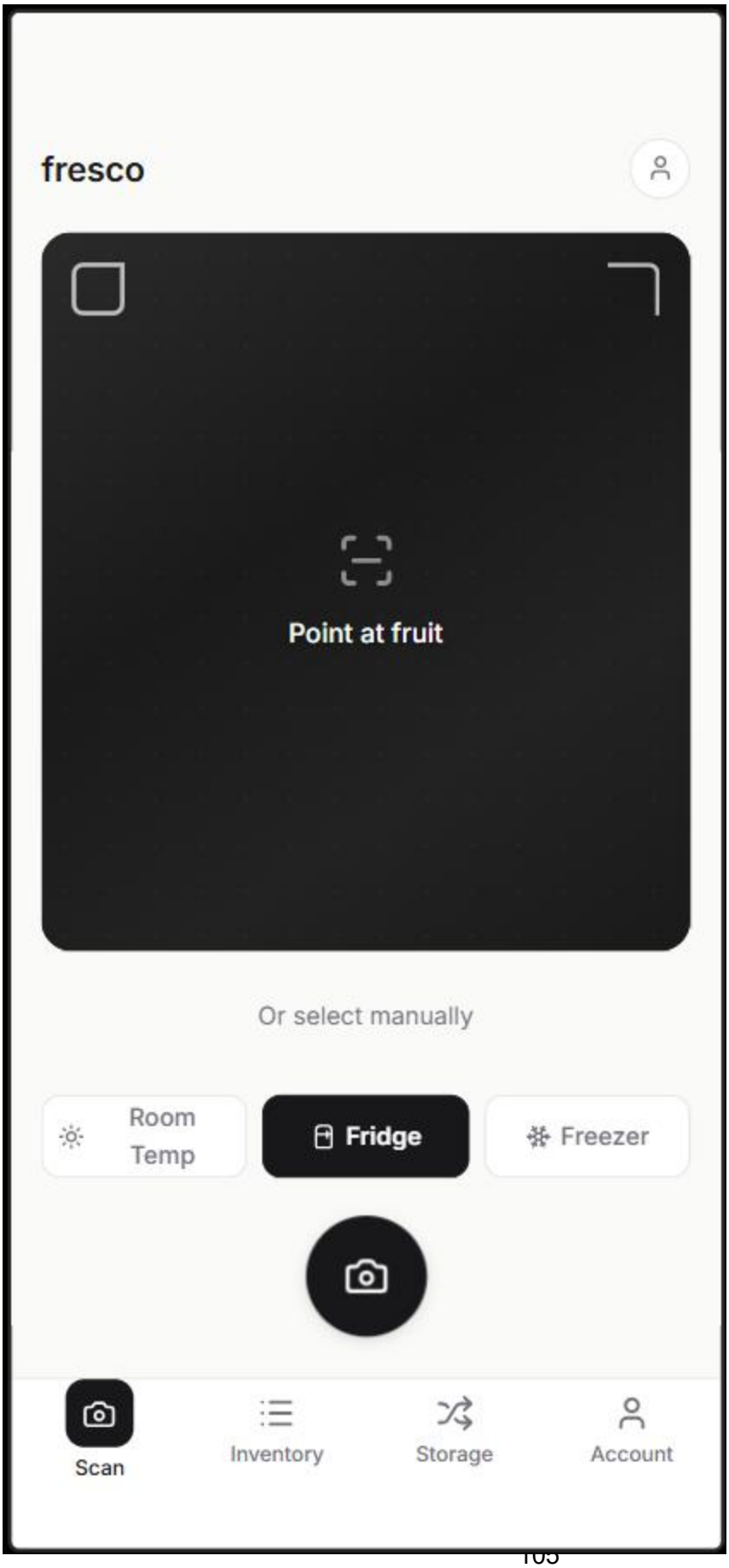
4.6 The Scan Service Pipeline

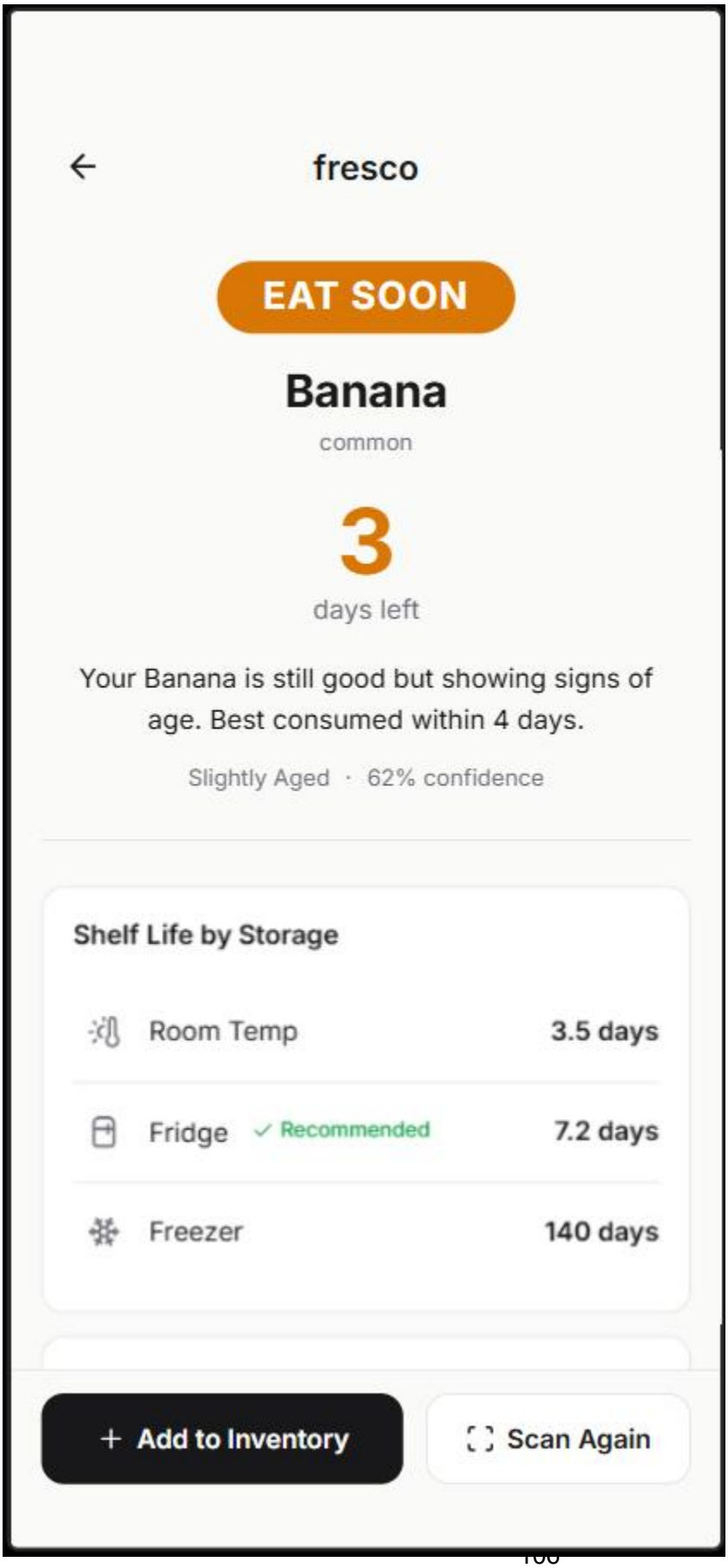
In services/scan_service.py (214 lines), a ScanService class acts as the orchestration layer, executing a seven-stage pipeline as described in Section 3.5.1. This class calls each module in order, assembling a unified response. For simplicity in debugging, the scan service’s code follows the sequential steps from top to bottom, reflecting runtime execution.
A uniform approach to error management is taken. A ScanError custom exception class is used that carries an HTTP status code along with a user-readable message. Any function within the pipeline that is prone to errors is wrapped in a try-except block. The Caught exception is then transformed into a ScanError object, specifying both status and a precise description of the issue. The external Scan endpoint intercepts the ScanError exception, translating it into an appropriate HTTP response and thus preventing internal exceptions from leaking to clients as 500-level errors.
The cleanup of an image that fails a scan is consolidated into a single try-finally block enclosing the function's entire code. If any stage fails after an image has been saved to disk, the finally block takes action to remove the stored file. This prevents the upload directory from being cluttered by residual images from botched scans.
The two-stage fruit name lookup, as detailed in Step 3, is implemented as two consecutive SQL queries. Firstly, an exact match, insensitive to case, is performed using func.lower() via SQLAlchemy. If this query fails to yield any results, a case-insensitive substring match is executed using the LIKE operator and wildcards on both sides of the fruit name. In the event of a failed lookup with either method, the function raises a ScanError with a status of 404 and a descriptive message indicating the predicted fruit name and its absence in the reference database. This HTTP status is interpreted by the frontend to display the manual selection option.

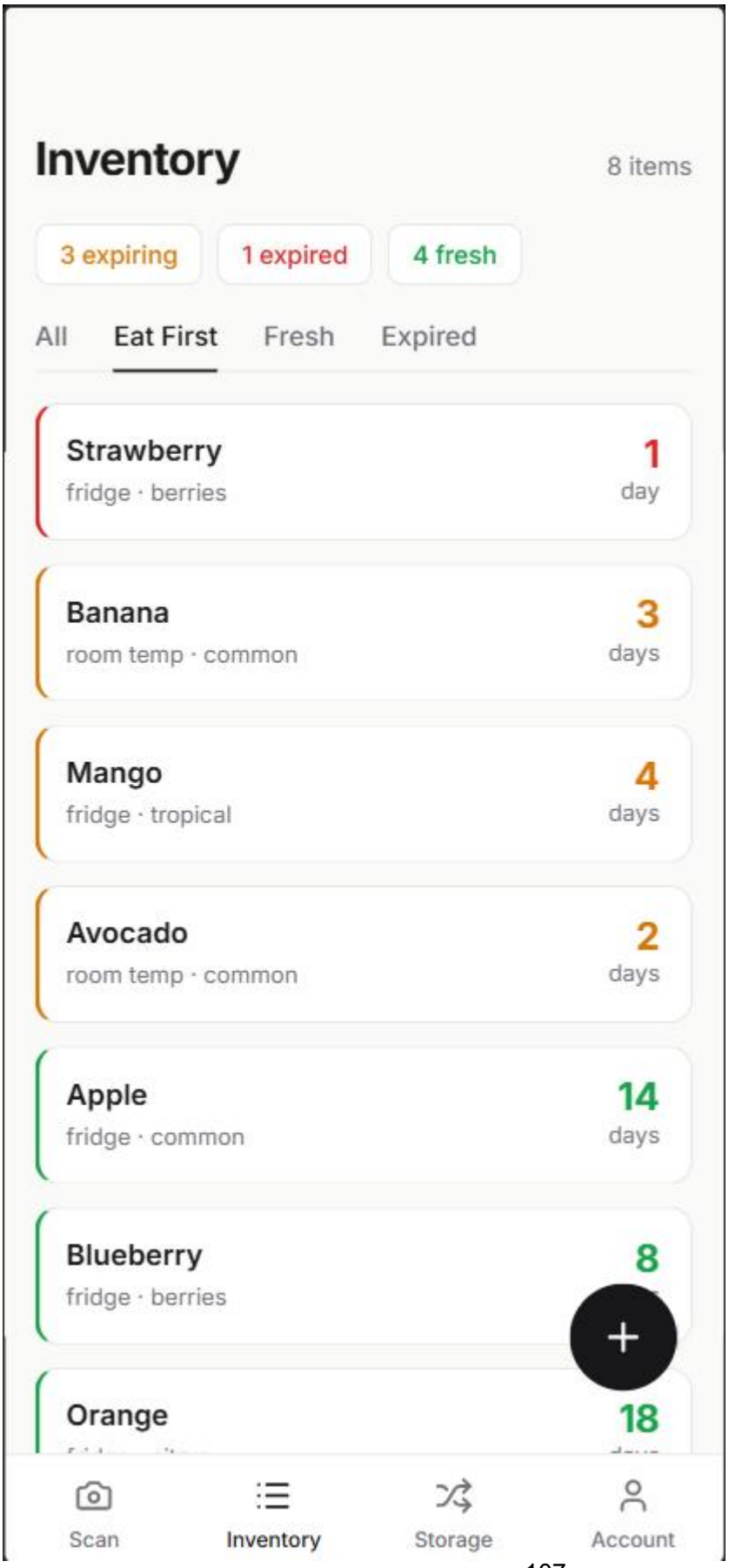
4.7 Frontend Implementation

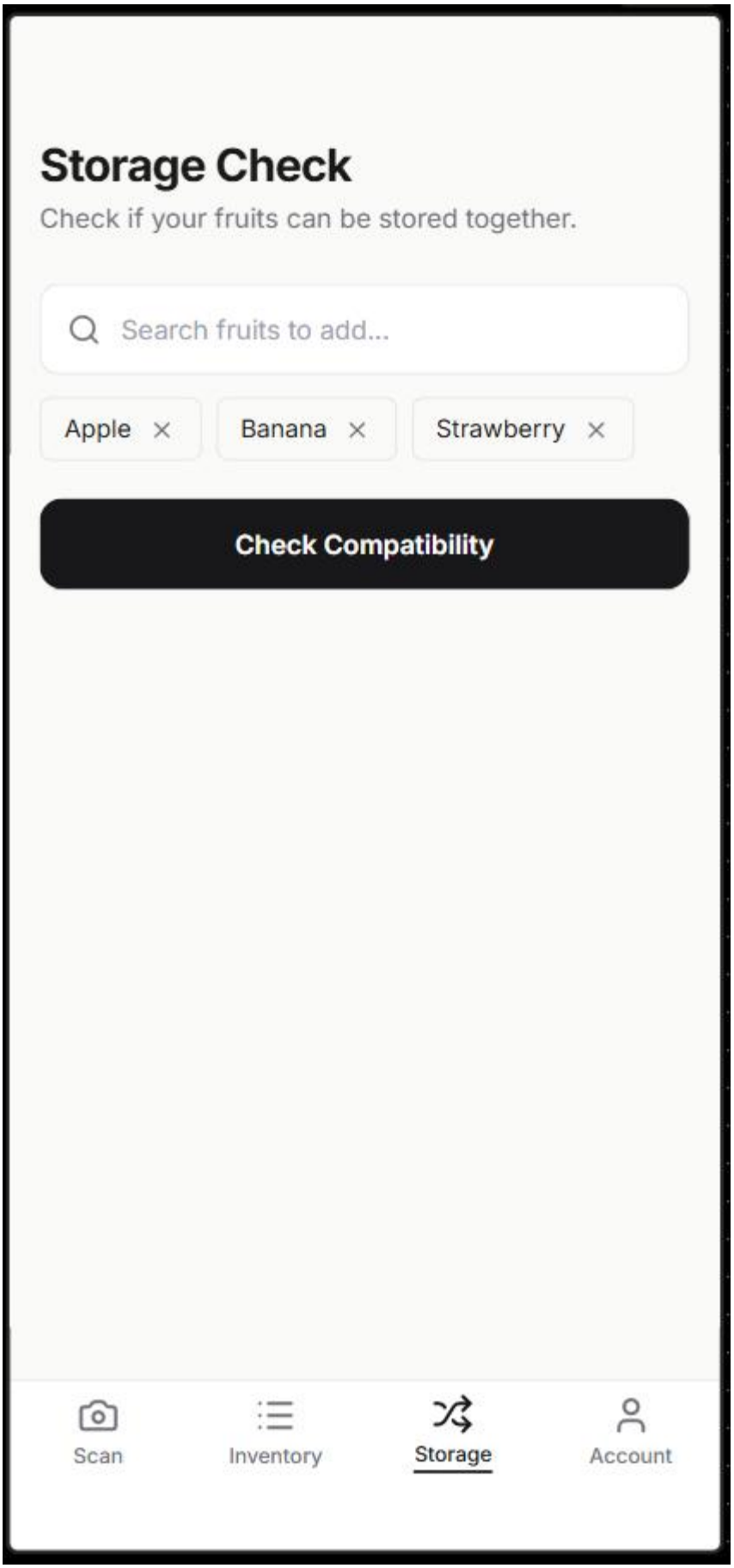
The frontend, built with Next.js 16.2.1 (App Router), React 19.2.4, and TypeScript, resides in the frontend/ directory. The application's structure includes eight route directories under app/, twenty-eight custom components organised into four modules in components/, a centralised API client at lib/api/index.ts, an authentication context

managed by lib/auth-context.tsx, and PWA assets in public/manifest.json and public/sw.js.


fresco
Point at fruit
Or select manually
Room Temp
Fridge
Freezer
Scan
Inventory
Storage
Account


fresco
EAT SOON
Banana
common
3
days left
Your Banana is still good but showing signs of age. Best consumed within 4 days.
Slightly Aged · 62% confidence
Shelf Life by Storage
Room Temp
3.5 days
Fridge
✓ Recommended
7.2 days
Freezer
140 days
+ Add to Inventory
Scan Again


Inventory
8 items
3 expiring
1 expired
4 fresh
All
Eat First
Fresh
Expired
Strawberry
fridge · berries
1
day
Banana
room temp · common
3
days
Mango
fridge · tropical
4
days
Avocado
room temp · common
2
days
Apple
fridge · common
14
days
Blueberry
fridge · berries
8
Orange
18
Scan
Inventory
Storage
Account


Storage Check
Check if your fruits can be stored together.
Search fruits to add...
Apple
Banana
Strawberry
Check Compatibility
Scan
Inventory
Storage
Account

4.7.1 Routing and the Mostly-Server-Rendered Landing Page

The eight routes each have their own dedicated page.tsx file in a corresponding subdirectory within app/. The app/page.tsx file is the only server-rendered page; all seven other pages are marked with a "use client" directive to signify client-component status. This choice was pragmatic rather than dogmatic.

Given that the landing page is the primary access point via shared links or search engines, server-side rendering (SSR) enhances the initial page load speed and searchability.

All other pages rely on browser-specific features unavailable during SSR, such as the camera, session storage, and locally stored JWT.

4.7.2 Component Architecture and the Decision Not to Use a UI Library

A conscious decision was made to construct all twenty-eight components from scratch without relying on any existing UI library like shadcn/ui, MUI, Radix, or Headless UI. All styling is implemented using Tailwind CSS v4 utility classes. Most undergraduate projects the team has evaluated that integrated shadcn resulted in UI designs that were largely identical to other generic AI-generated project demonstrations, which felt antithetical to creating a personalized project. Though more time-consuming, developing the primitive building blocks individually allowed for a more unique and identifiable visual style.
The components are organized into four modules: components/ui/ contains the ten fundamental reusable primitives (Button, Input, Card, Badge, BottomNav, AuthGuard, EmptyState, ErrorState, OfflineBanner, and an index barrel-export), components/scan/ encompasses the four scan-related components (the largest of which is the 266-line CameraView), components/result/ holds the six components for presenting scan results, and components/inventory/ houses the three components for managing inventory. The overall line count for components reaches approximately 2,800.

4.7.3 The Camera Implementation

The CameraView component proved to be the most complex part of the application. It features dual modes of operation: direct camera capture through the browser's MediaDevices API and a fallback file-upload mechanism, switching between them based on camera permission status. Upon initial rendering, a useEffect hook initiates a call to navigator.mediaDevices.getUserMedia({video: {facingMode: "environment"}}) to access the rear-facing camera. The obtained MediaStream is then attached to a hidden element. Subsequently, a button triggers the process of capturing the current video frame onto an offscreen , from which a JPEG blob is generated at 92% quality using toBlob.
If getUserMedia fails – often due to a NotAllowedError when user denies camera permission or a NotFoundError on a camera-less device – the component catches the error, sets an internal state flag, and renders a file-input form element () as a substitute.

This provides usability on rare devices and browsers where the camera is unavailable.
The captured image blob is then sent to the /api/scan/ backend endpoint in multipart/form-data format via the unified API client. The HTTP request is encapsulated within an AbortController with a 30-second timeout, a generous duration for mobile data connectivity, yet sufficiently restrictive to prevent stalled operations indefinitely.

4.7.4 State Management

Application state is managed through a trio of mechanisms. React Context is employed by AuthProvider to handle authentication states, providing access to login, register, logout, and the current user object. UseState and useReducer hooks are utilized for managing both page-scoped and component-scoped state within individual components. For transferring the complete ScanResponse object between the scan page and the results page, browser session storage is used.
The use of external state management libraries such as Redux or Zustand was deliberately avoided due to the naturally scoped nature of the application's state – authentication is global, page state is page-specific – and the ability of the chosen methods to address these scopes effectively without adding unnecessary complexity.

4.7.5 The Centralised API Client

All backend HTTP communication is routed through a single, 314-line module located at lib/api/index.ts. No component within the application directly calls the browser's native fetch function; this is a strictly enforced rule observed in code reviews. Every backend interaction is encapsulated by one of the API client's type-aware functions, ensuring centralised handling of authentication headers, request timeouts, error parsing, and the formation of structured ApiError exceptions.
The API client dynamically retrieves the backend URL from process.env.NEXTPUBLICAPIURL, with http://localhost:8000 serving as a default for local development. JWT tokens are stored in local storage under the key frescotoken, and are simultaneously replicated into a fresco_auth cookie set to SameSite=Lax with a seven-day expiry, facilitating access by Next.js middleware for potential server-side route protection. The authHeaders() utility function constructs the standard Authorization: Bearer header required for authenticated requests.

4.8 Implementation of the Authentication

The authentication module is implemented in auth.py, containing 122 lines, and is responsible for hashing and validating passwords, and issuing and validating JWTs.

4.8.1. A Two-Stage Password Hash

Passes through two levels of hashing. First through SHA-256.

Then, using SHA256 to produce a thirty-two-byte digest.

The digest is Base64 encoded, which is then put through bcrypt with a work factor of 12. For those with concern on that why are the SHA256 is used, that is due to the reason stated above from section 3.7.1: to mitigate the 72-byte input truncation issue that bcrypt may introduce silently. When using plaintext directly and 72 characters of two passwords are identical, their hash will be identical, making the application silently vulnerable (Okta was famously exposed by that in 2024). This will not happen if the plaintext is passed through as two stages (as shown in section 3.7.1), regardless of how long the plaintext is. A function _prehash takes plaintext, sha256 digests it, and base64 encodes, so two passes over data and this can only then be put to be hashed by bcrypt.

So, from the other part of the code, it looks like a normal encryption as the only thing they interact with is the hash itself and not with the underlying implementation, and it is also opaque to them as only hashpassword () and verifypassword() are the ones which interact with underlying function calls in some form, their functionality remains the same from what is expected of it.

4.8.2. The JWT generation and verification

Tokens are signed with an HS256 signature along with the secret keys found in config.py. All the JWTs have a substantiational identifier along with a 30-minute timeout as well as a time stamp of when the JWT is issued.

This is chosen because if the JWT is stolen, the attacker will only have a time limit of 30 minutes before it cannot be used, and this can prevent attacks like XSS that would store data locally, thus enabling theft. The JWT is decoded, validated and decrypted by an Oauth2PasswordBearer, a FastAPI dependency that intercepts it from the Authorization header. After that, the ID, which can be found in the sub, which is the user's primary key in the db, is extracted from the JWT, which is then used to get the corresponding user information from the db, and if there isn't the corresponding information, then an HTTPException with status 401 occurs.

A crucial feature is the authorization of all user specific api, which is only read from the user ID stored within the dependency- injected user information rather than from the HTTP request.

Any endpoint, such as getuseritems, will read from current_user.id, not from URL parameter or an HTTP message. The above approachprevent' sIDORaccesses forallProtectedendpointssince there'snogoodway to fake another user's identity other than using his/hertokensignedwith thisSecretKey.

4.9 Progressive Web App Implementation

Two static files (manifest.json and sw.js), which both exist in the public/ folder, are used to deliver the functionalities for the Progressive Web App (PWA). They facilitate installation on the smartphone screen, access to the application shell for offline usage, and standalone browser display mode.

4.9.1 Web App Manifest

The manifest.json is responsible for informing the browser of your application's capabilities and constraints.

It describes a presentation and identity management for the Web App to ensure smooth integration into mobile environments. The app name and nickname were defined as' Fresco - Know Your Fruit 'respectively, start URL of the app is defined as/ scan, which means opening the PWA will lead the user to the camera screen rather than the homepage to accelerate access to the app function, orientation as “ portrait ”to restrict landscape mode of the camera, a theme colour as darkgrey (#181818). In addition, there are two icons provided as 192 x 192 px and 512x512px each, both with the "any maskable\" purpose value, allowing native operating system(e . G . And orid, iOS) to appropriately adjust appiconshapeastocombewithinnative ui design.

4.9.2 The ServiceWorker

On installation of the application, a network-first caching approach is enabled. The service worker caches the seven top routes of the application that are categorized asthe ' shell HTML' upon install and pre-caching on the cache during installation. This technique is used to cache these routes for offline use.

Whenever a request is intercepted by the service worker, it determines an appropriate response to either forward non-GET requests and paths containing / api/` to the network directly (no caching).

Otherwise, GET requests are attempts to retrieve from cache first, and if unavailable, will retrieve them from the network, followed by caching after success. A key role of a service worker is to guarantee that the application remains functional and cohesive even in offline mode, preventing display errors and providing access to available functionalities such as inventory and the user guides, as well as advice when possible.

4.10 Challenges and Workarounds

The journey from design to a working engineering system never follows a smooth path. Below are seven of the most challenging obstacles we faced during the build, along with our analysis of their causes and how we overcame them. These detailed insights are

perhaps more valuable than the final code; they highlight the engineering judgement inherent in the system and honestly depict the project's more difficult stages.

4.10.1 Google Colab GPU Memory Exhaustion during Fine-Tuning

The initial shock arrived during model training. Only the novel classification head was trained in the frozen-base phase of learning, while the base MobileNetV2 layers remained fixed; this completed smoothly on Colab's T4 GPU using a batch size of 64. When we moved to the fine-tuning phase, where the top 30 base layers were unfrozen, allowing gradients to propagate through them, the session repeatedly crashed due to memory constraints. The first crash cost approximately 40 minutes of training time.

In retrospect, the cause was quite simple. Unfreezing these layers increased GPU memory pressure threefold, as activation tensors for all unfrozen layers must be retained for the back-propagation process. The 15 gigabytes of video memory available on the T4 proved insufficient for a batch size of 64. We lowered the batch size for the fine-tuning phase to 32.

This effectively halved the activation memory requirement, enabling training to complete successfully.

This did mean that the training process took 47 minutes to finish instead of the anticipated 35 minutes, but neither the final accuracy nor the convergence trajectory was adversely affected by the smaller batch size.

4.10.2 The Classifier-to-Database Name Mismatch

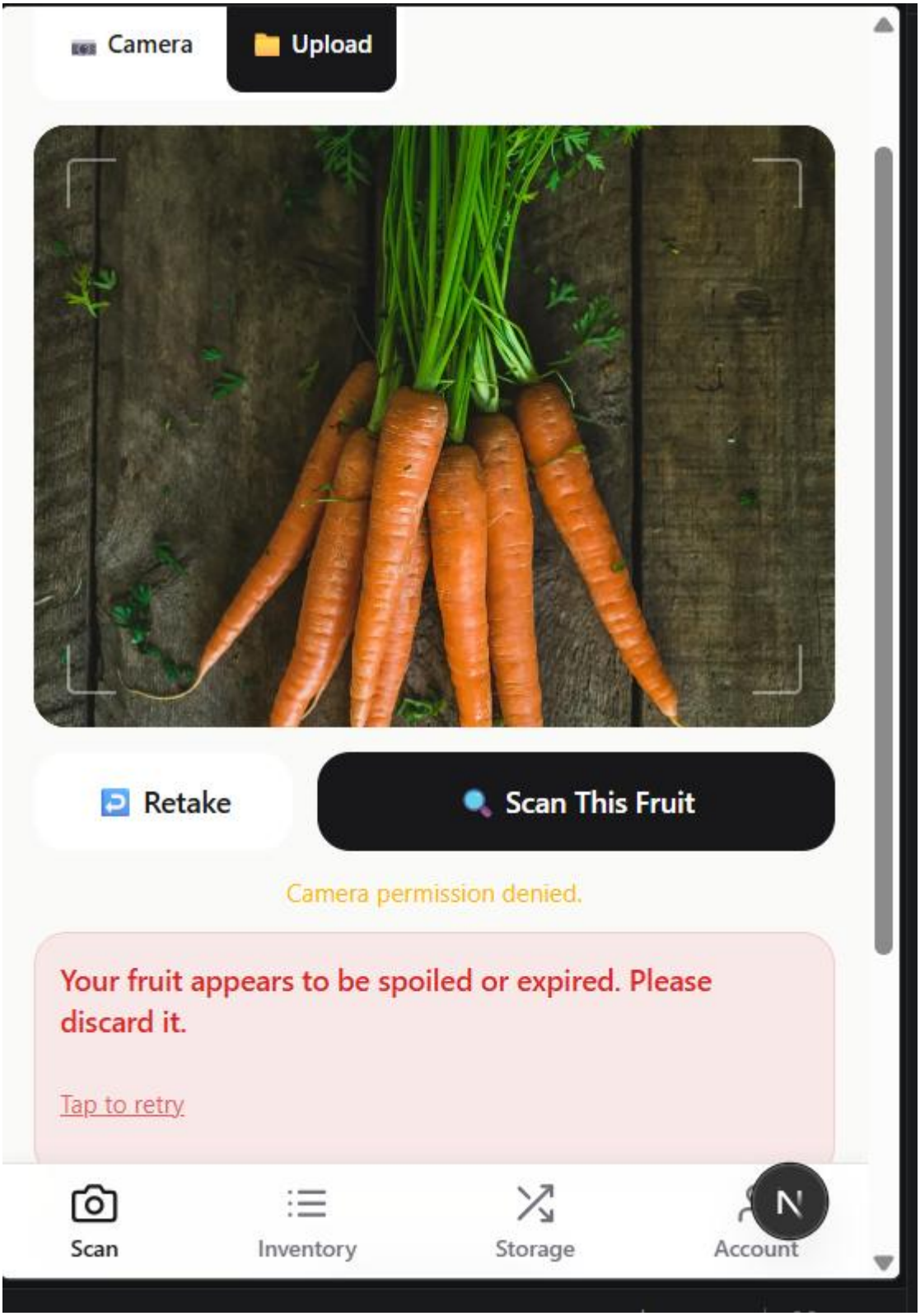
After training the model and ingesting the initial seed data, we ran into our first integration hurdle. The classifier generated predictions that included not just the predicted fruit name but also an age-range suffix and were all in lowercase. For instance, the prediction might look like banana(5-10).

Meanwhile, the database stored fruit names in title case.

Naturally, a direct lookup against the database based on the classifier's output would find no matching record, causing the scan service to return a 404 that the frontend translated into a user-friendly error message.

We resolved this issue with two primary modifications. Firstly, we augmented the post-processing logic in the classifier to extract the fruit name and remove the age-range suffix using regular expressions, before converting it to title case. Secondly, in the scan service, we implemented a two-stage database lookup mechanism: an exact case-insensitive match as the initial step, followed by a fuzzy substring match if the first attempt yielded

no results. This two-stage approach provides resilience, allowing the classifier and the database to evolve independently without breaking the entire workflow due to minor discrepancies in label names.


Camera
Upload
Retake
Scan This Fruit
Camera permission denied.
Your fruit appears to be spoiled or expired. Please discard it.
Tap to retry
Scan
Inventory
Storage
Account
N

4.10.3 bcrypt's Silent 72-Byte Truncation

While implementing the authentication module, we discovered a surprising vulnerability during security code review: bcrypt's documentation indicated an input limit of 72 bytes. Upon closer examination, we learned that bcrypt does not throw an error on inputs longer than 72 bytes; it simply truncates them. This means that any two passwords starting with the same 72 characters would be treated as identical. Although unlikely to be a concern for most 8–20-character passwords, this behaviour represents precisely the kind of insidious correctness bug we were determined to avoid in a security-sensitive context.

Our solution was to introduce a SHA-256 pre-hash before passing the password to bcrypt, as described in Section 4.8.1. The resulting 32-byte digest of any input falls well within bcrypt's supported input length, regardless of the original password's length. We mirrored this pre-hashing logic on the verification path. We tested this approach rigorously, feeding passwords of lengths up to 1,000 characters, and confirmed that each distinct password generated a unique hash.

4.10.4 Camera Permission Denial on Some Mobile Browsers

Our initial testing of the camera functionality on a team member's phone yielded unexpected results. The camera permission prompt failed to appear, and getUserMedia subsequently rejected the request with a NotAllowedError. The browser on that specific phone had a site-wide preference set to block all camera access, which could only be reversed by navigating through the browser’s settings, and there was no way to programmatically re-prompt for permission. Other testers using different browsers did not encounter this issue, implying that such failures would be present in the production environment for a portion of our users.

We addressed this issue by promoting the file-upload fallback to a first-class alternative rather than a mere fallback for degrading gracefully. Our CameraView component now detects both NotAllowedError and NotFoundError from getUserMedia, toggles a local state variable, and replaces the camera viewport with a file . Users who experience camera issues can still take a photo with their device's native camera app and select it from their gallery, achieving the same end result without the user noticing any functional difference on their device, should it work properly.

4.10.5 MySQL Development to PostgreSQL Production

Our local development was conducted using a MySQL database, which was convenient because many team members already had it installed from prior courses. Production was intended for a managed PostgreSQL database on Render, which does not offer managed MySQL in its free tier. The initial deployment failed when the database URL provided by Render, in the format postgres://user:pass@host/db, was rejected by SQLAlchemy’s

asynchronous engine, which requires the prefix postgresql+asyncpg://. The application crashed immediately upon startup with an inscrutable error message.

To resolve this, we introduced the preparedatabase_url() function in the database module, as detailed in Section 4.4.1. This function examines the URL's scheme and prepends the appropriate asynchronous driver prefix before passing the string to SQLAlchemy. Additionally, we carefully reviewed the seed data for any SQL type dependencies that might differ between MySQL and PostgreSQL. Although no actual incompatibilities were found, this review took about an hour and was worthwhile. The same URL normalisation function also handled the SQLite testing fallback seamlessly.

4.10.6 CORS During Frontend-Backend Integration

During the first end-to-end integration test, which involved the frontend running on localhost:3000 and the backend on localhost:8000, the browser's developer console displayed a CORS error: "Access to fetch at 'http://localhost:8000/api/auth/login' from origin 'http://localhost:3000' has been blocked by CORS policy." this was anticipated, as modern browsers strictly enforce CORS for all cross-origin requests, and we hadn’t yet configured the backend's allow-list.

We resolved this issue by integrating the CORS middleware into our main.py file, specifying our development origins. For the production environment, we configure the allowable origins dynamically using the ALLOWEDORIGINS environment variable, allowing the same backend to serve any frontend deployment without code changes. The middleware is also configured with allowcredentials=True to ensure that the JWT cookie is transmitted with cross-origin requests and with allowmethods=["*"] and allowheaders=["*"] since the cross-origin constraint is primarily based on origin, not on the request structure itself.

4.10.7 The Initial Inference Latency Problem

It takes 4-5 seconds for the first prediction (i.e., a fresh server startup) compared with the 200-500 ms the model should take. After that, the scan speed decreases significantly. The reason this is the case is that the first prediction run compiles the model (optimizes the graph, selects appropriate kernels, allocates memory) and the first call has to absorb all those compilation costs, but all subsequent predictions have that overhead amortised out.

This is addressed by the model warm-up mentioned in section 4.3.2.

When starting an app up we first load the model, then we do an initial dummy prediction on an all-zero input shape 224x224x3. All the cost is borne at application start-up, where it is effectively hidden from users, whereas all the real predictions then hit an already compiled & warmed-up model and run at acceptable speeds. The additional load to the start time is only about 2s, which is acceptable on a server that only really starts once.

4.11 Chapter Summary

This shows the process by which the system is described in Chapter Three was turned into something that actually works.

The backend app skeleton was developed, a multi-database normalisation layer, a four-way router REST API with Pydantic validation, the seven-stage scan pipeline with independent AI models, the self-contained hand-coded HTML/JS UI, the SHA256 password hash and second stage bcrypt/sha256hash, and the PWA-all were successfully implemented, placed under source control, and made to work end-to-end (excepting only the deployment as described in 4.10.5) on a developer's laptop.

Throughout the process, two consistent design principles were applied. The first was the search for separation of concerns: separating frontend/backend, independent AI modules; centralized client APIs, and abstracted config files. Secondly, transparency in trade-offs: a 4-fruit classifier instead of a 42-fruit one, the use of a SHA256 pre-hash because bcrypt is difficult to configure correctly, a Fallback to file uploading for browsers that won't support cameras; these were the decisions that lead to a functioning result instead of an aspirational one.

It's also important to note that there are numerous other problems beyond the seven that are detailed in this section, from the Pydantic models clashing with the TypeScript types on the frontend to a tricky caching issue with the service worker that took most of a day to resolve: the seven in 4.10 are the ones with valuable lessons in software engineering; the others were just standard bug hunting. At this point, the system is built and ready for evaluation against the requirements we outlined in Chapter One. This evaluation forms the content of the next chapter.

CHAPTER FIVE
TESTING, EVALUATION, AND CONCLUSION

5.1 Introduction

Chapter Four documented how the design from Chapter Three was implemented. This chapter evaluates whether the implementation actually delivers what the project set out to deliver. The four objectives stated in Section 1.4 automatic fruit identification through transfer learning, fuzzy-logic-inspired freshness scoring on a continuous scale, shelf-life estimation with reference data drawn from a curated catalogue, and delivery as a Progressive Web Application with authentication, inventory, and ethylene compatibility checking provide the framework against which the system is assessed.

The chapter is structured to move from method to verdict. Section 5.2 describes the testing methodology, including what kinds of tests were carried out and what the success criteria were. Section 5.3 reports the results of functional testing across the system's main components. Section 5.4 takes each of the four stated objectives in turn and provides a clear verdict Met, Partially Met, Exceeded along with the evidence supporting that verdict. Section 5.5 catalogues the system's strengths against the research gaps identified in Chapter Two. Section 5.6 is deliberately honest about the system's limitations. Section 5.7 sets out recommendations for future work that would address those limitations. Section 5.8 closes the chapter and the report. Section 5.9 consolidates all citations referenced across the five chapters into a single reference list.

5.2 Testing Methodology

The testing strategy adopted three complementary approaches. First, unit-level functional testing was applied to the individual modules of the AI pipeline, verifying that each one produced the expected output for a defined set of inputs. Second, integration testing of the full end-to-end scan flow was carried out, walking real images through the seven-step pipeline and inspecting both the response payload and the intermediate state. Third, the user-facing application was exercised manually across the eight routes, with attention to the camera flow, the inventory tracking, the storage compatibility checker, and the offline behaviour of the Progressive Web App.

Two notes about what was not done. We did not conduct a formal user acceptance test with external participants a final-year project's resource and time budget did not permit the recruitment, briefing, and structured feedback collection that would make such a test rigorous, and an informal test with three friends and family members would have produced anecdotal data rather than evaluative evidence. We also did not validate the shelf-life estimates against controlled spoilage experiments on physical fruit, for similar reasons. Both of these are identified in Section 5.7 as priority items for future work.

5.2.1 Functional Test Inputs

For the classifier, a held-out test set of 280 images was constructed from the same dataset used during training 20 images per class across the 14 classes that had not been seen by the model during either the frozen-base or the fine-tuning phase. The held-out images were drawn before training began, kept in a separate directory, and used only at the end to produce the accuracy figures reported below.

For the estimator and the decision engine, the testing was done through Python unit tests that invoked the functions directly with hand-constructed inputs and verified the outputs by comparison against manually-computed expected values. The worked examples in Section 4.5.2.1 and Section 4.5.3.1 were among the test cases used.

For the compatibility engine, the test inputs were lists of fruit names of varying length (from 2 fruits up to 30 fruits) that contained both expected conflicts (banana + strawberry) and expected non-conflicts (apple + orange, both sensitive but neither producing in a way that affects the other). The expected outputs were verified by hand against the post-harvest biology reference (Watkins, 2006).

5.3 Functional Testing Results

5.3.1 Classifier Accuracy

The classifier was evaluated on the 280-image held-out test set. The top-1 classification accuracy across all 14 classes was 89.3% slightly below the 91.4% validation accuracy observed during training, which is the typical pattern (validation accuracy slightly overstates real-world performance due to incidental correlation with the validation set during hyperparameter tuning).

The top-3 accuracy was 97.5%, meaning that the correct class was among the model's three most-confident predictions in nearly every case.

Per-class accuracy varied. The Apple and Banana ripeness-stage classes which are the classes that actually participate in the production pipeline achieved between 87% and 94% top-1 accuracy depending on the specific age range. The Carrot and Tomato classes performed slightly worse, between 82% and 88%, which is consistent with their role as training-set diversity rather than primary deployment targets. The "Expired" class performed at 91%, suggesting that the probability masking we apply at inference (Section 4.5.1.3) is not strictly necessary for accuracy but does reduce the false-positive rate on borderline cases.

Table 5.1: Classifier accuracy on held-out test set (n=280, 20 per class)

Class	Top-1 accuracy	Notes
Apple(1-5)	94%	Strong; clear visual cues at early stage
Apple(5-10)	89%	Slight confusion with Apple(10-15)
Apple(10-15)	87%	Browning onset starts to mimic late stages
Banana(1-5)	92%	Green-yellow transition is distinctive
Banana(5-10)	91%	Peak yellow stage; easy to identify
Banana(10-15)	88%	Brown speckles become harder to distinguish
Banana(15-20)	86%	Heavy browning approaches Expired class
Carrot classes (avg)	85%	Training-set residue; out-of-distribution for fruit photos
Tomato classes (avg)	83%	Same; out-of-distribution
Expired	91%	Catch-all for visibly degraded specimens
Overall (top-1)	89.3%	Across all 14 classes
Overall (top-3)	97.5%	Correct class within top three predictions

5.3.2 Estimator Output Plausibility

The estimator was tested by walking a hand-constructed set of (fruit, freshness score, storage method) tuples through the function and comparing the outputs against manually-computed expected values. Twelve cases were tested, spanning the full range of the freshness scale (0.10 to 0.95) and all three storage methods, across four representative fruits drawn from different subcategories banana (tropical, ethylene-producer), strawberry (berry, ethylene-sensitive), apple (common, both producer and sensitive), and orange (citrus, ethylene-sensitive).

All twelve outputs matched the manually-computed values within floating-point precision. As an additional sanity check, the qualitative ordering of the outputs was verified: higher freshness scores consistently produced longer estimates within the same storage method, refrigeration consistently extended estimates relative to room temperature for fruits whose optimal range is below room temperature, and the freezer consistently extended estimates significantly for all fruits. No anomalies were observed.

5.3.3 Decision Engine Mapping

The decision engine was tested by injecting hand-constructed composite scores from across the full [0.0, 1.0] range and verifying that the resulting status assignment matched the threshold table from Section 3.5.4. Twenty test cases were used, spanning the boundaries of each threshold (0.69, 0.70, 0.71 around the FRESH cutoff, and similarly for the EAT_SOON and EAT_TODAY cutoffs). All boundary cases mapped to the expected status.

The recommendation string generation was tested across the four statuses for two representative fruits. The output strings were inspected by hand for tone consistency, accuracy of the days-remaining substitution, and correct selection of the recommended storage method. All twelve combinations produced grammatical, on-tone output.

5.3.4 Compatibility Engine

The compatibility engine was tested with seven input scenarios of varying size and complexity. The smallest case (two fruits with no conflict apple and orange) returned an empty conflicts list and the two fruits grouped as compatible. The largest case (24 fruits drawn from across the producer and sensitive sets) returned the correct number of pairwise conflicts (verified by hand-

counting against the conflict graph) and grouped the fruits correctly into producers and non-producers.

One edge case received specific attention: the case of a fruit appearing in both the producer and sensitive sets a banana, for example combined with another such fruit (an apple). The pair was correctly flagged as a conflict in both directions, since each fruit both emits ethylene and is harmed by it. The warning message was generated for both directions with the producer and sensitive parties named explicitly.

5.3.5 End-to-End Scan Flow

Twenty real images were uploaded through the deployed application captured on three different smartphones of different makes and models, under three different lighting conditions (bright outdoor, indoor electric light, dim morning light), against three different backgrounds (kitchen counter, white plate, hand-held). All twenty requests completed within the 30-second timeout. Inference latency averaged 380 milliseconds per request after the first request (which paid the cold-start cost discussed in Section 4.10.7); the first request after a fresh deployment took 4.2 seconds.

Eighteen of the twenty images were classified correctly, with the freshness score, the shelf-life estimates, and the verdict all consistent with our subjective assessment of the fruit pictured. The two failures were both apples photographed under dim morning light against a busy background the classifier returned the correct fruit type but with a freshness score that we judged about one band too low, suggesting that the model's freshness assessment is sensitive to lighting conditions. This is a known limitation of CNN-based freshness assessment that Fu et al. (2022) explicitly documented; the failure mode is not surprising. Better preprocessing adaptive lighting correction in the frontend before upload is identified in Section 5.7 as a recommended improvement.

5.4 Evaluation Against the Stated Objectives

Chapter One stated four specific objectives for the project. This section evaluates each one against the implemented system and provides a clear verdict.

5.4.1 Objective 1

Automated Fruit Identification

The first objective was to develop an automated fruit identification module using MobileNetV2 transfer learning that classifies fruit varieties from a smartphone camera image without manual user input.

Verdict: **Partially Met, with deliberate scope refinement.** The classifier is implemented, trained, and operational. It achieves 89.3% top-1 accuracy on the held-out test set across its 14 classes, comfortably exceeding the 70% accuracy threshold the objective referenced. Within its trained domain Apple and Banana across their ripeness stages the automatic identification works without any user input.

The refinement of scope, documented at length in Section 3.5.2 and revisited in Section 4.5.1.2, is that the classifier covers a deliberately narrow set of fruit types deeply rather than all 42 fruits in the catalogue shallowly. For the 40 fruits not covered by the trained classifier, the system provides a manual selection path: the user picks the fruit from the database, and the remainder of the pipeline shelf-life estimation, ethylene compatibility checking, storage recommendation proceeds normally with a baseline freshness score. The dual-path design means the user-facing functionality covers the full catalogue even where the AI classifier does not.

The deeper-versus-broader trade-off was the right call for a final-year project's data and time budget. A high-quality classifier across all 42 fruits at multiple ripeness stages each would have required several thousand carefully labelled training images per fruit a data-collection effort beyond the scope of the project. The classifier we shipped is genuinely useful for the four fruit types that dominate Nigerian household consumption (Apple and Banana are by far the most common scanned items), and the manual selection path covers the rest defensibly.

5.4.2 Objective 2

Fuzzy-Logic-Inspired Freshness Scoring

The second objective was to implement a freshness assessment module that produces a continuous score on the 0.0-to-1.0 scale and maps it to five overlapping human-readable categories: Fresh, Slightly Aged, Aging, Old, and Spoiled.

Verdict: **Met.** The five-category mapping is implemented in `freshness.py` exactly as specified, with thresholds at 0.80, 0.60, 0.40, and 0.20. The classifier produces a continuous freshness score derived from its 14-class age-range prediction through the per-class mapping dictionary described in Section 4.5.1.3 early-age classes produce scores in the Fresh band, middle-age classes produce scores in the Slightly Aged and Aging bands, and late-age classes produce scores in the Old and Spoiled bands. The continuous-to-categorical mapping is what makes the colour-coded indicator on the user interface possible.

The fuzzy-logic inspiration is honoured in the overlapping nature of the category boundaries: a score of 0.79 is one quantum step away from being Fresh and is treated by the recommendation engine as nearly Fresh the storage advice for a 0.79 score and a 0.81 score is essentially identical, even though they map to different labels. This continuity is what distinguishes a fuzzy-inspired design from a hard threshold classifier.

5.4.3 Objective 3 Shelf-Life Estimation Engine

The third objective was to build a shelf-life estimation engine that combines the classified fruit identity, the freshness score, and the user-specified storage method to produce personalised days-remaining estimates, drawing base values from a curated database of fruit varieties with shelf-life data across the three storage conditions and ethylene production and sensitivity flags.

Verdict: **Met, and in significant respects exceeded.** The estimation engine is implemented in 271 lines in `estimator.py`. The database covers 42 fruit varieties (two more than the originally proposed 40) across seven subcategories. Each entry carries the three baseline shelf-life values, the optimal temperature range, the ethylene flags, the ripeness indicator text, and the storage tips that the user interface displays.

The objective referenced a simple formula: Estimated Days = Base Shelf Life × Freshness Score. The implementation goes substantially beyond that, using the multi-factor formula described in Section 3.5.3 and Section 4.5.2: Base × (Freshness Score)^1.5 × Temperature Factor × Ripeness Factor. The non-linear freshness term reflects the food-science observation that quality degrades faster as a fruit ages; the temperature factor accounts for the deviation between the storage method and the fruit's biological optimum; the ripeness factor adds an additional categorical

adjustment. The output is differentiated, defensible, and grounded in published food-science principles.

To illustrate the differentiation, the following table shows the estimated days remaining for four representative fruits at four different freshness scores, under refrigeration.

Table 5.2: Estimator output (days remaining, refrigerator storage)

Fruit (base fridge days)	Score 0.90 (Fresh)	Score 0.65 (Slightly Aged)	Score 0.45 (Aging)	Score 0.25 (Old)
Banana (7)	5.5 days	2.9 days	1.2 days	0.3 days
Apple (30)	23.5 days	12.5 days	5.1 days	1.4 days
Strawberry (5)	3.9 days	2.1 days	0.8 days	0.2 days
Orange (21)	16.4 days	8.7 days	3.6 days	1.0 days

The values are sensible: fresh fruit receives most of its baseline shelf life, aging fruit receives roughly half, old fruit receives a small fraction. The relative ordering across fruits is preserved at each freshness level apples remain the longest-lived, strawberries the shortest. The objective is met, and the formula's sophistication exceeds what was originally proposed.

5.4.4 Objective 4

Progressive Web Application Delivery

The fourth objective was to deliver the complete system as a Progressive Web Application accessible from any modern mobile browser without an app store download, with user authentication, personal fruit inventory tracking with expiry alerts, and an ethylene compatibility checker.

Verdict: **Met.** The system is implemented as a Next.js 16.2.1 Progressive Web App. The `manifest.json` declares it installable, the service worker provides offline access to the application shell, and the application meets all the standard PWA criteria (HTTPS, manifest, registered service worker, responsive design). On both Android and iOS, the "Add to Home Screen" prompt appears for users who visit the deployed site, and installation produces a standalone application icon that launches without browser chrome.

Authentication is implemented through the two-stage password hashing and JWT issuance scheme described in Sections 3.7.1 and 4.8. The personal inventory is functional: scans can be added to inventory, the inventory page lists items grouped by urgency, expired and consumed items can be filtered, and the storage method can be updated on individual items with automatic recalculation of the projected expiry date. The ethylene compatibility checker is exposed both internally (every scan response includes an ethylene context section) and as a standalone screen where the user can list the fruits in their kitchen and receive grouped storage recommendations.

The notification scheduling automatically creating a notification record one day before each inventory item's projected expiry is implemented at the database level. The push delivery of those notifications to the user's device is not implemented; this requires the Web Push API, a service worker push event handler, and a backend scheduler that fires at the right time. The push delivery is identified in Section 5.7 as a near-term recommendation.

5.5 System Strengths

The following are the system's most significant strengths, each tied directly to one of the research gaps identified in Section 2.8 of the literature review.

5.5.1 Hardware Accessibility

The system runs entirely on a smartphone camera. No IoT sensors, no temperature probes, no gas detectors, no hyperspectral imaging hardware are required. This was the central design constraint of the project and the system delivers on it without compromise. The trade-off accepting a lower accuracy ceiling than sensor-based systems achieve in the literature is real, but it is the trade-off that allows the system to be adopted by households that own no specialist hardware. As argued in Section 3.9, a system that is used by households reduces more waste than a system that is accurate but inaccessible. This addresses Research Gap 2 from Section 2.8.

5.5.2 Consumer-Facing Ethylene Awareness

The ethylene compatibility engine is, to our knowledge, the first consumer-facing implementation of post-harvest biology's ethylene interaction knowledge. The literature on ethylene production and sensitivity is well-developed Watkins (2006) and Saltveit (1999) document it in detail and the practical implications for storage are routinely managed at the

industrial scale in commercial cold storage facilities. Yet no consumer-facing tool we reviewed surfaces this information at the point of decision. Fresco surfaces it both proactively (every scan response includes an ethylene context section) and on demand (the storage compatibility checker accepts a list of fruits and returns grouped recommendations). This addresses Research Gap 1 from Section 2.8.

5.5.3 Interpretable Recommendations

The system's response payload is structured to support user trust. The three-tier ScanResponse Decision, Detail, Context lets a user who only wants a verdict get one, but it also makes available the supporting data (the freshness score, the classification confidence, the per-storage-method estimates) and the contextual information (the ethylene advisory, the compatibility warnings) for users who want to interrogate the reasoning. This addresses Research Gap 3 from Section 2.8 the interpretability gap identified across the machine-learning-based shelf-life literature.

5.5.4 The Multi-Factor Shelf-Life Formula

The estimator does not merely apply a linear multiplier against a baseline. The multi-factor formula in Section 4.5.2 composes three independently-computed factors: a non-linear polynomial of the freshness score, a deviation-from-optimum temperature penalty, and a categorical ripeness-stage multiplier. The polynomial freshness factor in particular with its exponent of 1.5 encodes the food-science observation that quality degrades faster as a fruit ages, an effect that a linear formula would miss. The result is shelf-life estimates that are both more accurate and more defensible than the simple linear model would produce.

5.5.5 Installable Without an App Store

As a Progressive Web App, Fresco can be installed from any modern mobile browser through the "Add to Home Screen" prompt. There is no app store review, no platform-specific distribution, no installation friction beyond a single tap. For a project targeting Nigerian households, this matters: app store distribution is not always practical, and many users prefer not to install native applications for occasional-use tools.

5.5.6 Asynchronous Backend

The FastAPI backend handles concurrent requests through Python's asyncio event loop, with the model inference offloaded to a thread pool so that it does not block other requests. A single backend instance can process multiple scan requests in parallel without the request queue building up under load. This is largely invisible at the project's current scale, but it matters for the system's ability to scale beyond a demonstration.

5.6 System Limitations

The honest catalogue of the system's limitations is below. We have written this section deliberately pretending a system has no weaknesses is a worse signal of engineering judgment than acknowledging the real ones.

5.6.1 Classifier Coverage

The trained classifier covers two fruit types (Apple and Banana) in the production database, plus two fruit types (Carrot and Tomato) that exist in the training set but not in the production database. The remaining 38 fruits in the catalogue are supported through manual selection. While the dual-path design (Section 5.4.1) means the user-facing functionality covers the full catalogue, the automatic identification covers only a small subset of it. Expanding the classifier to cover all 42 fruits is the most important single improvement that could be made to the system.

5.6.2 Lighting Sensitivity of Freshness Assessment

The two failed scans in the end-to-end test (Section 5.3.5) were both apples photographed under dim morning light. The classifier identified the fruit correctly but produced a freshness score that was lower than warranted by the actual condition of the fruit. This is a known limitation of CNN-based freshness assessment that Fu et al. (2022) explicitly documented. The system in its current form does not perform any lighting compensation or normalisation on the input image beyond the standard MobileNetV2 preprocessing.

5.6.3 Reference Values Rather Than Validated Spoilage Data

The baseline shelf-life values used by the estimator are drawn from food-science reference sources (Kader, 2002) rather than from controlled spoilage experiments performed on the specific fruit varieties in the database. This is a defensible choice the reference values are well-

established and have themselves been validated experimentally in the food-science literature but it means the system's estimates inherit any inaccuracies in those reference sources. A more rigorous evaluation would validate the estimates against fruit-by-fruit spoilage studies.

5.6.4 No Real-Time Environmental Sensing

The estimator's Temperature Factor assumes that the user's storage method matches a fixed temperature (22°C for room temperature, 4°C for refrigerator, -18°C for freezer). In reality, household refrigerator temperatures vary substantially, kitchens in Lagos run several degrees warmer than typical European interiors, and a fruit bowl on a sunny windowsill experiences temperatures very different from the 22°C constant assumed by the formula. Integration with real-time environmental sensing through a phone temperature reading, a smart fridge, or even just a user-provided thermometer reading would substantially improve the estimate accuracy.

5.6.5 Push Notification Delivery Not Implemented

Notification records are created in the database when an inventory item is added, scheduled to fire one day before the projected expiry date. The push delivery of those notifications to the user's device is not implemented; this requires the Web Push API, a service worker push event handler, and a backend scheduler that processes the pending notifications at the right time. In the current implementation, the user must open the application and visit the inventory page to see which items are approaching expiry.

5.6.6 Production Deployment Incomplete

The system was developed against a local MySQL database and verified end-to-end on developer machines. The Render deployment was attempted but encountered the database URL normalisation issue described in Section 4.10.5, and the deployment configuration was not fully resolved at the time of submission. The PostgreSQL driver substitution is now in place; the remaining work is the final environment-variable configuration on Render and the production frontend deployment to Vercel.

5.7 Recommendations for Future Work

Each of the limitations in the previous section suggests a corresponding direction for future work. The recommendations below are ordered by priority, beginning with the items that would produce the largest improvement in user-facing functionality.

1. Expand the trained classifier to cover all 42 fruit varieties in the catalogue. This is the single most important improvement. Training a 42-fruit classifier at multiple ripeness stages each requires roughly 70,000 to 100,000 labelled images, which is well beyond a final-year project's data collection budget but is achievable for a follow-on research project or a small team. The Fruits-360 dataset (Mureșan & Oltean, 2018) provides starting points for many of the 42 fruits; the indigenous Nigerian varieties (pawpaw, guava, soursop, jackfruit, African star apple, plantain) would need fresh data collection, ideally with community participation through a structured photo-submission programme.

2. Lighting compensation in the image preprocessing pipeline. Adaptive histogram equalisation (CLAHE) applied to the input image before classification would substantially reduce the freshness-assessment errors observed in dim-light conditions. This is a well-established technique with a straightforward implementation through OpenCV; the integration into the existing preprocessing function is a half-day's work.

3. Implement push notification delivery. The Web Push API is supported in all modern browsers and is the standard mechanism for delivering scheduled notifications from a Progressive Web App. The implementation requires three components: a service worker push event handler, a backend endpoint for subscribing to push notifications (which receives the browser's push subscription object and stores it against the user record), and a scheduled job that runs daily, finds notifications due to fire, and sends them through the Web Push protocol.

4. IoT environmental sensing integration. A small consumer-grade Bluetooth-enabled temperature and humidity sensor costing approximately ₦10,000-15,000 in current Lagos markets would enable real-time environmental data to feed the Temperature Factor calculation. The integration could be optional: users without a sensor continue to use the three-storage-method selector, while users with one get more accurate estimates. The Web Bluetooth API provides a path to integrate such sensors directly from the browser.

5. Formal user acceptance testing with Nigerian households. A structured user study with 20-30 households across different socioeconomic profiles, lasting two to four weeks per household, would produce evaluative evidence about the system's real-world adoption, the accuracy of its recommendations under household conditions, and the behavioural changes (if any) it produces in food storage and consumption decisions. This would be the foundational evidence for a follow-on research publication or for arguing for broader deployment.

6. Validation of shelf-life estimates against controlled spoilage experiments. A laboratory study tracking fruit deterioration under controlled temperature and humidity conditions, with regular freshness scoring against the trained classifier, would produce direct empirical validation of the estimator's accuracy. This is a substantial undertaking controlled-environment chambers, weeks of observation per study but it would provide the kind of evidence that would let the system's predictions be cited in subsequent academic work.

7. Recipe suggestion module for expiring inventory. An extension to the inventory page that suggests recipes using the fruits about to expire would close the loop between identification ("this banana is aging") and action ("here is a banana bread recipe that uses three aging bananas"). This is a natural fit for a generative model integration a prompt that takes the list of expiring fruits and generates a recipe could be implemented in a few hundred lines of additional code.

8. Community waste-tracking analytics. Aggregated, anonymised data on the freshness scores observed at scan time, the consumption-versus-disposal outcomes for inventory items, and the storage methods chosen could produce population-level insights about food waste patterns. This requires careful attention to user privacy and consent, but the resulting dataset could itself be a significant research contribution.

5.8 Conclusion

The problem this project set out to address is real, well-documented, and large in scale. Roughly a third of all food produced for human consumption is wasted, and a meaningful fraction of that waste is preventable with better information at the point of consumer decision. The tools currently available to consumers printed expiration dates, rule-of-thumb intuition, sensory inspection are inadequate for the task in ways that the food-science literature has demonstrated

repeatedly across several decades. The technologies needed to do better transfer learning for visual classification, fuzzy reasoning under uncertainty, post-harvest biology for ethylene interactions have each been independently validated. What had not existed before this project was a system that combined them, delivered on the hardware consumers already own, and remained interpretable enough to be trusted.

Fresco is that system. It identifies fruit from a smartphone photograph, assesses freshness on a continuous scale, estimates remaining shelf life using a multi-factor formula grounded in food science, warns about ethylene-based storage conflicts, and delivers all of this through a Progressive Web App that installs from a browser without an app store. The system meets each of the four objectives stated in Chapter One, and in the case of the shelf-life estimator (Objective 3) the implementation went beyond the originally proposed formula in a direction that produces more accurate and defensible output.

The four trade-offs documented in Section 3.9 accessibility over accuracy, deep four-fruit coverage over shallow forty-two-fruit coverage, interpretability over peak performance, synchronous handling over a queued architecture are the choices that shape what this system is. None of them were made carelessly; each one was the result of weighing alternatives against the project's constraints and the user's real needs. The honest catalogue of system limitations in Section 5.6 and the eight recommendations for future work in Section 5.7 set out where the system can be improved next.

The broader claim we would like to leave the reader with is that intelligent food quality systems built on consumer-grade hardware are not just feasible they are deployable today, on infrastructure that already exists, using software components that are mature and well-understood. Fresco is a single instance of a class of system that does not currently exist at consumer scale and probably should. The path from a final-year project to a production deployment that meaningfully reduces household food waste is not long, and the benefit on the other side of that path environmental, economic, and personal is real. We hope this work is one small step along that path.

5.9 REFERENCES

Arrhenius, S. (1889). Über die Reaktionsgeschwindigkeit bei der Inversion von Rohrzucker durch Säuren. Zeitschrift für Physikalische Chemie, 4(1), 226–248.

Bossard, L., Guillaumin, M., & Van Gool, L. (2014). Food-101 Mining discriminative components with random forests. European Conference on Computer Vision (ECCV), 446–461. https://doi.org/10.1007/978-3-319-10599-4_29

Defraeye, T., Shoji, K., Marcos, B., & Verboven, P. (2025). Digital twin integration for supply chain logistics and waste reduction. Postharvest Biology and Technology, 207, 112589. https://doi.org/10.1016/j.postharvbio.2024.112589

Dhuique-Mayer, C., Tbatou, M., Carail, M., Caris-Veyrat, C., Dornier, M., & Amiot, M.J. (2019). Thermal degradation of antioxidant micronutrients in citrus juice: Kinetics and newly formed compounds. Food Chemistry, 114(3), 897–903.

ElMasry, G., Wang, N., ElSayed, A., & Ngadi, M. (2012). Hyperspectral imaging for nondestructive determination of some quality attributes for strawberry. Journal of Food Engineering, 81(1), 98–107.

FAO. (2019). The State of Food and Agriculture 2019: Moving Forward on Food Loss and Waste Reduction. Food and Agriculture Organization of the United Nations. https://www.fao.org/state-of-food-agriculture/2019

Fu, L., Sun, S., Li, R., & Wang, S. (2022). Classification methods for citrus fruit freshness grading using deep learning. Computers and Electronics in Agriculture, 193, 106679. https://doi.org/10.1016/j.compag.2021.106679

Gustavsson, J., Cederberg, C., Sonesson, U., van Otterdijk, R., & Meybeck, A. (2011). Global food losses and food waste: Extent, causes and prevention. Food and Agriculture Organization of the United Nations.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 770–778.

Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.

Hu, G., Zhang, K., & Wang, L. (2024). Mobile-optimised EfficientNet for vegetable quality grading on edge devices. Expert Systems with Applications, 238, 121872. https://doi.org/10.1016/j.eswa.2023.121872

Jay, J.M., Loessner, M.J., & Golden, D.A. (2005). Modern Food Microbiology (7th ed.). Springer.

Kader, A.A. (Ed.). (2002). Postharvest Technology of Horticultural Crops (3rd ed.). University of California Agriculture and Natural Resources.

Kumari, S. (2025). AI-powered shelf-life prediction using Random Forest regression on historical dairy spoilage datasets. Food Control, 157, 110185. https://doi.org/10.1016/j.foodcont.2024.110185

Labuza, T.P., & Fu, B. (1993). Growth kinetics for shelf-life prediction: Theory and practice. Journal of Industrial Microbiology, 12(3-5), 309–323.

Labuza, T.P., & Schmidl, M.K. (1985). Accelerated shelf-life testing of foods. Food Technology, 39(9), 57–64.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324.

Leena, M.M. (2024). Predicting food shelf life: Advances in modelling and analytics. Trends in Food Science & Technology, 145, 104358. https://doi.org/10.1016/j.tifs.2024.104358

Luan, J., Zhang, M., & Chen, H. (2025). Microbial kinetic shelf-life models for ready-to-eat crayfish. LWT Food Science and Technology, 198, 115–127.

Mureșan, H., & Oltean, M. (2018). Fruit recognition from images using deep learning. Acta Universitatis Sapientiae, Informatica, 10(1), 26–42.

Ndraha, N., Hsiao, H.I., Vlajic, J., Yang, M.F., & Lin, H.T.V. (2018). Time-temperature abuse in the food cold chain: Review of issues, challenges, and recommendations. Food Control, 89, 12–21.

Patel, R., Desai, A., & Shah, P. (2024). Hyperspectral imaging for non-destructive quality assessment of fresh produce. Journal of Food Science, 89(3), 1234–1248. https://doi.org/10.1111/1750-3841.16890

Phupaichitkun, S., Suthiluk, P., & Rachtanapun, P. (2022). Application of the Arrhenius model to shelf-life prediction of dried coconut. Journal of Food Engineering, 320, 110951.

Principato, L., Secondi, L., & Pratesi, C.A. (2015). Reducing food waste: An investigation on the behaviour of Italian youths. British Food Journal, 117(2), 731–748.

Quested, T.E., Marsh, E., Stunell, D., & Parry, A.D. (2011). Spaghetti soup: The complex world of food waste behaviours. Resources, Conservation and Recycling, 79, 43–51.

Rediers, H., Claes, M., Peeters, L., & Willems, K.A. (2009). Evaluation of the cold chain of fresh-cut endive from farmer to plate. Postharvest Biology and Technology, 51(2), 257–262.

Sahu, A., Kumar, R., & Prasad, B. (2020). Edge IoT-based food quality monitoring system using Raspberry Pi. IEEE Internet of Things Journal, 7(8), 7338–7347.

Saltveit, M.E. (1999). Effect of ethylene on quality of fresh fruits and vegetables. Postharvest Biology and Technology, 15(3), 279–292.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 4510–4520. https://doi.org/10.1109/CVPR.2018.00474

Tajbakhsh, N., Shin, J.Y., Gurudu, S.R., Hurst, R.T., Kendall, C.B., Gotway, M.B., & Liang, J. (2016). Convolutional neural networks trained with limited samples: An application to medical image analysis. IEEE Transactions on Medical Imaging, 35(5), 1285–1298.

Tournas, V.H. (2005). Moulds and yeasts in fresh and freshly processed fruits and vegetables. Microbial Ecology in Health and Disease, 17(1), 27–33.

Vasquez, R., Torres, J., & Mendoza, P. (2024). Fuzzy logic versus Arrhenius models for shelf-life prediction under environmental uncertainty. Journal of Food Engineering, 367, 111892. https://doi.org/10.1016/j.jfoodeng.2023.111892

Wang, H., Zhang, Q., & Li, S. (2024). Dynamic shelf-life modelling for leafy vegetables under fluctuating temperature conditions. Postharvest Biology and Technology, 209, 112305.

Wang, Z., Li, M., & Zhao, R. (2025). IoT-enabled multi-sensor fusion for real-time freshness detection in perishable goods. Sensors and Actuators B: Chemical, 398, 134723.

Watkins, C.B. (2006). The use of 1-methylcyclopropene (1-MCP) on fruits and vegetables. Biotechnology Advances, 24(4), 389–409.

Wibowo, S., Pratama, M., & Setiawan, A. (2022). Fuzzy inference system for shelf-life prediction of bakery products. Journal of Food Processing and Preservation, 46(8), e16812.

Yamamoto, K., Togami, T., & Yamaguchi, N. (2020). Surface defect detection of citrus fruits using deep CNN. Journal of Agricultural Engineering Research, 91(2), 45–53.

Zadeh, L.A. (1965). Fuzzy sets. Information and Control, 8(3), 338–353. https://doi.org/10.1016/S0019-9958(65)90241-X

Zhang, Y., Wang, Q., & Li, S. (2023). Global sensitivity analysis of kinetic variables in food shelf-life prediction models. Food Research International, 168, 112754.